

RTL Design Sherpa

Bridge Micro-Architecture Specification 1.1

October 15, 2026

Table of Contents

1 Bridge Mas Index....26

2 Document Information....27

2.1 Overview....27

2.1.1 Bridge Micro-Architecture Specification....27

2.1.2 Document Purpose....27

2.1.3 Intended Audience....27

2.2 Design Notes....28

2.2.1 Revision History....28

2.3 References....35

2.3.1 Related Documents....35

3 Overview....36

3.1 Overview....36

3.1.1 Bridge Micro-Architecture....36

3.1.2 Multi-Protocol Support....36

3.1.3 Channel-Specific Masters....36

3.1.4 Automatic Converters....36

3.2 Functional Description....38

3.2.1 Block Organization....38

3.2.2 Signal Flow....38

3.2.3 Figure 1.1: Bridge Signal Flow....38

3.3 References....39

3.3.1 Related Documentation....39

3.4 Navigation....40
3.4.1 Document Organization....40
4 2.1 Master Adapter....41
4.1 Overview....41
4.1.1 Purpose and Function....41
4.1.2 Figure 2.1: Master Adapter Architecture....42
4.2 Parameters....43
4.2.1 Per-Master Parameters (from TOML/CSV)....43
4.2.2 Global Parameters (affect all adapters)....43
4.3 Ports....44
4.3.1 External Master Interface (per master)....44
4.3.2 Internal Crossbar Interface (per master)....44
4.4 Functional Description....45
4.4.1 Channel Specialization....45
4.4.2 Per-Port Monitor Wrappers (when use_monitor = true)....46
4.4.3 Skid Buffer Architecture....46
4.4.4 Bridge ID Management....47
4.4.5 Protocol Normalization....48
4.4.6 Response Path Slave Selection Tracking....48
4.4.7 Several Targets in Flight (limitation fixed: GitHub #119 L1)....49
4.5 Timing....51
4.5.1 Latency....51
4.5.2 Throughput....51
4.5.3 Critical Paths....51

4.6 Design Notes..... 52
4.6.1 Resource Utilization..... 52
4.6.2 Future Enhancements..... 52
4.7 Related Modules..... 53
4.8 Testing..... 54
4.8.1 Debug and Observability..... 54
5 2.2 Slave Router..... 55
5.1 Overview..... 55
5.1.1 Purpose and Function..... 55
5.1.2 Figure 2.2: Slave Router Architecture..... 56
5.2 Parameters..... 57
5.2.1 Per-Router Parameters..... 57
5.2.2 Global Parameters..... 57
5.3 Functional Description..... 58
5.3.1 Address Decoding Algorithm..... 58
5.3.2 Request Routing..... 60
5.3.3 Out-of-Range Handling..... 61
5.3.4 Default Slave Support..... 62
5.3.5 Address Aliasing..... 63
5.4 Timing..... 64
5.4.1 Decode Latency..... 64
5.4.2 Throughput..... 64
5.4.3 Critical Paths..... 64
5.5 Design Notes..... 65

5.5.1 Resource Utilization..... 65
5.5.2 Performance Optimization..... 65
5.5.3 Future Enhancements..... 66
5.6 Related Modules..... 67
5.7 Testing..... 68
5.7.1 Debug and Observability..... 68
5.7.2 Verification Considerations..... 68
6 2.3 Crossbar Core..... 70
6.1 Overview..... 71
6.1.1 Figure 2.3: Crossbar Core Architecture..... 71
6.2 Parameters..... 72
6.3 Functional Description..... 73
6.3.1 Connectivity Matrix..... 73
6.3.2 Request Path Multiplexing..... 73
6.3.3 Response Path Demultiplexing..... 73
6.3.4 Per-Slave Request Arbitration..... 73
6.3.5 Request Multiplexers with Stable Address Gating..... 74
6.3.6 AW→W Burst Tracking FIFO..... 75
6.3.7 Backpressure Propagation..... 76
6.3.8 Bridge ID and Response Routing..... 76
6.3.9 Response Arbiters and Demultiplexers..... 76
6.3.10 AXI Ordering Requirements..... 77
6.3.11 Monitor Aggregation at Bridge Top (when variants includes mon)..... 78
6.4 Timing..... 79

6.4.1 Latency....79
6.4.2 Throughput....79
6.4.3 Critical Paths....79
6.5 Design Notes....81
6.5.1 Resource Utilization....81
6.5.2 Debug and Observability....81
6.5.3 Common Issues....82
6.5.4 Future Enhancements....82
6.6 Related Modules....83
6.7 Testing....84
6.7.1 Functional Tests....84
6.7.2 Corner Cases....84
6.7.3 Protocol Compliance....84
7 2.4 Arbitration....85
7.1 Overview....86
7.2 Parameters....87
7.3 Functional Description....88
7.3.1 Per-Slave, Per-Channel Arbiters....88
7.3.2 Block Diagram....88
7.3.3 Figure 2.4: Per-Slave Address Arbiter....88
7.3.4 Round-Robin Arbitration....89
7.3.5 Fixed-Priority Arbitration....90
7.3.6 Weighted Arbitration....91
7.3.7 Burst Handling....92

7.4 Timing....94
7.4.1 Arbitration Latency....94
7.4.2 Critical Paths....94
7.5 Design Notes....95
7.5.1 Resource Utilization....95
7.5.2 Performance Impact....95
7.5.3 Debug and Observability....96
7.5.4 Common Issues....96
7.5.5 Future Enhancements....97
7.6 Related Modules....98
7.7 Testing....99
7.7.1 Test Scenarios....99
7.7.2 Corner Cases....99
8 2.5 ID Management....100
8.1 Overview....101
8.1.1 Figure 2.5: ID Management Architecture....102
8.2 Parameters....103
8.3 Functional Description....104
8.3.1 Bridge ID Concept....104
8.3.2 ID Width Calculation....104
8.3.3 Multi-Master OR-Merge Gating....104
8.3.4 ID Injection (Request Path, built)....104
8.3.5 ID Stripping and Response Routing (built)....105
8.3.6 Content Addressable Memory: bridge_cam (built)....105

8.3.7 In-Order FIFO Tracker (built)....106
8.3.8 Outstanding Transaction Limits....106
8.4 Timing....107
8.5 Design Notes....108
8.5.1 Resource Utilization....108
8.5.2 Debug and Observability....108
8.5.3 Common Issues....108
8.6 Related Modules....109
8.7 Testing....110
9 2.6 Width Conversion....111
9.1 2.6.1 Purpose and Function....112
9.2 2.6.2 Internal Data Width....113
9.2.1 64-Bit Standard....113
9.2.2 Width Conversion Locations....113
9.3 2.6.3 Block Diagram....114
9.3.1 Figure 2.6: Width Conversion Architecture....114
9.4 2.6.4 Upsizing (Narrow to Wide)....115
9.4.1 Overview....115
9.4.2 Write Upsizing....115
9.4.3 Read Upsizing....115
9.4.4 Strobe Mapping (Upsizing)....116
9.5 2.6.5 Downsizing (Wide to Narrow)....117
9.5.1 Overview....117
9.5.2 Write Downsizing....117

9.5.3 Read Downsizing....117
9.5.4 Strobe Mapping (Downsizing)....118
9.6 2.6.6 Address Alignment....119
9.6.1 Address Adjustment for Width....119
9.6.2 Unaligned Access Handling....119
9.7 2.6.7 Burst Length Adjustment....120
9.7.1 Length Calculation....120
9.7.2 Odd Burst Lengths....120
9.8 2.6.8 Resource Utilization....121
9.8.1 Per-Converter Resources....121
9.8.2 Scaling....121
9.9 2.6.9 Timing Characteristics....122
9.9.1 Latency....122
9.9.2 Throughput....122
9.10 2.6.10 Configuration Parameters....123
9.10.1 Width Conversion Configuration (TOML)....123
9.11 2.6.11 Debug and Observability....124
9.11.1 Recommended Debug Signals....124
9.11.2 Common Issues and Debug....124
9.12 2.6.12 Verification Considerations....125
9.12.1 Test Scenarios....125
9.13 2.6.13 Performance Considerations....126
9.13.1 Conversion Overhead....126
9.13.2 Optimization Strategies....126

9.14 2.6.14 Future Enhancements....127
9.14.1 Planned Features....127
9.14.2 Under Consideration....127
10 2.7 Protocol Conversion....128
10.1 2.7.1 Purpose and Function....129
10.2 2.7.2 Supported Protocols....130
10.2.1 Current Support (Phase 2)....130
10.2.2 Current Limitation: AXI4-Lite Conversion....130
10.2.3 Future Support (Phase 2+)....131
10.3 2.7.3 Conversion Architecture Overview....132
10.3.1 Figure 2.7: Protocol Conversion Architecture....132
10.4 2.7.4 Block Diagram....133
10.4.1 Figure 2.8: AXI4 to APB Shim....133
10.5 2.7.5 AXI4-Lite to AXI4 Conversion (Master-Side)....134
10.5.1 AXI4-Lite Protocol Overview....134
10.5.2 Conversion Requirements....134
10.5.3 Signal Mapping....134
10.5.4 Implementation....136
10.5.5 Resource Utilization....138
10.5.6 Performance Impact....138
10.5.7 Configuration....139
10.5.8 Common Issues and Debug....139
10.6 2.7.6 AXI4 to APB Conversion (Slave-Side)....140
10.6.1 APB Protocol Overview....140

10.6.2 APB Signals....140
10.6.3 APB State Machine....140
10.6.4 Read Transaction Conversion....141
10.6.5 Write Transaction Conversion....141
10.6.6 Burst Handling....142
10.6.7 Response Mapping....142
10.7 2.7.7 Implementation....143
10.7.1 AXI4-to-APB Converter FSM....143
10.7.2 Address Generation....144
10.8 2.7.8 Resource Utilization....145
10.8.1 APB Converter Resources....145
10.8.2 Scaling....145
10.9 2.7.9 Timing Characteristics....146
10.9.1 Latency....146
10.9.2 Throughput....146
10.10 2.7.10 Configuration Parameters....147
10.10.1 Protocol Conversion Configuration (TOML)....147
10.11 2.7.11 Debug and Observability....148
10.11.1 Recommended Debug Signals....148
10.11.2 Common Issues and Debug....148
10.12 2.7.12 Verification Considerations....149
10.12.1 Test Scenarios....149
10.13 2.7.13 Performance Considerations....150
10.13.1 When to Use APB....150

10.13.2 APB vs. AXI4 Comparison....150
10.14 2.7.14 Mixed Protocol Bridges....151
10.14.1 Example Configuration....151
10.14.2 Routing Optimization....151
10.15 2.7.15 Master-Side vs Slave-Side Conversion....152
10.15.1 Comparison....152
10.15.2 When to Use Each....152
10.16 2.7.16 Generator-Emitted Conversion Shims....153
10.16.1 AXI4 to AXI4-Lite Conversion....153
10.16.2 AXI4 to APB Conversion....153
10.16.3 Master-Side APB Front End....153
10.16.4 Wishbone B4 at Either Boundary....154
10.16.5 AXI3 Boundary Conversion....154
10.16.6 Master-Side AXI5-Lite Sideband....155
10.16.7 Master-Side AXIL→Wider-Slave Alignment....155
10.17 2.7.17 Future Protocol Support....157
10.17.1 Planned Features....157
10.17.2 Under Consideration....157
10.18 2.7.18 Best Practices....158
10.18.1 Design Recommendations....158
10.18.2 Performance Tips....158
11 2.8 Response Routing....159
11.1 2.8.1 Purpose and Function....160
11.2 2.8.2 Block Diagram....161

11.2.1 Figure 2.9: Response Routing Architecture....161
11.3 2.8.3 Stable Response Routing via FIFO-Tracked Slave Select....162
11.3.1 Architecture: FIFO-Tracked Master Routing....162
11.3.2 BID Extraction....162
11.3.3 CAM-Based Extraction....163
11.4 2.8.4 Response Demultiplexing....164
11.4.1 Per-Master Response Paths....164
11.4.2 Demux Implementation....164
11.5 2.8.5 Multi-Slave Response Merging....165
11.5.1 Problem Statement....165
11.5.2 Response Arbitration....165
11.5.3 Response Buffering....165
11.6 2.8.6 Independent Write-Tracker FIFO (W-Channel Deadlock Fix)....167
11.7 2.8.7 Backpressure Management....168
11.7.1 Ready Signal Routing....168
11.7.2 Stall Conditions....168
11.8 2.8.8 Response Path Latency....169
11.8.1 End-to-End R Channel....169
11.8.2 End-to-End B Channel....169
11.9 2.8.9 Error Response Routing....170
11.9.1 Slave Error Responses....170
11.9.2 Out-of-Range Error Responses....170
11.10 2.8.10 Resource Utilization....171
11.10.1 Response Router Resources....171

11.10.2 Scaling....171
11.11 2.8.11 Configuration Parameters....172
11.11.1 Response Routing Configuration (TOML)....172
11.12 2.8.12 Debug and Observability....173
11.12.1 Recommended Debug Signals....173
11.12.2 Performance Counters....173
11.13 2.8.13 Common Issues and Debug....174
11.14 2.8.14 Verification Considerations....175
11.14.1 Test Scenarios....175
11.15 2.8.15 Performance Optimization....176
11.15.1 Techniques....176
11.16 2.8.16 Advanced Features....177
11.16.1 Response Reordering (Future)....177
11.16.2 Response Coalescing (Future)....177
11.16.3 Response QoS (Future)....177
11.17 2.8.17 Timing Considerations....178
11.17.1 Critical Paths....178
11.17.2 Optimization Strategies....178
11.18 2.8.18 Future Enhancements....179
11.18.1 Planned Features....179
11.18.2 Under Consideration....179
12 2.9 Error Handling....180
12.1 2.9.1 Purpose and Function....181
12.2 2.9.2 Error Categories....182

12.2.1 Transaction Errors....182
12.2.2 System Errors....182
12.3 2.9.3 Block Diagram....184
12.3.1 Figure 2.10: Error Handling Architecture....184
12.4 2.9.4 Out-of-Range Address Handling....185
12.4.1 Detection....185
12.4.2 Error Response Generation....185
12.4.3 Configurable Data Patterns....186
12.5 2.9.5 Protocol Violation Handling....187
12.5.1 Common Violations....187
12.5.2 Protocol Checker Implementation....187
12.6 2.9.6 Timeout Detection....189
12.6.1 Per-Transaction Watchdog....189
12.6.2 Timeout Configuration....190
12.7 2.9.7 Error Logging and Reporting....191
12.7.1 Error Status Register....191
12.7.2 Error History Buffer....191
12.7.3 Error Interrupts....192
12.8 2.9.8 Resource Utilization....193
12.8.1 Error Handling Resources....193
12.9 2.9.9 Configuration Parameters....194
12.9.1 Error Handling Configuration (TOML)....194
12.10 2.9.10 Debug and Observability....195
12.10.1 Recommended Debug Signals....195

12.11 2.9.11 Common Issues and Debug....196
12.12 2.9.12 Verification Considerations....197
12.12.1 Test Scenarios....197
12.13 2.9.13 Recovery Mechanisms....198
12.13.1 Automatic Recovery....198
12.13.2 Manual Recovery....198
12.14 2.9.14 Safety and Reliability....199
12.14.1 Error Containment....199
12.14.2 Fail-Safe Behavior....199
12.14.3 Error Injection (Debug/Test)....199
12.15 2.9.15 Future Enhancements....200
12.15.1 Planned Features....200
12.15.2 Under Consideration....200
13 AMBA5 Boundary and Native Sideband....201
13.1 Boundary Wrappers....202
13.2 Native Sideband Through the Structs (A5-2)....203
13.2.1 AWAKEUP (A14.1.1)....203
13.2.2 Data-scaled fields: Memory Tagging and chunking (bridge TASK-007, was BRIDGE-018)....204
13.3 Connectivity Gating (poison, atomic, mte) and Droppable Chunking....205
13.4 Atomic Filter (A5-3a, write-only masters)....206
13.4.1 Figure 2.11: Atomic Filter on a Write-Only Port....206
13.5 Read-Return Atomics (A5-3b, rw masters)....208
13.5.1 Figure 2.12: Read-Return Atomics....208

13.6 APB5 Slaves (A5-3c)..... 210
13.7 Lite and APB Requesters (bridge TASK-004, was BRIDGE-014)..... 211
13.8 Verification Anchors..... 212
14 Bridge Arbiter Finite State Machines..... 214
14.1 Overview..... 214
14.2 AW Channel Arbiter FSM..... 215
14.2.1 Figure 3.1: AW Arbiter FSM..... 215
14.3 AR Channel Arbiter FSM..... 217
14.3.1 Figure 3.2: AR Arbiter FSM..... 217
14.4 Round-Robin Arbitration Algorithm..... 219
14.5 Grant Locking Mechanism..... 220
14.6 Independent Read/Write Arbitration..... 222
14.7 FSM Instance Breakdown by Configuration..... 223
14.8 Performance Characteristics..... 224
14.9 Comparison with Other Crossbar Arbiters..... 225
14.10 Future Enhancements (Phase 3+)..... 226
14.11 Related Documentation..... 227
15 Transaction Tracking FSMs..... 228
15.1 Overview..... 228
15.2 Write Data Tracking (FIFO, not an FSM)..... 229
15.2.1 Purpose..... 229
15.2.2 Implementation..... 229
15.2.3 Figure 3.3: W Data Tracking FIFOs..... 230
15.3 Response Pending Counter..... 231

15.3.1 Purpose....231
15.3.2 Implementation....231
15.4 Burst Counter FSM....232
15.4.1 Purpose....232
15.4.2 States....232
15.4.3 Implementation....232
15.5 Related Documentation....233
16 CAM Architecture....234
16.1 Overview....234
16.2 Functional Description....235
16.2.1 What the CAM Does....235
16.2.2 Figure 4.1: CAM Entry Format....235
16.2.3 CAM Sizing....235
16.2.4 Insertion (AW / AR Acceptance)....236
16.2.5 Lookup and Deletion (B / last R)....236
16.2.6 Duplicate IDs: the Ordering Counter (Mode 2)....236
16.2.7 Implementation....237
16.2.8 CAM Full....237
16.3 Related Modules....238
17 Response Tracking....239
17.1 Overview....239
17.2 Functional Description....240
17.2.1 Figure 4.2: ID Table Structure....240
17.2.2 Sizing....241

17.2.3 ID Extension (Master Adapter)....241
17.2.4 ID Stripping and Routing....241
17.2.5 Tracker Operations....241
17.2.6 Multi-ID Considerations....242
17.3 Design Notes....243
17.3.1 Resource Utilization....243
17.4 Related Modules....244
18 Width Converters....245
18.1 Overview....245
18.2 Functional Description....246
18.2.1 Upsize Converter....246
18.2.2 Figure 5.1: Upsize Converter (64-bit to 512-bit)....246
18.2.3 Implementation....246
18.2.4 Burst Length Conversion....247
18.2.5 Downsize Converter....248
18.2.6 Figure 5.2: Downsize Converter (512-bit to 64-bit)....248
18.2.7 Implementation....248
18.2.8 Strobe Handling....250
18.2.9 AXIL-to-Wider-Slave Master-Side Alignment Converters....250
18.3 Design Notes....252
18.3.1 Resource Utilization....252
18.4 Related Modules....253
19 APB Converters....254
19.1 Overview....254

19.2 Functional Description....255
19.2.1 Conversion Requirements....255
19.2.2 Write Conversion....256
19.2.3 Figure 5.3: AXI4 Write to APB Write....256
19.2.4 Read Conversion....257
19.2.5 Figure 5.4: AXI4 Read to APB Read....257
19.2.6 Burst Handling....258
19.2.7 Error Handling....259
19.3 Timing....260
19.3.1 Throughput Impact....260
19.3.2 Latency Impact....260
19.4 Design Notes....261
19.4.1 Resource Utilization....261
19.5 Related Modules....262
20 Module Structure....263
20.1 Overview....263
20.2 Parameters....264
20.2.1 Compile-Time Parameters....264
20.2.2 Per-Port Parameters....264
20.3 Ports....265
20.3.1 Module Declaration....265
20.3.2 Port Organization....265
20.3.3 Master-Side Signals (External)....265
20.3.4 Slave-Side Signals (External)....266

20.4 Functional Description....267
20.4.1 Component Instantiation....267
20.4.2 Internal Signals....268
20.4.3 Reset Style and the Emitted Filelist....269
20.4.4 Generated Variants....269
20.4.5 Generated File Structure....270
20.5 Related Modules....271
21 Signal Naming....272
21.1 Overview....272
21.2 Ports....273
21.2.1 External Port Naming....273
21.2.2 Master Port Examples....273
21.2.3 Slave Port Examples....273
21.2.4 APB Port Naming....274
21.3 Functional Description....275
21.3.1 Internal Signal Naming....275
21.3.2 Debug Signal Naming....275
21.3.3 Verification Pattern Matching....276
21.4 Related Modules....277
22 Test Strategy....278
22.1 Overview....278
22.2 Related Modules....279
22.3 Testing....280
22.3.1 Test Categories....280

22.3.2 Test Parameterization....281
22.3.3 Protocol-BFM-Only Testing with Memory-Backed Slaves....286
22.3.4 Coverage Goals....289
22.3.5 Test Execution....289
23 Debug Guide....292
23.1 Overview....292
23.2 Waveforms....293
23.2.1 Generating Waveforms....293
23.2.2 Key Signals to Observe....293
23.2.3 Timing Analysis....293
23.3 Related Modules....294
23.4 Testing....295
23.4.1 Debug Signal Access....295
23.4.2 Common Issues....296
23.4.3 Assertion Failures....298
23.4.4 Performance Debugging....298

List of Figures

Figure 1.1: Bridge Signal Flow 38
Figure 2.1: Master Adapter Architecture 42
Figure 2.2: Slave Router Architecture 55
Figure 2.3: Crossbar Core Architecture 71
Figure 2.4: Per-Slave Address Arbiter 88
Figure 2.5: ID Management Architecture 102
Figure 2.6: Width Conversion Architecture 114
Figure 2.7: Protocol Conversion Architecture 132
Figure 2.8: AXI4 to APB Shim 133
Figure 2.9: Response Routing Architecture 160
Figure 2.10: Error Handling Architecture 183
Figure 2.11: Atomic Filter on a Write-Only Port 205
Figure 2.12: Read-Return Atomics 207
Figure 3.1: AW Arbiter FSM 214
Figure 3.2: AR Arbiter FSM 216
Figure 3.3: W Data Tracking FIFOs 229
Figure 4.1: CAM Entry Format 234
Figure 4.2: ID Table Structure 239
Figure 5.1: Upsize Converter (64-bit to 512-bit) 245
Figure 5.2: Downsize Converter (512-bit to 64-bit) 247
Figure 5.3: AXI4 Write to APB Write 254
Figure 5.4: AXI4 Read to APB Read 256

List of Tables

Table 1: Document information....27
Table 2: Revision history....28
Table 3: Supported Protocols....36
Table 4: Channel-Specific Master Types....36
Table 5: Document Organization....40
Table 6: Bridge ID Management....47
Table 7: Address decoder responsibilities....59
Table 8: Bridge ID and Response Routing....76
Table 9: Parameters....103
Table 10: Content Addressable Memory: bridge_cam....105
Table 11: Timeout Configuration....189
Table 12: Error Status Register....190
Table 13: AW arbiter FSM transitions....214
Table 14: AR arbiter FSM transitions....216
Table 15: Arbiter comparison....224
Table 16: Transaction tracking states....231
Table 17: Slave response tracker selection....238
Table 18: Tracker operations by event....240
Table 19: Burst Length Conversion Examples....246
Table 20: AXI4 vs APB Protocol Comparison....254
Table 21: APB to AXI4 Error Mapping....257
Table 22: External Port Naming....272
Table 23: Test Parameter Matrix....280
Table 24: AMBA5 Fixtures....280
Table 25: Test Level Depth Profile....289
Table 26: Debug Signals That Exist....294

List of Waveforms

Waveform 7.1: Expected Transaction Timing.....292

1 Bridge Mas Index

Generated: 2026-10-15

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

2.1 Overview

2.1.1 Bridge Micro-Architecture Specification

Document information

Property	Value
Document Title	Bridge Micro-Architecture Specification
Version	1.1
Date	2026-10-15
Status	Released
Classification	Open Source - MIT License

2.1.2 Document Purpose

This Micro-Architecture Specification (MAS) documents how the Bridge component is built:

- Block-level architecture and internal structure
- FSM state diagrams and transition tables
- ID management (and the CAM architecture that was never built)
- Signal-level interface details
- Width and protocol converter implementation
- Timing diagrams and debug information

Yes, the CAM gets called out in its own bullet — it was documented, never built, and the pages say so rather than letting you go looking for it in the RTL. For high-level architecture, system integration, and performance overview, refer to the companion **Bridge Hardware Architecture Specification (HAS)**.

2.1.3 Intended Audience

- RTL designers implementing or modifying Bridge
- Verification engineers developing testbenches
- System integrators debugging Bridge behavior
- Technical staff requiring implementation details

2.2 Design Notes

2.2.1 Revision History

Revision history

Version	Date	Author	Description
1.0	2026-01-03	RTL Design Sherpa	Initial release - restructured from single spec
1.1	2026-06-04	RTL Design Sherpa	Content sync to RTL state at 2026-06-04. Adds (1) monitor-aggregation discussion in the crossbar-core chapter — per-port `axi4_{master,slave}_{rd,wr}_mon` wrappers feed a `monbus_arbiter` tree into a single bridge-top `monbus_axil4_axil4_group`, packet+timestamp carried atomically through a 192-bit skid; references the canonical 128-bit packet spec at `docs/markdown/rtl-amba/includes/ocelus_package_spec.md`; (2) generator-emitted protocol-conversion shims

Version	Date	Author	Description
			at the slave boundary — `axi4_to_axil4_{rd,wr}` for AXIL slaves, chained with the AXIL→APB shim for APB slaves; (3) AXIL→wider-slave master-side alignment converters in the width-converters chapter (`axil_to_axi4_wide_align_{rd,wr}` — partial-word alignment, not protocol bridging); (4) generated-RTL chapter now lists monitor-variant module emission and per-port `(UNIT_ID, AGENT_ID)` assignment conventions; (5) verification-strategy chapter rewritten for protocol-BFM-only TBs with memory-backed slaves, the new `boundary_probe`

Version	Date	Author	Description
			test pattern, and the four `test_bridge_*_monitor_*.py` canonical patterns (smoke/capture/error_inject/irq); (6) response-routing chapter notes the independent W-tracker FIFO + split AW/W-ready MUX fix (commit `d24bd617`) that closed an interleaved-master deadlock.
1.2	2026-09-08	RTL Design Sherpa	Correctness and voice pass. Four external qc rounds (65/54/53/57 findings, all triaged against rtl/generated and the generator) removed the planned-but-never-built ID-table/CAM out-of-order scheme, the fixed 64-bit internal path, response arbitration/FIFOs and the AXI4-Lite-as-future-work

Version	Date	Author	Description
			status from the text; annotated the TOML keys the loader has never known and its three-tier unknown-key behaviour; replaced asserted latency figures with measured ones. Four RTL defects found by the rounds are fixed and mutation-checked (bridge BUG-009 (was BRIDGE-011) response-FIFO overrun, axi4_subtractive_slave simultaneous AW+AR fault loss, axi5_atomic_filter duplicate B and B-before-WLAST). Prose then voice-passed with all correctness annotations preserved.
1.3	2026-09-11	RTL Design Sherpa	bridge TASK-004 (was BRIDGE-014): AXI5-Lite and APB/APB5 as master protocols

Version	Date	Author	Description
			(front-end converters); bridge TASK-005 (was BRIDGE-016) slave ID widening
1.4	2026-09-11	RTL Design Sherpa	bridge TASK-008 (was BRIDGE-019): Wishbone B4 on both sides of the fabric
1.5	2026-09-11	RTL Design Sherpa	bridge TASK-006 (was BRIDGE-017): measured throughput and latency, registered crossbar, QoS arbitration, CDC slave ports, out-of-context synthesis flow and measured resources, CAM ordering-count fix
1.6	2026-09-13	RTL Design Sherpa	bridge TASK-007 (was BRIDGE-018): native AXI5 fabric – Memory Tagging and read-data chunking through the structs
1.7	2026-09-13	RTL Design Sherpa	Document sync: one fabric clock plus CDC ports throughout, Wishbone best-effort stance, out-

Version	Date	Author	Description
			of-order chunk proof
2.0	2026-10-12	RTL Design Sherpa	Protocol-set correction: overview/summary bullets now state the full eight-protocol support (axi3/axi4/axi5/axil/axil5/apb/apb5/wb4 — per-port, slave or master side; ACE coherency NOT supported) verified against `bin/bridge_pkg/config_validator.py`. Version strings aligned across index/front-matter/renders.
1.1 (renumbered)	2026-10-15	RTL Design Sherpa	The bridge document set (GAS, HAS, MAS) is renumbered to one common rev, 1.1; rows above keep the numbers they were published under. Content: the one-target-per-master limitation is fixed (GitHub #119 L1) – per-ID target

Version	Date	Author	Description
			tracking (`[bridge].id_track_bits`), B/R response arbiters per master width path, the crossed-W hold; the generated TB carries its connectivity map and the generator emits the AXIL narrow-to-wide alignment test (GitHub #119 L8); per-port outstanding limits, `max_outstanding` (power of two 2..64, default 16), size every tracker – the fixed 16 removed (GitHub #119 L5).

2.3 References

2.3.1 Related Documents

- **Bridge HAS** - Hardware Architecture Specification (high-level)
- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI development guide

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 Overview

3.1 Overview

3.1.1 Bridge Micro-Architecture

Bridge is a multi-protocol AXI4 crossbar generator: it turns CSV/TOML configuration files into parameterized SystemVerilog RTL. This document describes what that RTL looks like on the inside.

3.1.2 Multi-Protocol Support

Bridge supports three AMBA protocols:

Supported Protocols

Protocol	Use Case	Conversion
AXI4 Full	High-bandwidth memory	Direct or width convert
AXI4-Lite	Register access	Protocol downgrade
APB	Low-speed peripherals	Full conversion

3.1.3 Channel-Specific Masters

Masters that only read or only write don't pay for the channels they never use:

Channel-Specific Master Types

Type	Channels	Signals	Use Case
Full (rw)	AW, W, B, AR, R	100%	CPU, config master
Write-only (wr)	AW, W, B	~60%	DMA write engine
Read-only (rd)	AR, R	~40%	DMA read engine

3.1.4 Automatic Converters

Bridge inserts converters automatically:

- **Width converters** - Handle data width mismatches
- **Protocol converters** - AXI4 to APB conversion

- **Per-path optimization** - Only where needed

You never instantiate these by hand; the generator looks at each master/slave pairing and drops in exactly the conversion that path needs.

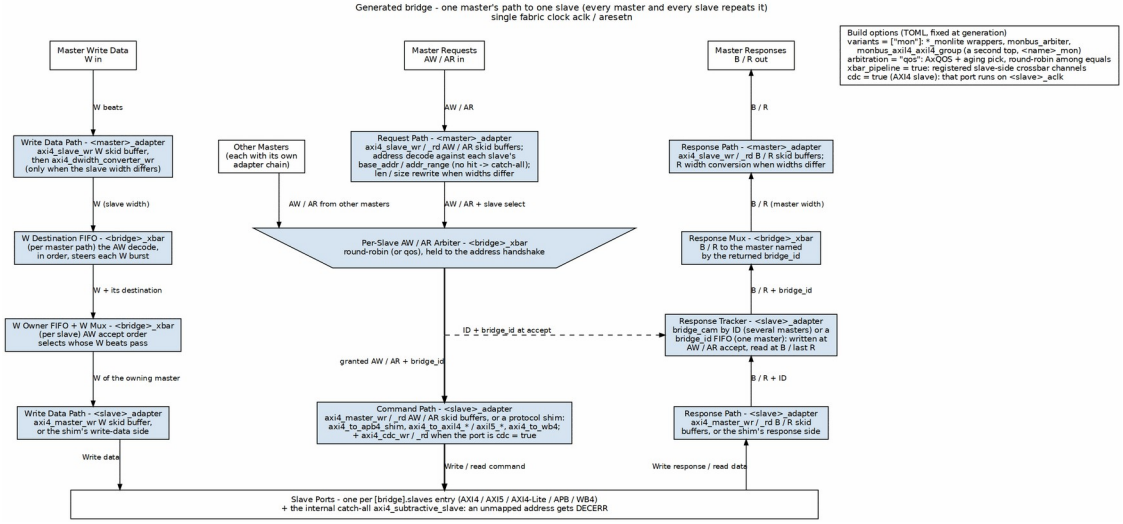
3.2 Functional Description

3.2.1 Block Organization

```
Bridge NxM
├── Master Adapters (M instances)
│   ├── Skid buffers, address decode, width paths
│   └── fabric ID = {BRIDGE_ID, id}; bridge_id sideband
├── Crossbar Core
│   ├── Address decode
│   ├── Per-slave arbiters
│   └── Channel muxes
├── Slave Adapters (N instances)
│   ├── Protocol shims (axi4_to_apb4_shim / axi4_to_axil4_*)
│   └── Response tracker: bridge_cam by ID (multi-master), or
bridge_id FIFO
├── Subtractive slave (always present, internal)
│   └── Answers unmapped addresses: DECERR + 0xDEADBEEF + sticky
status
└── Response Routing
    ├── the tracker's bridge_id selects the master
    └── Master response mux (the master's own id bits returned)
```

3.2.2 Signal Flow

3.2.3 Figure 1.1: Bridge Signal Flow



Bridge Signal Flow

One master's path to one slave; every master and every slave repeats it. Write data takes the left lane, AW / AR the center through the per-slave arbiter, and B / R climb the right lane back to the master named by the returned `bridge_id`.

3.3 References

3.3.1 Related Documentation

- **Bridge HAS** - High-level architecture and integration
- **PRD.md** - Product requirements
- **Generator source** - `bin/bridge_generator.py` (with `bin/bridge_pkg/`)

3.4 Navigation

3.4.1 Document Organization

Document Organization

Chapter	Content
Ch 2	Block Descriptions
Ch 3	FSM Design
Ch 4	ID Management
Ch 5	Converters
Ch 6	Generated RTL
Ch 7	Verification

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

4 2.1 Master Adapter

4.1 Overview

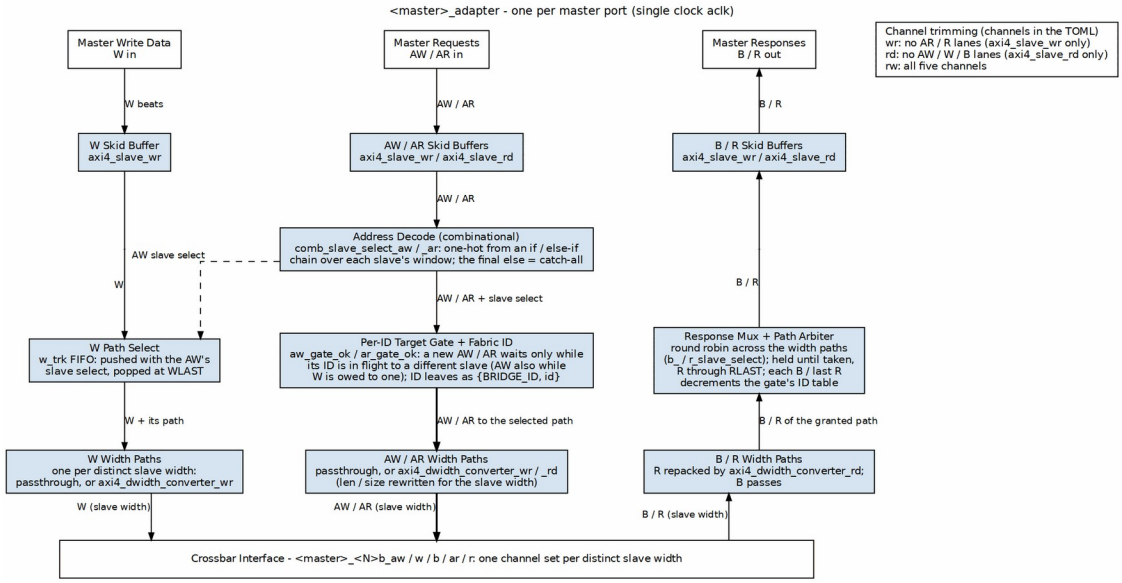
Every master port in the bridge gets its own Master Adapter. The adapter sits between the master and the crossbar core, and it exists to make the crossbar's life simple: timing is isolated at the boundary, channels are specialized to what the master actually uses, and every transaction carries a Bridge ID before it enters the fabric.

4.1.1 Purpose and Function

Concretely, the adapter does five things:

1. **Timing Isolation**: Inserts pipeline registers (skid buffers) to break combinatorial paths from master to crossbar
2. **Channel Specialization**: Separates read-only, write-only, and read-write masters for optimal resource utilization
3. **Fabric ID**: Widens the AXI ID to `{BRIDGE_ID, id}` so IDs are unique across masters
4. **Address Decode and Width Paths**: Selects the target slave and converts width on paths that need it (`axi4_dwidth_converter_wr` / `_rd`)
5. **Per-ID Target Gate and Response Steering**: lets a master keep requests in flight at several slaves (same-ID requests stay on one slave), and arbitrates B / R across its width paths

4.1.2 Figure 2.1: Master Adapter Architecture



Master Adapter Architecture

The figure follows the generated `<master>_adapter`: skid buffers at the port, the combinational address decode, the per-ID target gate that also widens the ID to `{BRIDGE_ID, id}`, and one width path per distinct slave width. W follows its AW through the `w_trk` FIFO, and B / R climb back through a response mux whose path arbiter picks among the width paths.

4.2 Parameters

Everything here comes straight out of the TOML/CSV configuration — the generator does the arithmetic, you just declare the ports.

4.2.1 Per-Master Parameters (from TOML/CSV)

```
[[bridge.masters]]
name = "cpu"
channels = "rw"              # "rd", "wr", or "rw"
arid_width = 4               # External ID width
awid_width = 4               # Can differ from ARID
addr_width = 32              # Address bus width
data_width = 64              # Data bus width
# pipeline_depth: NOT A KEY. Skid depth is set per channel by the
generator;
# the contract is 2..8 inclusive, so 1 is rejected.
```

4.2.2 Global Parameters (affect all adapters)

```
[bridge]
# internal_data_width: NOT A KEY -- there is no fixed internal
crossbar width.
# Width conversion and the ID prefix width are derived, not
configured:
# converters are emitted wherever a master and slave width differ, and
# ID_PREFIX_WIDTH = $clog2(num_masters) lands in <bridge>_pkg.sv.
```

4.3 Ports

Two interfaces per adapter: one facing the master, one facing the fabric. Note the ID widths change across that boundary — wider on the inside.

4.3.1 External Master Interface (per master)

```
Input Channels (from master):
- ARVALID, ARREADY, ARADDR, ARBURST, ARCACHE, ARID, ARLEN,
  ARLOCK, ARPROT, ARQOS, ARSIZE

- AWVALID, AWREADY, AWADDR, AWBURST, AWCACHE, AWID, AWLEN,
  AWLOCK, AWPROT, AWQOS, AWSIZE

- WVALID, WREADY, WDATA, WSTRB, WLAST

Output Channels (to master):
- RVALID, RREADY, RDATA, RID, RLAST, RRESP

- BVALID, BREADY, BID, BRESP
```

4.3.2 Internal Crossbar Interface (per master)

```
Output Channels (to crossbar):
- AR* signals + Internal ARID (wider)
- AW* signals + Internal AWID (wider)
- W* signals (unchanged)

Input Channels (from crossbar):
- R* signals + Internal RID (wider)
- B* signals + Internal BID (wider)
```

4.4 Functional Description

4.4.1 Channel Specialization

The Bridge generator emits one adapter per master, named after the port: `<name>_adapter.sv` in the bridge's `rtl/generated/<bridge>/` directory (for example `cpu_adapter.sv` in `bridge_2x2_rw`, `cpu_rd_adapter.sv` in `bridge_1x2_rd`, where the port itself is called `cpu_rd`). There is no separate file per channel type; the `channels` key decides which channel logic the one module contains:

4.4.1.1 Read-Only Masters

Channels: AR (Address Read), R (Read Data)

```
Configuration: channels = "rd"
RTL Generated: <name>_adapter.sv  (module <name>_adapter)

Optimizations:
- No write address channel logic
- No write data channel logic
- No write response channel logic
- Reduced arbitration participation (read path only)
```

4.4.1.2 Write-Only Masters

Channels: AW (Address Write), W (Write Data), B (Write Response)

```
Configuration: channels = "wr"
RTL Generated: <name>_adapter.sv  (module <name>_adapter)

Optimizations:
- No read address channel logic
- No read data channel logic
- Reduced arbitration participation (write path only)
```

4.4.1.3 Read-Write Masters

Channels: AR, R, AW, W, B (Full AXI4 interface)

```
Configuration: channels = "rw"
RTL Generated: <name>_adapter.sv  (module <name>_adapter)

Full Functionality:
- All five AXI4 channels implemented
- Participates in both read and write arbitration
- Maximum flexibility but larger resource footprint
```

4.4.2 Per-Port Monitor Wrappers (when `use_monitor = true`)

When the bridge TOML configuration sets `use_monitor = true` for a master port, the generator instantiates per-direction monitor wrappers immediately downstream of the master adapter:

- **`axi4_master_rd_mon`**: Wraps the AR/R channels, emits packets with `UNIT_ID=1`
- **`axi4_master_wr_mon`**: Wraps the AW/W/B channels, emits packets with `UNIT_ID=1`

Each wrapper: - Samples all channel signals and timestamp at handshake points - Emits 128-bit monitor packets on internal monbus - Passes signals through transparently in the common case, but it CAN stall: `axi4_master_rd_mon` gates the address channel on the monitor's back-pressure, `assign w_gated_arvalid = fub_axi_arvalid & (w_block_ready | ~cfg_monitor_enable);` so a saturated transaction table holds off new addresses until it drains. With `cfg_monitor_enable` low the gate is bypassed and the path really is transparent. - Is instantiated only when requested (generator-time configuration)

The generated per-port monbus streams are later aggregated by a tree of `monbus_arbiter` instances at the bridge top.

4.4.3 Skid Buffer Architecture

Each AXI4 channel passes through a skid buffer for timing isolation. The skid buffer implements:

4.4.3.1 Registered Forward Path

```
// Simplified skid buffer concept
always_ff @(posedge clk) begin
    if (!rst_n) begin
        valid_reg <= 1'b0;
    end else if (!valid_reg || ready_out) begin
        valid_reg <= valid_in;
        data_reg  <= data_in;
    end
end
```

4.4.3.2 Single-Cycle Backpressure Response

- When downstream is not ready, accepts one additional beat into holding register
- Signals ready_in = 0 in same cycle to upstream
- No combinatorial paths between upstream and downstream

4.4.3.3 Pipeline Depth

- Default: 2 (the `gaxi_skid_buffer` minimum)

- Range: **2..8 inclusive, any integer**. The value is the skid buffer's `DEPTH`, and its elaboration guard rejects anything outside that: `if (DEPTH < 2 || DEPTH > 8) $error(...)`. A depth of 1 does not elaborate, so the "default 1" this line used to claim was never buildable.
- Trade-off: Latency vs. timing closure

Performance Impact: - Adds 1 cycle latency per channel - Enables higher clock frequencies - Prevents critical paths through crossbar

4.4.4 Bridge ID Management

With more than one master, the adapter widens every request ID to a fabric ID `{BRIDGE_ID, id}` so two masters can never alias an ID at a slave (bridge TASK-005). The generated assignment in `cpu_adapter.sv` (`bridge_2x2_rw`) is:

```
assign xbar_axi_awid = {BRIDGE_ID_WIDTH'(BRIDGE_ID),
MASTER_ID_WIDTH'(fub_axi_awid)};
assign xbar_axi_arid = {BRIDGE_ID_WIDTH'(BRIDGE_ID),
MASTER_ID_WIDTH'(fub_axi_arid)};
```

`BRIDGE_ID` is a per-instance parameter (the master's index). The widths come from `<bridge>_pkg.sv`:

Bridge ID Management

Package constant	Meaning	`bridge_2x2_rw`
`MASTER_ID_WIDTH`	widest master-side ID	4
`ID_PREFIX_WIDTH`	master-index bits prepended, `$clog2(NUM_MASTERS)`	1
`XBAR_ID_WIDTH`	fabric and slave-port ID width	5

A single-master bridge prepends nothing (`ID_PREFIX_WIDTH = 0`; `bridge_1x2_rd` is 4 bits on both sides). A slave's declared `id_width` must be at least `XBAR_ID_WIDTH`, or the config validator rejects the bridge.

4.4.4.1 ID Width Examples

```
ID_PREFIX_WIDTH = $clog2(num_masters)   (0 for one master)

- 2 masters  -> 1 prefix bit
- 4 masters  -> 2 prefix bits
- 8 masters  -> 3 prefix bits
```

Example: 4-master system, 4-bit master IDs (6-bit fabric IDs)

```
Master 0: BRIDGE_ID = 2'b00, ID = 4'h3 -> fabric ID = 6'b00_0011
Master 1: BRIDGE_ID = 2'b01, ID = 4'h3 -> fabric ID = 6'b01_0011
Master 2: BRIDGE_ID = 2'b10, ID = 4'h5 -> fabric ID = 6'b10_0101
Master 3: BRIDGE_ID = 2'b11, ID = 4'hA -> fabric ID = 6'b11_1010
```

4.4.4.2 ID Stripping (Response Path)

On B and R the adapter returns only the low `MASTER_ID_WIDTH` bits to the master (`fub_axi_bid = cpu_32b_b.id[3:0]` in `bridge_2x2_rw`), so the master sees its original ID.

Routing a response back to the right master does not decode those prefix bits. Each slave adapter also receives the originating master's index as a sideband (`xbar_bridge_id_aw` / `_ar`) and stores it on the address handshake; the crossbar steers B/R by the `bid_bridge_id` / `rid_bridge_id` it returns. For an AXI4/AXI5 slave of a multi-master bridge that store is a `bridge_cam` looked up by the returned BID/RID; single-master bridges and shim/subtractive slaves use an in-order FIFO. See Section 2.5 and ch04.

4.4.5 Protocol Normalization

The Master Adapter enforces several protocol requirements:

4.4.5.1 Valid Address Ranges

- Checks addresses against slave address maps
- Flags out-of-range accesses for error handling
- Prevents deadlocks from illegal addresses

4.4.5.2 Burst Constraints

- Validates ARLEN/AWLEN vs. 4KB boundary rules
- Ensures burst types are supported (FIXED, INCR, WRAP)
- Checks SIZE vs. DATA_WIDTH compatibility

4.4.5.3 Signal Defaults

- Provides default values for unused signals
- Ensures ARCACHE/AWCACHE have legal values
- Sets ARPROT/AWPROT based on configuration

4.4.6 Response Path Slave Selection Tracking

4.4.6.1 Problem: Stale Slave Select Decode

Here's the part that bites people. The master adapter decodes the incoming address combinationally to pick a target slave. When the wrapper's skid buffer pops and `fub_axi_awaddr`/`fub_axi_araddr` reverts, the decode changes even though the converter is still in flight pushing the request to the crossbar. This causes the response MUX to route read/write responses from the wrong slave.

4.4.6.2 Solution: Per-Channel Response-Tracking FIFOs

Added separate FIFOs for AR and AW channels that capture the slave index (`comb_slave_select_ar/aw`) at the moment of the FUB-level handshake:

```
// At request time (when fub_axi_arvalid && fub_axi_arready)
ar_trk_push <= comb_slave_select_ar;  // Capture decoded slave index

// At response time (when m_axi_rvalid && m_axi_rready && m_axi_rlast)
r_slave_select <= ar_trk_pop;  // Stable slave index for response MUX
```

Benefits: - R/B responses route correctly even after address reverts - Supports multi-slave masters spanning different widths - Stable path for both data width and target slave

Implementation: - One FIFO per (master, AW/AR channel) - Width: `clog2(NUM_SLAVES)` bits (4 bits typical for 16-slave systems) - Depth: Matches maximum outstanding transactions (8-16 typical)

The FIFOs above steer responses ONLY on ports that carry a read-return atomic (`rr_atomic`) or a trace echo. Those ports hang per-request state on the FIFO entry, and they still hold one target slave at a time. Every other port uses the scheme below.

4.4.7 Several Targets in Flight (limitation fixed: GitHub #119 L1)

A master used to be held to one target slave at a time, because B / R were replayed in issue order from the tracking FIFO. Now the adapter keeps only the ordering AXI requires:

- **Per-ID table** (`aw_idt_cnt[]` / `aw_idt_tgt[]`, same for AR): `2**id_track_bits` entries (TOML `[bridge].id_track_bits`, default 4, and never more bits than the port's ID), indexed by the ID's low bits. Each entry holds an in-flight count and that ID's target slave. The address handshake increments the count; a B, or an R with RLAST, decrements it by the returned ID. IDs that share the index bits are treated as one ID: safe, at worst a needless stall.
- **Gates**: `ar_gate_ok = ar_id_ok && !ar_trk_full`, and `aw_gate_ok = aw_id_ok && !aw_trk_full && (w_trk empty || w_slave_select == target)`. Here `*_id_ok` means the ID is idle, or already in flight to the SAME slave. The W term exists because W has no ID and follows AW order. Two masters each owing W to two slaves would otherwise wait on each other at the slaves' W ports.
- **Outstanding limit**: `aw_out_cnt` / `ar_out_cnt` cap the port at its `max_outstanding` (default 16; bridge GitHub #119 L5) in flight (bridge BUG-009's not-full rule), so no count wraps.

- **Path arbiter**: with more than one width path, `b_pgnt` / `r_pgnt` pick a path round robin. A grant holds while offered and not taken; an R grant holds through RLAST. The selection feeds `b_slave_select` / `r_slave_select` as a representative slave of the path, so the response mux and the path readies are unchanged. One width path needs no arbiter. Among the slaves on one path, the crossbar's response arbiter has already chosen (MAS 2.3).

4.5 Timing

4.5.1 Latency

Request Path (Master → Crossbar): 1 cycle – one registered skid stage.

Response Path (Crossbar → Master): 1 cycle – one registered skid stage.

Total: 2 cycles each way through the bridge (one skid in the master adapter, one in the slave adapter). Measured, not derived: see HAS Table 5.7 and `test_bridge_2x2_rw_latency`, which asserts 2 exactly.

> **Depth is not latency.** This block read "with 4-stage pipeline: 4 cycles" and a round trip of "2-8 cycles depending on pipeline depth". A `gaxi_skid_buffer` presents its output one cycle after the write regardless of DEPTH; the depth sets how many beats it can ABSORB under backpressure, not how long a beat takes to cross it. The ID prefix and strip are pure wiring (a concatenation and a bit-select) and add no cycles.

4.5.2 Throughput

- **Maximum**: 1 transaction per cycle per channel (fully pipelined)
- **Backpressure**: Propagates with 1-cycle latency
- **Burst Performance**: No degradation; bursts flow continuously when ready

4.5.3 Critical Paths

Typical critical paths (if skid buffers not used): - Master ARVALID → Crossbar arbitration logic - Crossbar grant → Master ARREADY - Slave RDATA → Master RDATA

Solution: Skid buffers break all these paths at the cost of 1-cycle latency.

4.6 Design Notes

4.6.1 Resource Utilization

4.6.1.1 Per-Master Adapter Resources (Typical)

Read-Write Master (64-bit data, 32-bit addr, 4-bit ID):

```
Logic Elements:  ~200-400 (depending on ID width and features)
Registers:       ~150-250 (pipeline stages + control)
Block RAM:       0 (the master adapter holds no CAM; `bridge_cam`
lives in the slave adapters)

Breakdown:
- Skid buffers (5 channels × ~30 regs each): ~150 regs
- ID manipulation logic: ~50 LEs
- Control FSMs: ~100 LEs
- Routing logic: ~50 LEs
```

Read-Only Master: ~60% of read-write resources
Write-Only Master: ~65% of read-write resources

4.6.1.2 Scaling Considerations

Resource usage scales with: - Number of masters (linear scaling, one adapter per master) - ID width (logarithmic: +BID_WIDTH bits per transaction) - Pipeline depth (linear: deeper pipelines = more registers) - Data width (linear: wider data = wider skid buffers)

4.6.2 Future Enhancements

4.6.2.1 Planned Features

- **Dynamic Pipeline Depth**: Adjust depth based on operating frequency
- **FIFO Mode**: Deeper buffering (16-256 entries) for burst-intensive masters
- **QoS Support**: Priority-based arbitration hints
- **Performance Counters**: Transaction counts, stall cycles, utilization metrics

4.6.2.2 Under Consideration

- **Clock Domain Crossing**: Per-master clock domains with async FIFOs
- **Width Conversion at Adapter**: Move width logic to adapters for distributed conversion
- **Outstanding Transaction Tracking**: Windowing for improved OOO performance

4.7 Related Modules

- Section 2.3: Crossbar Core (interconnect architecture)
- Section 2.4: Arbitration (how adapters compete for slaves)
- Section 2.5: ID Management (fabric ID widening, sideband bridge_id, CAM/FIFO tracking)
- HAS ch04_interfaces/01_axi4_interface.md (port signals)

4.8 Testing

4.8.1 Debug and Observability

4.8.1.1 Recommended Debug Signals

For ILA or waveform capture:

```
- Master adapter input valid/ready (all channels)
- Master adapter output valid/ready (all channels)
- Bridge ID assignments per transaction
- Pipeline stage occupancy
- Backpressure events (ready = 0 while valid = 1)
```

4.8.1.2 Common Issues and Debug

Symptom: Master hangs waiting for READY
Check: - Is adapter receiving grants from arbiters? - Is slave responding to requests? - Is response path clear?

Symptom: Incorrect data returned to master
Check: - Bridge ID extraction logic - ID width mismatches - Response routing in crossbar

Symptom: Timing violations
Check: - Increase pipeline_depth parameter - Verify clock frequency vs. design complexity - Check for combinatorial loops

5 2.2 Slave Router

5.1 Overview

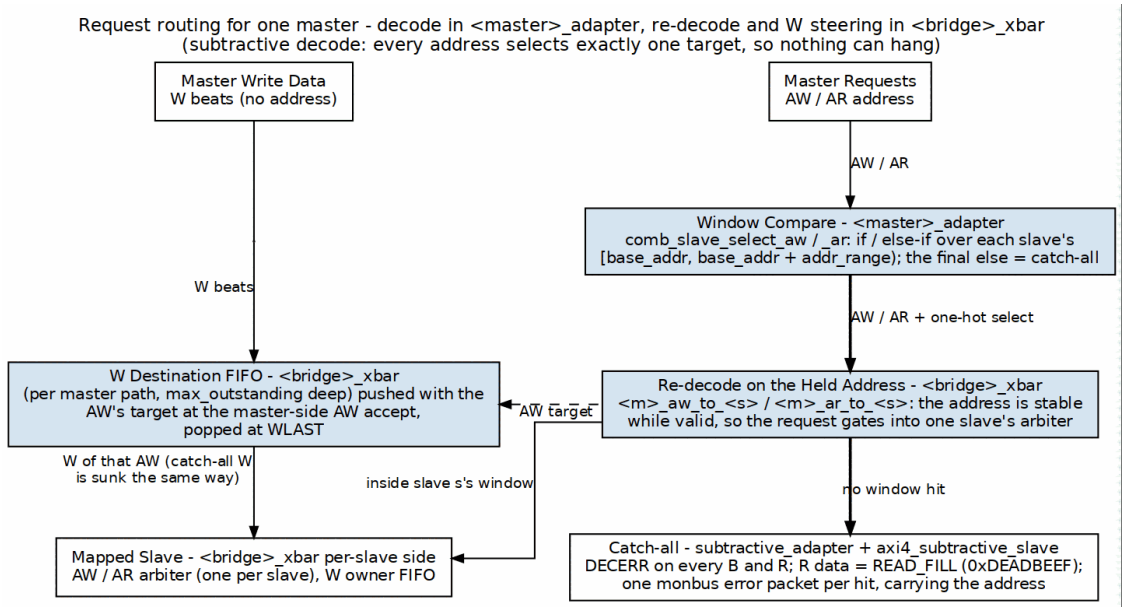
Every master gets its own Slave Router. The router examines each request address, steers the transaction to one of the configured slaves, and produces the error response itself when the address lands nowhere.

5.1.1 Purpose and Function

The router does five things:

1. **Address Decoding**: Matches request addresses against configured slave address ranges
2. **Request Routing**: Directs AR/AW/W channels to the selected slave
3. **Out-of-Range Detection**: Identifies addresses that don't map to any slave
4. **Error Response Generation**: Creates DECERR responses for invalid addresses
5. **Default Slave**: unmapped addresses go to an ALWAYS-PRESENT internal subtractive slave (`axi4_subtractive_slave`), which answers DECERR and returns `0xDEADBEEF` on reads. It is not optional and there is no `default = true` TOML key – the generator inserts it into every bridge.

5.1.2 Figure 2.2: Slave Router Architecture



Slave Router Architecture

There is no separate router module. The window compare lives in `<master>_adapter`; `<bridge>_xbar` re-decodes the held address to gate the request into one slave's arbiter and records each AW's target in a W destination FIFO, so W beats (which carry no address) follow their AW. An address in no window takes the final `else` of the decode chain to the always-present catch-all, which answers DECERR.

5.2 Parameters

5.2.1 Per-Router Parameters

```
# Router behavior is defined by slave configurations

[[bridge.slaves]]
name = "ddr_memory"
base_address = 0x8000_0000
size = 0x4000_0000           # 1 GB
default = false
# (no oor_data_pattern knob: READ_FILL is a module parameter,
0xDEADBEEF)
```

5.2.2 Global Parameters

```
[bridge]
enable_default_slave = false      # Allow default slave
strict_address_decode = true      # Flag overlapping ranges as errors
# (no oor_response_latency knob: the responder answers as fast as the
#  handshake allows; there is nothing to tune)
```

5.3 Functional Description

5.3.1 Address Decoding Algorithm

5.3.1.1 Configuration-Based Address Maps

Each slave is configured with:

```
[[bridge.slaves]]
name = "memory"
base_address = 0x4000_0000
size = 0x1000_0000        # 256 MB
default = false           # Not a default slave
```

The router generates address ranges:

```
Start Address = base_address
End Address   = base_address + size - 1
```

5.3.1.2 Decoding Priority

When multiple slaves have overlapping address ranges, the router uses **first-match priority**:

```
Priority Order = Order in configuration file (top to bottom)

Example:
  Slave 0: 0x0000_0000 - 0x0FFF_FFFF  (checked first)
  Slave 1: 0x1000_0000 - 0x1FFF_FFFF  (checked second)
  Slave 2: 0x2000_0000 - 0x2FFF_FFFF  (checked third)

Address 0x1000_5000 → Matches Slave 1
```

5.3.1.3 Range Checking Logic

For each address, the router performs:

```
// Simplified address decode logic
logic [NUM_SLAVES-1:0] slave_match;

for (int i = 0; i < NUM_SLAVES; i++) begin
    slave_match[i] = (addr >= SLAVE_BASE[i]) &&
                     (addr <= SLAVE_END[i]);
end

// Priority encode: Select first matching slave
logic [$clog2(NUM_SLAVES)-1:0] slave_select;
always_comb begin
    slave_select = 0;
    for (int i = 0; i < NUM_SLAVES; i++) begin
```

```
        if (slave_match[i]) begin
            slave_select = i;
            break;  // First match wins
        end
    end
end

// Out-of-range detection
logic oor = ~(|slave_match);  // No slaves matched
```

5.3.1.4 Subtractive decode: the `else` that removes the hang

The pseudocode above computes an out-of-range flag; the generated RTL goes further and gives that case a destination. The emitted decode chain ends in a bare `else` selecting an internal catch-all slave:

```
comb_slave_select_aw = '0;
if      (fub_axi_awaddr <= 32'h3FFFFFFF)  comb_slave_select_aw[0] =
1'b1;  // ddr
else if (fub_axi_awaddr >= 32'h40000000)  comb_slave_select_aw[1] =
1'b1;  // scratch
else                                      comb_slave_select_aw[3] =
1'b1;  // subtractive
```

So the one-hot is **never all-zero**. Before 2026-09-07 there was no `else`: an unmapped address selected nothing, the AW-ready MUX fell through to `default: // No slave selected` with `awready` low, and the master hung forever. The hang is now impossible by construction rather than handled.

5.3.1.5 Three decoders, not one

The catch-all is a synthetic slave appended last to the slave list, so it reuses the crossbar's routing rather than needing a new datapath. What made that harder than it sounds is that **three independent places derive the slave list for themselves**, and each needed telling:

Address decoder responsibilities

Where	What it derives	What the catch-all needed
master adapter	the if/else decode above	nothing – subtractive semantics come free from the `else`
crossbar	its own per-slave `_to_` terms, from address RANGES	the term must be `!(other ranges)`; a full-span slave otherwise reduces to `1'b1`
cfg regblock	per-slave configuration	exclusion – it has no

Where	What it derives	What the catch-all needed
	registers	monitor wrapper to configure

The crossbar case is the instructive one. Its decode is derived from each slave's address range independently of the adapter's chain, so a full-span catch-all became `wire cpu_32b_aw_to_subtractive = 1'b1;` – matching every address. Every transaction then routed to the real slave *and* the catch-all at once, and because the payload muxes OR their inputs, the master read back `addr = real | catchall`. Silent corruption, which is worse than the hang it replaced. The term is now the negation of the other ranges, **inlined** rather than referencing the sibling `<master>_<suffix>_<channel>_to_<slave>` wires: those are emitted per channel, so a slave with no read path from a given master has no `ar_to_` wire at all and naming it does not compile.

The cfg regblock case cost a whole debug cycle for a subtler reason: emitting per-slave cfg for the catch-all added 52 fields and **renumbered every register after them**, which moved the monitor-enable bits. One monbus stress test saw `pkts=0` while all 69 other tests passed, because only that test depended on register offsets. A register map is an ABI.

5.3.1.6 Power-of-Two Optimization

For slaves with power-of-two sizes starting at aligned addresses, simplified decode:

```
// Optimized decode for: base=0x8000_0000, size=0x1000_0000 (256MB)
// Mask off lower bits: Only check upper bits
logic match = (addr[31:28] == 4'h8);  // Much simpler than range check
```

The generator automatically detects and applies this optimization.

5.3.2 Request Routing

5.3.2.1 AR Channel Routing

Read address routing flow: 1. **Decode**: Determine target slave from ARADDR 2. **Check OOR**: If no slave matches, flag for error response 3. **Route**: Send AR transaction to selected slave's arbiter 4. **Track**: Remember routing decision for R channel responses

The decode and its out-of-range branch are the center column of Figure 2.2.

5.3.2.2 AW/W Channel Routing

Write transactions require coordinated routing:

1. **AW Phase**:
   - Decode AWADDR to determine target slave
   - Route AW transaction to selected slave's arbiter
   - **Store routing decision** for subsequent W beats

2. **W Phase**:
    - **Follow AW routing** (W channel has no address)
    - Route all W beats to same slave as AW
    - Continue until WLAST = 1

Figure 2.2 shows the W side: the W destination FIFO is pushed with the AW's target at the AW handshake and popped at WLAST.

5.3.2.3 Write Data Tracking (FIFO, no FSM)

There is no state machine on the W path. The crossbar keeps one W destination FIFO per master width-path (`<master>_<w>b_wdest_*` in `<bridge>_xbar.sv`, the master's `max_outstanding` entries, default 16):

```
Push:  on the AW handshake, write the encoded target slave
       (e.g. cpu_32b_wdest_enc: 0 = ddr, 1 = sram, 2 = subtractive)
Head:  while the FIFO is non-empty, its head selects which slave
       the master's W beats go to (cpu_32b_w_to_ddr, ..._to_sram, ...)
Pop:   on the W handshake that carries WLAST
```

Because the FIFO holds every accepted AW, a master can have several write bursts queued to different slaves; each W burst follows its own AW in order, with no IDLE/WRITING handshake between them. On the slave side a matching W owner FIFO (`<slave>_wowner_*`), pushed with the AW arbiter's grant and popped on WLAST, decides which master owns that slave's W channel. An out-of-range AW is routed to the subtractive slave like any other target, so its W beats are drained there and it answers DECERR (see below).

5.3.3 Out-of-Range Handling

5.3.3.1 Detection

An address is out-of-range if it matches no slave's range. The decode chain ends in an `else`, so such an address is not "detected and handled" – it is *routed*, to an internal subtractive slave that always answers.

5.3.3.2 Error Response Generation

The responder is `rtl/amba/axi4/axi4_subtractive_slave.sv`, instantiated by the generator as the last (internal) slave. It emits no top-level pins of its own; its behaviour is fixed, not configurable.

Read (AR -> R)

```
1. Accept ARVALID (ARREADY high while no read is active)
2. Return exactly ARLEN+1 beats:
     RID   = ARID as received (the fabric ID; the master adapter
strips the prefix)
     RDATA = 0xDEADBEEF, replicated to the data width
```

```
     RRESP = 2'b11 (DECERR)
     RLAST = 1 on the FINAL beat only
```

RLAST on every beat – which an earlier revision of this page specified – would make a burst master terminate early. The RTL asserts it as `r_r_active && (r_beats_left == 0)`, and the module test checks that it appears on the last beat and no other.

Write (AW/W -> B)

```
1. WREADY is unconditionally high -- W beats are sunk whether or not
AW has
   arrived yet. AXI4 permits write data before its address, and gating
WREADY
   on having seen AW deadlocks such a master.
2. One B per AW:
     BID   = AWID, unmodified
     BRESP = 2'b11 (DECERR)
3. Write data is discarded. There is nowhere for it to go.
```

Status. The first unmapped access latches a sticky flag with its address and a saturating count, raised on `unmapped_irq` and, on cfg-regblock builds, readable and clearable via `SUBTRACTIVE_STATUS` / `SUBTRACTIVE_ADDR`. See HAS 4.5.

Reachability. The catch-all is the decode `else`, so it can only fire if the slave ranges leave a gap. Of the 22 generated bridges, 18 tile the address space completely and the branch is unreachable logic that synthesis removes.

5.3.3.3 Read data pattern

`READ_FILL`, a module parameter, defaults to `32'hDEAD_BEEF` and is replicated to the bus width. It is not run-time configurable and there is no menu of options: an earlier revision of this page offered four (zeros, a debug signature, an address echo, a master/slave ID indicator), none of which was ever built.

The value matters more than the choice. Zeros are indistinguishable from real memory, so an all-zero error return reads as a plausible value and the fault stays hidden; `0xDEADBEEF` in a dump is unambiguous. Address-echo variants sound useful but the address is already captured in `SUBTRACTIVE_ADDR`, where software can read it without decoding it out of the data bus.

5.3.4 Default Slave Support

5.3.4.1 Configuration

```toml
[[bridge.slaves]]
name = "error_responder"
base_address = 0x0        # Ignored for default slave
size = 0x0                # Ignored for default slave
default = true            # Catch-all for unmapped addresses
```

5.3.4.2 Behavior

When a default slave is configured: - Addresses that don't match any specific slave → Routed to default slave - Default slave typically returns DECERR but with configurable response - Useful for prototyping (accept all addresses initially) - Can implement memory-mapped debug register for address capture

Note: Only ONE default slave allowed per bridge.

5.3.5 Address Aliasing

5.3.5.1 Multiple Slaves, Same Address

If configuration has overlapping ranges:

```toml
[[bridge.slaves]]
name = "fast_cache"
base_address = 0x8000_0000
size = 0x1000_0000

[[bridge.slaves]]
name = "slow_memory"
base_address = 0x8000_0000  # Same base!
size = 0x4000_0000
```

Result: First-match priority applies. All accesses go to `fast_cache`. The `slow_memory` range 0x9000_0000-0xBFFF_FFFF is **unreachable** from this master.

Warning: Generator can optionally flag this as error in DRC mode.

5.3.5.2 Intentional Aliasing Use-Cases

Legitimate uses of overlapping ranges: 1. **Cache hierarchy**: Small fast cache shadows larger slow memory 2. **Memory remapping**: Different views of same physical memory 3. **Peripheral mirroring**: Register block appears at multiple addresses

5.4 Timing

5.4.1 Decode Latency

Combinatorial Decode (Default): - Address → Slave selection: 0 cycles (combinatorial) - Critical path: ARADDR → Slave arbiter request - May limit maximum frequency in large systems

Registered Decode (Optional): - Address → Slave selection: 1 cycle (registered) - Adds latency but breaks critical path - Recommended for >8 slaves or >300 MHz operation

5.4.2 Throughput

- **Maximum**: 1 transaction per cycle per master
- **No blocking**: Router does not stall; arbiters handle conflicts
- **Pipelining**: AR and AW decode in parallel (independent paths)

5.4.3 Critical Paths

Typical critical paths: - ARADDR → Address comparators → Priority encoder → Arbiter request - w/4 slaves: ~10-15 logic levels (FPGA-dependent) - w/8 slaves: ~12-18 logic levels

Mitigation: - Enable registered decode mode (+1 cycle latency) - Use power-of-two slave sizes (simplified compare) - Synthesizer optimization directives

5.5 Design Notes

5.5.1 Resource Utilization

5.5.1.1 Per-Router Resources (Typical)

4-slave configuration (32-bit address):

```
Logic Elements:  ~150-200 LEs
Registers:       ~50 regs
Block RAM:       0

Breakdown:
- Address comparators (4 slaves × ~30 LEs): ~120 LEs
- Priority encoder: ~20 LEs
- W channel FSM: ~30 regs, ~20 LEs
- OOR error generator: ~20 regs, ~10 LEs
```

8-slave configuration:

```
Logic Elements:  ~250-350 LEs (scales roughly with slave count)
Registers:       ~60 regs
```

5.5.1.2 Scaling Considerations

Resource usage scales with: - **Number of slaves**: Linear (each slave adds comparator logic) - **Address width**: Minimal impact (wider comparators, but same structure) - **Optimizations**: Power-of-two sizes reduce logic significantly

5.5.2 Performance Optimization

1. Registered Decode (High Frequency)

```
Trade-off: +1 cycle latency for timing closure
Best for: >8 slaves, >300 MHz targets
```

2. Simplified Address Decode (Power-of-2)

```
Optimization: Mask-based compare instead of range check
Best for: Slaves with aligned, power-of-2 sizes
Savings: ~50% reduction in comparator logic
```

3. Parallel Decode (Low Logic)

```
Implementation: Separate decoders per address bit-field
Best for: Many non-overlapping slaves
Savings: Reduces logic depth for priority encoding
```

4. CAM-Based Decode (Many Slaves)

```
Trade-off: Uses block RAM for address lookup table
Best for: >16 slaves, non-contiguous ranges
Note: Requires RAM resources
```

5.5.3 Future Enhancements

5.5.3.1 Planned Features

- **Dynamic Address Remapping**: Runtime-configurable slave ranges
- **Transaction Filtering**: Block certain address ranges per-master
- **Priority Hints**: QoS-based prioritization (beyond first-match)
- **Address Translation**: Offset/mask transformations before slave routing

5.5.3.2 Under Consideration

- **Multi-region Slaves**: Slave spans multiple non-contiguous ranges
- **Secure Address Spaces**: Per-master access control lists
- **Debug Address Capture**: Log invalid addresses to register

5.6 Related Modules

- Section 2.1: Master Adapter (upstream from router)
- Section 2.3: Crossbar Core (downstream arbitration)
- Section 2.4: Arbitration (how routed requests compete)
- HAS ch04_interfaces/01_axi4_interface.md (slave port signals)

5.7 Testing

5.7.1 Debug and Observability

5.7.1.1 Recommended Debug Signals

```
- Address decode outputs (which slave matched)
- OOR flags (per channel)
- Default slave hit counter
- W channel FSM state
- Routing decision storage (for W tracking)
```

5.7.1.2 Common Issues and Debug

Symptom: Reads return all zeros
Check: - Is address out-of-range? - Check slave base/size configuration - Verify address decode logic in waveform

Symptom: Write data goes to wrong slave
Check: - W channel tracking FSM state - Did AW routing complete before W started? - Check for AWVALID/AWREADY handshake

Symptom: Unexpected DECERR responses
Check: - Address decode configuration - Overlapping slave ranges (wrong priority) - Off-by-one in size calculations

5.7.2 Verification Considerations

5.7.2.1 Address Decode Tests

Critical test scenarios: 1. **Boundary conditions**: base_address, base_address + size - 1 2. **Just out-of-range**: base_address - 1, base_address + size 3. **Each slave**: Verify routing to correct slave 4. **Overlapping ranges**: Verify priority encoding 5. **Default slave**: Unmapped addresses route correctly

5.7.2.2 Write Tracking Tests

W channel FSM testing: 1. **Simple write**: Single AW, single W (WLAST=1) 2. **Burst write**: AW with AWLEN=7, eight W beats 3. **Back-to-back writes**: New AW before previous W completes 4. **Interleaved masters**: Multiple masters writing simultaneously (if supported)

5.7.2.3 OOR Error Tests

```
Test: Read from unmapped address
- Send AR to 0xFFFF_FFFF (assuming unmapped)
- Verify R response with RRESP = DECERR
- Verify RID matches ARID
- Check response latency

Test: Write to unmapped address
```

```
- Send AW to invalid address
- Send W data
- Verify B response with BRESP = DECERR
- Verify BID matches AWID
```

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

6 2.3 Crossbar Core

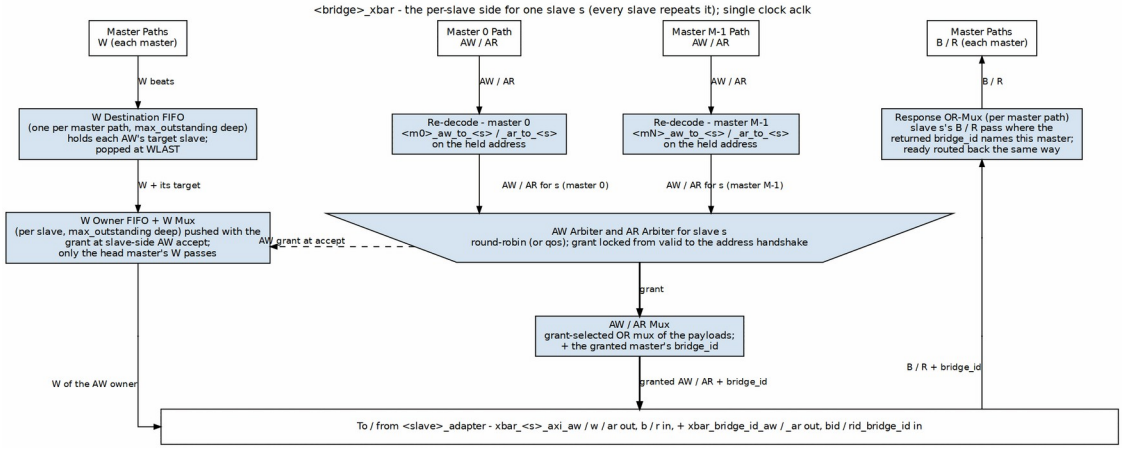
The crossbar core is the fabric itself — any master can reach any slave through it. Arbitration, request routing, and response management, the machinery that makes any-to-any work, all live here.

6.1 Overview

The core does five things:

1. **Full Connectivity**: Provides complete N×M master-to-slave interconnect matrix
2. **Independent Arbitration**: Per-slave arbiters allow parallel access to different slaves
3. **Response Routing**: Directs slave responses back to originating masters using Bridge IDs
4. **Transaction Ordering**: Maintains AXI ordering requirements within address dependencies
5. **Backpressure Management**: Handles flow control across multiple concurrent transactions

6.1.1 Figure 2.3: Crossbar Core Architecture



Crossbar Core Architecture

The per-slave side of `<bridge>_xbar` for one slave; every slave repeats it. Each master path re-decodes its held address, the AW and AR arbiters (the trapezium) pick one master per slave, and a grant-selected OR mux forwards the winner with its `bridge_id`. W beats are steered by a W destination FIFO per master path and a W owner FIFO per slave. B / R climb the right lane to the master named by the `bridge_id` the slave adapter returns.

6.2 Parameters

The crossbar-level knobs come straight from the TOML. Read the comments as carefully as the keys — several entries you'd expect to find here don't exist, and the file says so:

```
[bridge]
num_masters = 4
num_slaves = 3
# internal_data_width: NOT A KEY -- the crossbar has no fixed internal
width;
# each path carries its own port width and converters sit at the
boundaries.
arbiter_type = "round_robin"       # "round_robin", "fixed_priority",
"weighted"
registered_mux = false             # true = +1 cycle, better timing
registered_demux = false           # true = +1 cycle, better timing
# NOTE: there is no enable_cam / cam_depth key. The CAM is chosen per
slave by the
# generator (multi-master AXI4/AXI5 slave, or enable_ooo = true on
[[bridge.slaves]])
# and is the slave's max_outstanding deep (default 16; [bridge] or
per-port
# key, power of two 2..64 -- bridge GitHub #119 L5).
```

6.3 Functional Description

6.3.1 Connectivity Matrix

The bridge implements a **non-blocking crossbar** where: - N masters can each access different slaves simultaneously - Conflicts occur only when multiple masters target the same slave - Independent read and write paths increase parallelism

Example: 4 masters, 3 slaves

```
        S0   S1   S2
    M0  ●    ●    ●
    M1  ●    ●    ●
    M2  ●    ●    ●
    M3  ●    ●    ●
```

```
● = Connection available
```

6.3.2 Request Path Multiplexing

For each slave, requests from all masters are multiplexed:

```
Slave 0 Request Inputs:
  - M0 → S0 (AR, AW, W channels)
  - M1 → S0 (AR, AW, W channels)
  - M2 → S0 (AR, AW, W channels)
  - M3 → S0 (AR, AW, W channels)

Arbiter selects one master per channel per cycle
  → Grants propagate through MUX
  → Selected master's transaction forwarded to slave
```

6.3.3 Response Path Demultiplexing

Responses from slaves are demultiplexed back to masters:

```
Slave 0 Response Outputs (R, B channels):
  - Take the master index the slave adapter's tracker returns
    (bridge_cam lookup by RID/BID, or in-order FIFO head)
  - Route to corresponding master (M0, M1, M2, or M3)
  - Master adapter strips the {BRIDGE_ID} prefix from the ID
```

6.3.4 Per-Slave Request Arbitration

Each slave has **independent arbiters** for: 1. **AR Channel**: Read address arbitration (all masters competing) 2. **AW/W Channels**: Write address/data arbitration (all masters competing)

This separation allows: - Simultaneous read and write to same slave (if slave supports it) - Independent grant decisions for AR vs. AW channels - Better throughput for mixed read/write workloads

Policy (`[bridge] arbitration`, bridge TASK-006 (was BRIDGE-017)). `"rr"` (default) is round-robin among the requesting masters, grant locked until the slave-side handshake. `"qos"` picks the requester with the highest *effective* priority: `AxQOS` plus an age term that climbs one level every `2**qos_aging_shift` cycles a request waits (default 16), saturating at 15, cleared on grant; equals share round-robin. A QoS-0 requester behind a QoS-15 one is therefore served after at most `15 * 2**shift` cycles of waiting – delayed, never starved. Priority reorders only what is waiting at the arbiter: a slave that accepts an AW every cycle takes each master's AW as it arrives, faster than any master issues them, so no two requests are ever pending together and the W order is arrival order whatever the QoS values say. It takes effect where it is meant to, at a slave whose outstanding-write depth is full and whose AW channel backpressures, because that is when a backlog forms and every AW slot is a decision. Measured on `bridge_2x2_rw_qos` (`test_bridge_2x2_rw_qos_arb`) against a port that accepts one AW per 13 cycles: a QoS-8 stream against a QoS-0 stream takes >= 75% of the beats in either orientation, the QoS-0 stream's longest wait for an AW grant stays inside the aging bound, and equal QoS splits the port like the baseline. `AxQOS` itself is still passed through to the slave unchanged.

6.3.5 Request Multiplexers with Stable Address Gating

After arbitration, multiplexers select the granted master's signals. AR/AW gating uses **inline address re-decode** on the stable m_axi address bus (held stable across the full handshake):

```
// Inline address re-decode for stable gating (replaces slave_select_*
gating)
// The address on m_axi is held stable from fub_axi handshake through
m_axi handshake
always_comb begin
    // Re-decode each master's address to determine target slave
    m0_targets_s0 = (m0_m_axi_araddr >= S0_BASE) && (m0_m_axi_araddr <
S0_END);
    m1_targets_s0 = (m1_m_axi_araddr >= S0_BASE) && (m1_m_axi_araddr <
S0_END);
    m2_targets_s0 = (m2_m_axi_araddr >= S0_BASE) && (m2_m_axi_araddr <
S0_END);
    m3_targets_s0 = (m3_m_axi_araddr >= S0_BASE) && (m3_m_axi_araddr <
S0_END);
end

// Gate with valid signals
assign m0_ar_s0_valid = m0_arvalid && m0_targets_s0;
assign m1_ar_s0_valid = m1_arvalid && m1_targets_s0;
assign m2_ar_s0_valid = m2_arvalid && m2_targets_s0;
assign m3_ar_s0_valid = m3_arvalid && m3_targets_s0;
```

```
// Simplified AR channel MUX for Slave 0
always_comb begin
    case (ar_grant_s0)
        2'b00: begin  // Master 0 granted
            s0_arvalid = m0_arvalid;
            s0_araddr  = m0_m_axi_araddr;  // Stable across m_axi
handshake
            s0_arid    = m0_arid;    // Includes Bridge ID
            // ... other AR signals
        end
        2'b01: begin  // Master 1 granted
            s0_arvalid = m1_arvalid;
            s0_araddr  = m1_m_axi_araddr;  // Stable across m_axi
handshake
            s0_arid    = m1_arid;
            // ... other AR signals
        end
        // ... cases for M2, M3
    endcase
end
```

Why this matters: The address on `m_axi_*` is held stable for the entire AXI handshake (arvalid && arready). This allows address re-decoding to determine the target slave even as the master's internal address state changes.

6.3.6 AW→W Burst Tracking FIFO

For masters with multiple slave connections, W beats must follow the slave that the AW was driven to. A per-(master, slave) FIFO tracks which slave the AW was routed to:

```
// Track which slave the AW was driven to
always_ff @(posedge aclk or negedge aresetn) begin
    if (!aresetn) begin
        aw_trk_slave_id <= '0;
        aw_trk_valid <= 1'b0;
    end else if (m0_awvalid && m0_m_axi_awready && s_selected_aw ==
S0) begin
        aw_trk_slave_id <= S0;  // Track that AW went to slave 0
        aw_trk_valid <= 1'b1;
    end else if (m0_wvalid && m0_m_axi_wready && m0_m_axi_wlast) begin
        aw_trk_valid <= 1'b0;  // AW->W burst complete
    end
end

// Gate W path based on tracked slave
logic w_to_s0, w_to_s1, w_to_s2;
always_comb begin
    w_to_s0 = m0_wvalid && (aw_trk_slave_id == S0);
    w_to_s1 = m0_wvalid && (aw_trk_slave_id == S1);
```

```
    w_to_s2 = m0_wvalid && (aw_trk_slave_id == S2);
end
```

Why this matters: Without tracking, W beats would re-evaluate address decode which may have stale or different values, routing to the wrong width converter or slave.

6.3.7 Backpressure Propagation

Ready signals flow back from slave through arbiter to the granted master:

```
Slave S0 ARREADY → Arbiter → Granted Master ARREADY
                           ↘ Other Masters ARREADY = 0
```

This ensures: - Only granted master sees READY from slave - Non-granted masters see READY = 0 (backpressure) - No combinatorial loops in ready path (registered arbitration)

6.3.8 Bridge ID and Response Routing

With more than one master, slave-side IDs are `{BRIDGE_ID, id}` (bridge TASK-005; 5 bits at `ddr_s_axi_awid` for 4-bit master IDs in `bridge_2x2_rw`). The crossbar does not decode those prefix bits. Each slave adapter returns the originating master's index beside every response (`<slave>_axi_bid_bridge_id` / `_rid_bridge_id`, with a valid), produced by its tracker:

Bridge ID and Response Routing

Slave	Tracker	Lookup
AXI4 / AXI5, multi-master (or `enable_ooo`)	`bridge_cam`, `max_outstanding` deep (default 16), duplicates allowed	by the returned BID / RID, same cycle
single master, apb / axil / wb shim, subtractive	in-order FIFO, `max_outstanding` deep (default 16)	FIFO head

There is no timestamp or transaction-type field in the CAM, no timeout and no registered lookup.

6.3.9 Response Arbiters and Demultiplexers

A master may have requests in flight at several slaves (limitation fixed: GitHub #119 L1), so two slaves can offer it a response in the same cycle. Each master width path that reaches more than one slave has a B and an R response arbiter. A slave requests it when its response is valid and its tracker names this master; round robin picks one; a grant that is offered and not taken holds (AXI keeps the payload stable), and an R grant holds through RLAST so bursts never interleave:

```
// bridge_2x2_rw_xbar.sv: B arbiter for cpu's 32b path (excerpt)
assign cpu_32b_b_rarb_req = {
    ((subtractive_axi_bid_bridge_id == 0) &&
```

```
subtractive_axi_bid_valid) && subtractive_axi_bvalid,
    ((sram_axi_bid_bridge_id == 0) && sram_axi_bid_valid) &&
sram_axi_bvalid,
    ((ddr_axi_bid_bridge_id == 0) && ddr_axi_bid_valid) &&
ddr_axi_bvalid};
wire cpu_32b_b_rgnt_ddr = cpu_32b_b_rarb_gnt_valid &&
(cpu_32b_b_rarb_gnt == 2'd0);

// the mux toward the master and the slave's BREADY both follow the
grant
assign cpu_32b_bvalid = (cpu_32b_b_rgnt_ddr  ? ddr_axi_bvalid  : '0) |
                        (cpu_32b_b_rgnt_sram ? sram_axi_bvalid : '0) |
...;
assign ddr_axi_bready = (cpu_32b_b_rgnt_ddr ? cpu_32b_bready : '0) |
                        (dma_32b_b_rgnt_ddr ? dma_32b_bready : '0);
```

A path with one slave keeps a plain ownership gate on the tracker's master index; there is nothing to arbitrate. Width paths are arbitrated once more inside the master adapter (MAS 2.1, 2.8). The ID forwarded is still the fabric ID; the master adapter strips the prefix (`fub_axi_bid = cpu_32b_b.id[3:0]`).

6.3.10 AXI Ordering Requirements

The crossbar maintains AXI ordering rules.

6.3.10.1 Read-After-Write (RAW) Ordering

Rule: Read from address must see data from earlier write to same address

Crossbar Behavior: - Ordering enforced at slave level (slave handles RAW within itself) - Crossbar does NOT reorder transactions to same slave from same master - Different masters to same slave: No ordering guaranteed (slave must handle)

6.3.10.2 Write-After-Write (WAW) Ordering

Rule: Writes to overlapping addresses must complete in issue order

Crossbar Behavior: - Same master to same slave: Order preserved by arbiter (FIFO grant queue) - Different masters to same slave: Slave responsible for write ordering

6.3.10.3 Out-of-Order (OOO) Completion

Built, per slave. A multi-master AXI4/AXI5 slave (or one with `enable_ooo`) tracks with `bridge_cam` and may complete out of order across IDs; FIFO-tracked slaves (single master, shims, subtractive) must answer in request order. Across slaves a master sees responses in arrival order, and same-ID order is kept by the master adapter's per-ID table.

Allowed: - Read responses must return IN ORDER; a slave that reorders between RIDs misroutes (bridge BUG-008, was BRIDGE-010) - Reads to different slaves can complete in any order - Writes to different slaves can complete in any order

Crossbar Support: - Bridge ID tracking enables OOO response routing - Master sees responses in slave-determined order - Multi-master OOO requires careful slave design

6.3.11 Monitor Aggregation at Bridge Top (when `variants` includes `mon`)

When monitor collection is enabled, per-port monitor streams from `axi4_master_{rd,wr}_mon` and `axi4_slave_{rd,wr}_mon` wrappers are aggregated through a tree of `monbus_arbiter` instances. The final aggregated stream feeds a single `monbus_axil4_axil4_group` instance at the bridge top level:

Aggregation Hierarchy:

```
Master-side monitors → monbus_arbiter tree (if >2 masters)
Slave-side monitors  → monbus_arbiter tree (if >2 slaves)
                     ↓
            monbus_axil4_axil4_group
                     ↓
      s_mon_axil (slave), m_axil_mon (master), stream_irq
```

The `monbus_axil4_axil4_group` instance: - Provides a 64-bit slave AXIL interface for CPU read access (`s_mon_axil_*`) - Provides a master AXIL interface for DMA writes to system memory (`m_axil_mon_*`) - Contains an error FIFO tracking packets with error flags - Exposes `stream_irq` (asserted when error FIFO has records) - Includes an internal 64-bit free-running timestamp counter sampled at each packet arrival

Reference: See `docs/markdown/rtl-amba/_book_monitor_index.md` (monitor documentation book, with per-module pages under `docs/markdown/rtl-amba/monitor/`) for complete monitor design-surface documentation and `docs/markdown/rtl-amba/includes/ocelus_package_spec.md` for the packet layout.

6.4 Timing

6.4.1 Latency

Request Path (Master → Slave): - Arbiter decision: 1 cycle (registered) - MUX selection: 0 cycles (combinatorial) or +1 cycle with `xbar_pipeline` - **Total**: 1-2 cycles through crossbar (measured end to end: 2 combinational, 3 registered – HAS Table 5.7)

Response Path (Slave → Master): - BID extraction: 0 cycles (combinatorial) - DEMUX routing: 0 cycles (combinatorial) or +1 cycle with `xbar_pipeline` - **Total**: 0-1 cycles through crossbar (measured end to end: 2 combinational, 3 registered)

End-to-End (Master adapter → Slave adapter): - Adapters: 2 cycles (skid buffers) - Router: 0-1 cycles (decode) - Crossbar: 1-2 cycles (arbitration + MUX) - **Total**: 3-5 cycles minimum latency

6.4.2 Throughput

Best Case (no conflicts): - Each master to different slave: N transactions/cycle (N = master count) - Maximum aggregate bandwidth: N × data_width bits/cycle

Arbitration Limits: - Multiple masters to one slave: 1 transaction/cycle to that slave - Other slaves remain available for parallel access

Burst Performance: - Once granted, bursts flow at 1 beat/cycle - Grant held until burst completes (RLAST or WLAST)

6.4.3 Critical Paths

The paths that will bite you first:

1. **Arbiter Request → Grant**:
   - All masters' VALID signals → Arbiter logic → Grant decision
   - Depth: ~5-8 logic levels for 4 masters
2. **Grant → Ready Backpressure**:
   - Slave READY → Arbiter → Selected master READY
   - Depth: ~4-6 logic levels
3. **Response Demux**:
   - Slave RDATA/RID → BID extraction → Master select → Master RDATA
   - Depth: ~6-10 logic levels for 64-bit data

Mitigation Strategies:

1. **Registered crossbar** – BUILT (bridge TASK-006 (was BRIDGE-017), `[bridge] xbar_pipeline = true`). Every slave-side channel gets a 2-deep `gaxi_skid_buffer` inside the xbar: the request stages (AW, W, AR) sit

between the arbitrated mux and the slave port and carry the bridge id in their payload; the response stages (B, R) sit between the slave port and the OR-merge and carry the slave adapter's `bid/rid_bridge_id` and route-open flag WITH the beat, so the merge keys on the staged id and the stage's own valid. Both cones – decode -> arbiter -> N-way mux, and the response OR-merge – end at a register. The skid's ready is registered too, so both directions are cut and throughput is unchanged: `test_bridge_2x2_rw_perf` measures the same 1.00 beat/cycle on `bridge_2x2_rw_pipe` as on `bridge_2x2_rw`, and `test_bridge_2x2_rw_pipe_latency` pins the cost at exactly one cycle each way (3/3 against the baseline's 2/2). The routing and mux emitters are untouched; they drive `xs_<slave>_axi_*` nets and `_generate_pipeline_stages()` joins those to the ports. Off by default.

2. **Pipelined Arbitration**: Multi-cycle arbiter for >8 masters (not built)
3. **Hierarchical Crossbar**: For >16 masters, use tree structure (not built)

6.5 Design Notes

6.5.1 Resource Utilization

4 masters × 3 slaves configuration (64-bit data, 32-bit addr):

```
Logic Elements:  ~2000-3500 LEs
Registers:       ~800-1200 regs
Block RAM:       0 (no CAM is built)

Breakdown per slave:
- Arbiter (4 masters, RR):        ~200 LEs, ~50 regs
- Request MUX (AR/AW/W):          ~400 LEs, ~100 regs
- Response DEMUX (R/B):           ~300 LEs, ~80 regs
- Control FSMs:                   ~100 LEs, ~50 regs

Total for 3 slaves: 3 × 1000 LEs = ~3000 LEs
Plus routing overhead: +500 LEs
```

Scaling with Masters and Slaves — linear: - Adding 1 master: +~500 LEs per slave (new arbiter input) - Adding 1 slave: +~1000 LEs (complete new slave port)

Example: 8 masters × 6 slaves

```
Estimated:  ~12,000 LEs, ~3000 regs
Block RAM:  0 (no CAM is built)
```

Optimization Techniques:

1. **Read-Only/Write-Only Masters**: Reduces arbiter complexity by 40-50%
2. **Power-of-Two Master Count**: Simplifies BID width and routing logic
3. **Pipeline Stages**: Trading latency for frequency (deeper pipelines)

6.5.2 Debug and Observability

Recommended debug signals:

```
Per Slave:
- Arbiter grants (which master granted)
- Arbiter requests (which masters requesting)
- Request MUX outputs (VALID, READY, ADDR, ID)
- Response DEMUX inputs (VALID, READY, DATA, ID)

Global:
- Active transactions count
- Stall counters (arbiter conflicts)
- BID extraction errors
- bridge_id FIFO occupancy (wr_ptr/rd_ptr)
```

Useful metrics for profiling:

```
- Transactions per slave (read, write separate)
- Arbiter conflict rate (requests denied due to grant  contention)
- Average grant latency
- Utilization per master (% cycles active)
- Utilization per slave (% cycles busy)
```

6.5.3 Common Issues

Symptom: Master hangs with VALID=1, READY=0
Check: - Is another master holding grant to this slave? - Is arbiter logic functioning (check grant signals)? - Is slave responding with READY?

Symptom: Response goes to wrong master
Check: - Bridge ID values (verify correct BID per master) - bridge_id FIFO contents (wr_fifo/rd_fifo) - BID extraction logic (check bit positions)

Symptom: Throughput lower than expected
Check: - Arbiter conflicts (multiple masters to same slave?) - Pipeline depth (excessive latency reducing effective bandwidth?) - Burst efficiency (are bursts being granted properly?)

6.5.4 Future Enhancements

Planned Features: - **Weighted Round-Robin**: QoS support with configurable priorities - **Slave-Side Arbitration Policies**: Per-slave arbiter configuration - **Grant Prediction**: Speculative grant for lower latency - **Congestion Control**: Throttling to prevent hotspots

Under Consideration: - **Partial Crossbar**: Configurable master-to-slave connectivity (not full mesh) - **Multi-Tier Hierarchy**: For 32+ masters/slaves - **Virtual Channels**: Separate channels for different traffic classes - **Register Slicing**: Automatic pipeline insertion for timing

6.6 Related Modules

- Section 2.1: Master Adapter (request sources)
- Section 2.2: Slave Router (address decode before arbitration)
- Section 2.4: Arbitration (detailed arbiter algorithms)
- Section 2.5: ID Management (sideband bridge_id tracking; its CAM was never built)
- Section 3.1: Top-Level Integration (crossbar instantiation)

6.7 Testing

6.7.1 Functional Tests

1. **Single Master to Each Slave**: Verify basic connectivity
2. **All Masters to One Slave**: Stress arbiter fairness
3. **All Masters to All Slaves**: Maximum parallelism test
4. **OOO Responses**: Issue transactions with different latencies
5. **Burst Interleaving**: Multiple masters with bursts to same slave

6.7.2 Corner Cases

```
- Back-to-back grants (no idle cycles)
- Grant held for maximum burst length (256 beats)
- Rapid master switching (each gets 1 transaction then switches)
- Response while request in progress (pipelining)
- All masters requesting same slave simultaneously
```

6.7.3 Protocol Compliance

- AXI4 protocol checker at each master/slave interface
- Verify no READY → VALID dependencies (AXI violation)
- Check ID preservation through crossbar (modulo Bridge ID)
- Verify LAST signal handling

7 2.4 Arbitration

When several masters ask for the same slave in the same cycle, something has to pick a winner — that's the arbiter. Each slave gets dedicated arbiters for its read and write channels, so the pick is fair and nothing queues behind the wrong channel.

7.1 Overview

The arbiter does five things:

1. **Conflict Resolution**: Selects one master when multiple masters request same slave
2. **Fairness**: Ensures all masters eventually get access (starvation-free)
3. **Throughput Optimization**: Minimizes idle cycles and maximizes utilization
4. **Grant Management**: Maintains grants for burst transactions
5. **Priority Enforcement**: Supports priority-based access policies (optional)

7.2 Parameters

Arbiter configuration from the TOML — though here the comments carry the real story, since two of the three policies documented later in this chapter exist nowhere in the generated RTL:

```
[bridge]
# arbiter_type: NOT A KEY. The policy key is `arbitration`: "rr"
(default,
# round-robin) or "qos" (AWQOS / ARQOS + age, round-robin among
equals; the
# age step is 2**qos_aging_shift cycles). Both lock until the address
# handshake; fixed_priority and weighted exist nowhere in
bin/bridge_pkg/ or
# rtl/generated/.

[bridge.arbitration]
pipeline_stages = 1             # 1-3 (more = better timing, higher
latency)
# enable_priority_aging: NOT A KEY -- aging comes with arbitration =
"qos".
aging_threshold = 1000          # Cycles before priority boost

# Weighted arbitration does not exist.
[[bridge.arbitration.weights]]
master = "cpu"
weight = 4                      # Relative weight (1-255)

[[bridge.arbitration.weights]]
master = "dma"
weight = 2

[[bridge.arbitration.weights]]
master = "periph"
weight = 1
```

7.3 Functional Description

7.3.1 Per-Slave, Per-Channel Arbiters

Each slave has **two independent arbiters**:

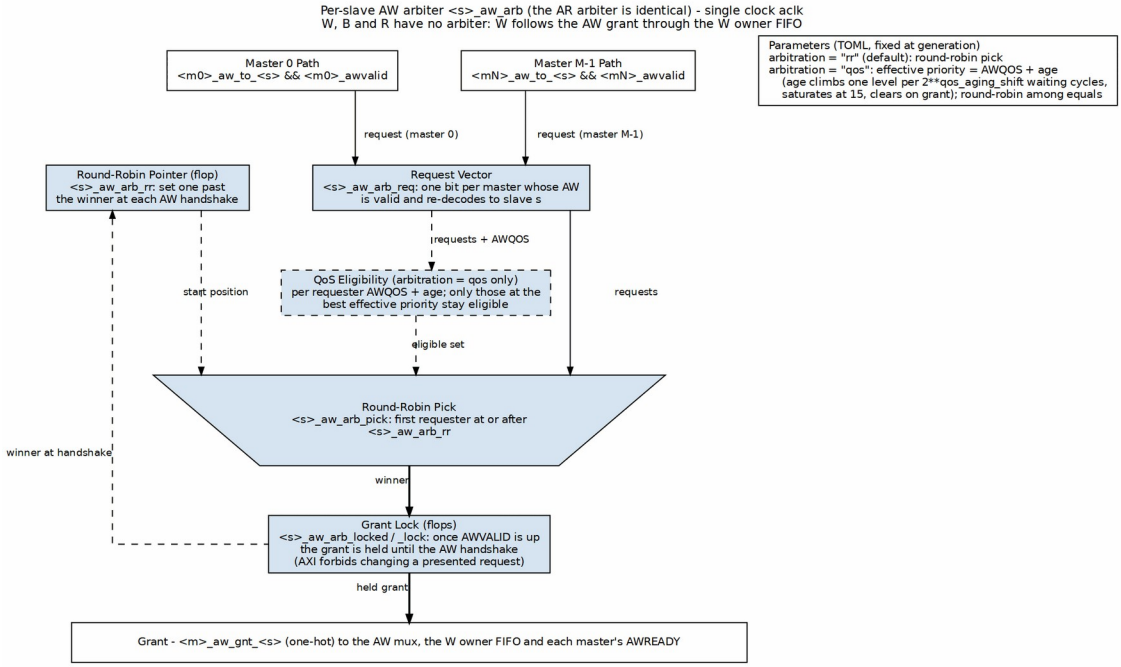
```
Slave 0:
  - AR Arbiter (Read Address Channel)
    * Inputs: M0_ARVALID, M1_ARVALID, M2_ARVALID, M3_ARVALID
    * Output: Grant[1:0] → Selects one master

  - AW Arbiter (Write Address Channel)
    * Inputs: M0_AWVALID, M1_AWVALID, M2_AWVALID, M3_AWVALID
    * Output: Grant[1:0] → Selects one master
    * Coordinates with W channel data flow
```

Independence: AR and AW arbiters operate independently, allowing simultaneous read and write to same slave (if slave supports full-duplex operation).

7.3.2 Block Diagram

7.3.3 Figure 2.4: Per-Slave Address Arbiter



Per-Slave Address Arbiter

One AW arbiter as generated in `<bridge>_xbar` (the AR arbiter is identical). The request vector holds one bit per master whose AW is valid and re-decodes to this slave. The round-robin pick starts one past the last winner, and the grant is locked from AWVALID until the AW handshake,

not until the burst's last beat. With `arbitration = "qos"` a QoS-eligibility stage (dashed) narrows the pick to the requesters at the best AWQOS-plus-age priority.

7.3.4 Round-Robin Arbitration

7.3.4.1 Algorithm

The **Round-Robin (RR)** arbiter cycles through masters in order, giving each master one opportunity to access the slave:

```
Priority Order (rotates after each grant):
  Cycle 0: M0 > M1 > M2 > M3
  Cycle 1: M1 > M2 > M3 > M0  (M0 was granted, now lowest priority)
  Cycle 2: M2 > M3 > M0 > M1  (M1 was granted)
  Cycle 3: M3 > M0 > M1 > M2  (M2 was granted)
  Cycle 4: M0 > M1 > M2 > M3  (M3 was granted, cycle repeats)
```

7.3.4.2 Implementation

```
// Simplified Round-Robin arbiter (4 masters)
logic [1:0] last_grant;      // Which master was granted last
logic [3:0] request;         // Current requests (VALID signals)
logic [1:0] grant;           // Grant decision

always_ff @(posedge clk) begin
    if (!rst_n) begin
        last_grant <= 2'b00;
    end else if (|request && slave_ready) begin
        // Update last_grant when transaction completes
        if (grant_accepted) begin
            last_grant <= grant;
        end
    end
end

always_comb begin
    // Default: No grant
    grant = 2'b00;

    // Priority encode starting after last grant
    case (last_grant)
        2'b00: begin  // Last grant to M0, try M1, M2, M3, M0
            if (request[1])      grant = 2'b01;
            else if (request[2]) grant = 2'b10;
            else if (request[3]) grant = 2'b11;
            else if (request[0]) grant = 2'b00;
        end
        2'b01: begin  // Last grant to M1, try M2, M3, M0, M1
            if (request[2])      grant = 2'b10;
```

```
            else if (request[3]) grant = 2'b11;
            else if (request[0]) grant = 2'b00;
            else if (request[1]) grant = 2'b01;
        end
        // ... similar for M2, M3
    endcase
end
```

7.3.4.3 Characteristics

Fairness: Excellent - Each master gets equal access over time
Latency: Bounded - Maximum wait = (N-1) × transaction_time
Throughput: Optimal - No idle cycles when requests pending
Starvation: None - Every master eventually gets grant

Best Use Cases: - Peers with similar priority - General-purpose interconnects - Default choice for most bridges

7.3.5 Fixed-Priority Arbitration

7.3.5.1 Algorithm

The **Fixed-Priority** arbiter always grants to the highest-priority requesting master:

```
Priority Order (never changes):
  M0 > M1 > M2 > M3  (M0 always highest priority)

Grant Decision:
  if (M0_VALID) → Grant = M0
  else if (M1_VALID) → Grant = M1
  else if (M2_VALID) → Grant = M2
  else if (M3_VALID) → Grant = M3
```

7.3.5.2 Implementation

```
// Fixed-priority arbiter (4 masters)
logic [3:0] request;
logic [1:0] grant;

always_comb begin
    // Priority encoder: M0 highest, M3 lowest
    if (request[0])      grant = 2'b00;  // M0 highest priority
    else if (request[1]) grant = 2'b01;
    else if (request[2]) grant = 2'b10;
    else if (request[3]) grant = 2'b11;  // M3 lowest priority
    else                 grant = 2'b00;  // Default (no requests)
end
```

7.3.5.3 Characteristics

Fairness: Poor - Low-priority masters can starve
Latency: Variable - High-priority: minimal, Low-priority: unbounded
Throughput: Optimal - No idle cycles when requests pending
Starvation: Possible for low-priority masters

Best Use Cases: - Real-time systems with priority requirements - CPU (high) vs. DMA (low) scenarios - Time-critical masters need guaranteed access

7.3.5.4 Starvation Prevention

Optional enhancement: **Priority aging**

```
- Track cycles since last grant per master
- Temporarily boost priority of starved masters
- Reset age counter after grant

Example:
  M0 age = 100 cycles → Boost M0 above normal priority
  M1 age = 5 cycles   → Normal priority
```

7.3.6 Weighted Arbitration

7.3.6.1 Algorithm

The **Weighted** arbiter assigns different access rates to masters based on weights:

```
Configuration:
  M0 weight = 4  (50% of bandwidth)
  M1 weight = 2  (25% of bandwidth)
  M2 weight = 1  (12.5% of bandwidth)
  M3 weight = 1  (12.5% of bandwidth)

Grant Sequence (repeating pattern):
  M0, M0, M0, M0, M1, M1, M2, M3  (8 cycles total)
  M0 gets 4/8, M1 gets 2/8, M2 gets 1/8, M3 gets 1/8
```

7.3.6.2 Implementation

```
// Weighted arbiter using credit counter
logic [7:0] credits [0:3];  // Credit counters per master
logic [1:0] grant;

always_ff @(posedge clk) begin
    if (!rst_n) begin
        credits[0] <= 8'd4;  // M0 weight
        credits[1] <= 8'd2;  // M1 weight
        credits[2] <= 8'd1;  // M2 weight
        credits[3] <= 8'd1;  // M3 weight
```

```
    end else if (grant_accepted) begin
        // Decrement granted master's credits
        credits[grant] <= credits[grant] - 1;

        // Reload when all credits exhausted
        if (all_credits_zero) begin
            credits[0] <= 8'd4;
            credits[1] <= 8'd2;
            credits[2] <= 8'd1;
            credits[3] <= 8'd1;
        end
    end
end

always_comb begin
    // Grant to master with highest credits and active request
    grant = select_highest_credit_with_request(credits, request);
end
```

7.3.6.3 Characteristics

Fairness: Configurable - Proportional to weights
Latency: Bounded - Depends on weight ratio
Throughput: Near-optimal - Small overhead for credit tracking
Starvation: None - All masters get share per weight

Best Use Cases: - QoS requirements (guaranteed bandwidth) - Mixed workload (video + CPU + DMA) - Service-level agreements

7.3.7 Burst Handling

7.3.7.1 Grant Locking

Once granted, a master **holds the grant** until its burst completes:

```
AR Channel Burst:
  1. Master asserts ARVALID with ARLEN = 7 (8-beat burst)
  2. Arbiter grants to this master
  3. Grant held until slave returns RLAST
  4. Other masters blocked during this time

AW/W Channel Burst:
  1. Master asserts AWVALID with AWLEN = 15 (16-beat burst)
  2. Arbiter grants to this master
  3. Grant held until master asserts WLAST
  4. Other masters blocked during W data transfer
```

7.3.7.2 Implementation

```
// Burst grant locking
logic grant_locked;
logic [1:0] locked_master;

always_ff @(posedge clk) begin
    if (!rst_n) begin
        grant_locked <= 1'b0;
    end else begin
        if (grant_accepted && !last_beat) begin
            // Lock grant for burst
            grant_locked <= 1'b1;
            locked_master <= grant;
        end else if (last_beat) begin
            // Release grant when burst completes
            grant_locked <= 1'b0;
        end
    end
end

assign grant = grant_locked ? locked_master : arbiter_decision;
```

7.3.7.3 Fairness Considerations

Long bursts can block other masters: - Maximum AXI burst: 256 beats - At 1 cycle/beat: Up to 256 cycles blocked - Impacts latency for other masters

Mitigation: - Limit burst lengths in master configuration - Use burst interleaving (complex, not implemented in Phase 1) - Monitor arbiter stall counters

7.4 Timing

7.4.1 Arbitration Latency

Single-Cycle Arbitration (default):

```
Request → Grant Decision: 1 cycle
Grant Decision → MUX Output: 0 cycles (combinatorial)
Total: 1 cycle
```

Multi-Cycle Arbitration (high frequency):

```
Request → Grant Decision: 2-3 cycles (pipelined)
Improves timing at cost of latency
```

7.4.2 Critical Paths

Typical critical paths in the arbiter: 1. **Request collection**: All master VALID signals → Arbiter input 2. **Priority encode**: Compare all requests → Grant decision 3. **Grant propagate**: Grant → MUX select → Slave interface

Path Depth: - 4 masters: ~6-8 logic levels - 8 masters: ~8-12 logic levels - 16 masters: ~12-16 logic levels

7.5 Design Notes

7.5.1 Resource Utilization

Round-Robin (4 masters):

```
Logic Elements:  ~150 LEs
Registers:       ~40 regs

Breakdown:
- Priority encoder:        ~80 LEs
- Last grant tracking:     ~10 regs
- Request capture:         ~20 regs
- Grant FSM:               ~40 LEs, ~10 regs
```

Fixed-Priority (4 masters):

```
Logic Elements:  ~80 LEs
Registers:       ~20 regs

Simpler than RR (no rotation logic)
```

Weighted (4 masters):

```
Logic Elements:  ~200 LEs
Registers:       ~60 regs

Additional credit counters and comparison logic
```

Scaling with Master Count — resource usage grows approximately **O(N²)** where N = master count:

```
2 masters:  ~50 LEs
4 masters:  ~150 LEs
8 masters:  ~400 LEs
16 masters: ~1200 LEs
```

The N² scaling comes from: - Priority encoder: Compares all pairs of masters - Grant MUX: N-to-1 multiplexing increases with N

7.5.2 Performance Impact

Round-Robin: - **Pros**: Fair, starvation-free, predictable latency - **Cons**: Cannot prioritize time-critical masters - **Use**: General-purpose, peer masters

Fixed-Priority: - **Pros**: Guaranteed low latency for high-priority masters - **Cons**: Low-priority can starve, unpredictable for low-priority - **Use**: Real-time, priority-sensitive systems

Weighted: - **Pros**: Configurable bandwidth allocation, QoS support - **Cons**: More complex, small overhead - **Use**: Mixed workload, SLA requirements

Arbiter Efficiency — assuming all masters request the same slave continuously:

```
Best Case: 100% efficiency (one master)
  - No arbitration conflicts
  - Zero idle cycles

Typical Case: 25-50% per-master efficiency (4 masters, round-robin)
  - Each master gets ~25% of time grants
  - Load balancing to other slaves improves

Worst Case: Heavy contention
  - 4 masters to 1 slave: 25% efficiency per master
  - Solution: Add slaves or use more slaves in parallel
```

7.5.3 Debug and Observability

Recommended debug signals and counters:

```
Per Arbiter:
- Request vector (all masters requesting)
- Grant decision (which master granted)
- Grant locked status (burst in progress)
- Last grant (for RR rotation tracking)

Performance Counters:
- Grants per master (utilization)
- Denied requests per master (contention)
- Average grant latency per master
- Arbiter conflict cycles (multiple requests, one grant)
```

7.5.4 Common Issues

Symptom: Master never gets grant (starvation)
Check: - Arbiter type (fixed-priority can starve low-priority masters) - Other masters continuously requesting? - Enable priority aging if using fixed-priority

Symptom: Unfair access (one master dominates)
Check: - Arbiter type (should be round-robin for fairness) - Burst lengths (long bursts block others) - Request patterns (one master requesting more frequently)

Symptom: Low throughput despite requests
Check: - Arbiter conflicts (too many masters to one slave) - Pipeline depth (excessive arbitration latency) - Burst efficiency (short bursts waste cycles)

7.5.5 Future Enhancements

Planned Features: - **Dynamic Weight Adjustment**: Runtime-configurable weights via registers - **QoS Classes**: Multiple priority levels with configurable policies - **Deadline-Based Arbitration**: Grant based on transaction deadlines - **Predictive Arbitration**: Speculative grants to reduce latency

Under Consideration: - **Multi-Level Arbitration**: Hierarchical for >16 masters - **Token Bucket**: Rate limiting per master - **History-Based**: Learn access patterns and optimize grants - **ECC Protection**: For arbitration state (safety-critical systems)

7.6 Related Modules

- Section 2.3: Crossbar Core (arbiter integration)
- Section 2.1: Master Adapter (request sources)
- Section 2.5: ID Management (transaction tracking during grants)
- Chapter 6: Performance (arbiter impact on throughput)

7.7 Testing

7.7.1 Test Scenarios

1. **Fair Access Test** (Round-Robin):

```
- All masters continuously request same slave
- Verify each master gets equal grants over 1000 cycles
- Expected: Each of 4 masters gets ~250 grants
```

2. **Priority Test** (Fixed-Priority):

```
- High-priority master requests intermittently
- Low-priority masters request continuously
- Verify high-priority always wins when requesting
```

3. **Burst Locking Test**:

```
- Master issues 16-beat burst
- Other masters request during burst
- Verify grant held until LAST
- Verify other masters blocked during burst
```

4. **Weighted Bandwidth Test**:

```
- Configure M0=4, M1=2, M2=1, M3=1
- All masters request continuously for 10000 cycles
- Verify grants proportional: M0≈50%, M1≈25%, M2≈12.5%, M3≈12.5%
```

7.7.2 Corner Cases

```
- Single master requesting (trivial grant)
- All masters requesting simultaneously (worst-case arbitration)
- Grant released and new grant same cycle (back-to-back)
- Master drops request after arbiter latency (stale grant)
- Maximum length burst (256 beats blocking)
```

8 2.5 ID Management

ID Management is how the bridge remembers which master sent each outstanding transaction, so the response can find its way back.

8.1 Overview

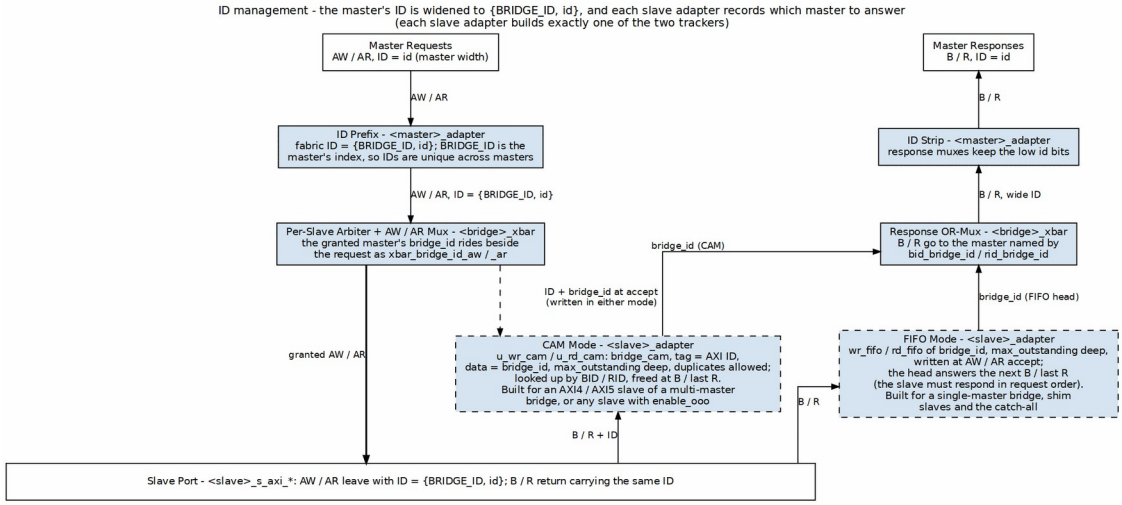
Before anything else in this chapter, read this — it colors everything below:

> **What the generated bridge actually does** (measured against `rtl/generated/`). Two mechanisms, both in Figure 2.5:
>
> - **The ID is widened.** The master adapter sends `{BRIDGE_ID, id}` into the fabric (bridge TASK-005), so IDs are unique across masters: in `bridge_2x2_rw`, `cpu_m_axi_awid` is 4 bits and `ddr_s_axi_awid` is 5. The response muxes return the low bits to the master.
> - **Each slave adapter tracks the originating master.** The granted master's index rides beside the request as `xbar_bridge_id_aw` / `xbar_bridge_id_ar` and is written into the slave adapter's tracker at the address handshake. An AXI4 / AXI5 slave of a multi-master bridge (or any slave with `enable_ooo`) gets `bridge_cam` (`u_wr_cam` / `u_rd_cam`), keyed by the AXI ID and looked up with the returned BID / RID, so the slave may complete different IDs in any order. A single-master bridge, the protocol shims and the catch-all keep an in-order `bridge_id` FIFO, which requires the slave to answer in request order (bridge BUG-008's simulation-only check flags a violation).
>
> Either tracker is fixed-depth (16), so the address handshake is gated on it being not full (bridge BUG-009).
>
> An earlier revision of this chapter said there was no CAM, that IDs passed through at equal width, and that everything below described a design that was never built. The prefix and the CAM are both built; the sections below say, per item, what the generated RTL does and what was only ever proposed.

The ID management system does four things:

1. **Transaction Tracking**: each slave adapter records which master issued each outstanding request
2. **Response Routing**: the crossbar steers B / R by the recorded master index (`bid_bridge_id` / `rid_bridge_id`)
3. **Out-of-Order Support**: a CAM-tracked slave may return different IDs in any order
4. **ID Space Isolation**: the `{BRIDGE_ID, id}` prefix keeps two masters' IDs distinct at every slave

8.1.1 Figure 2.5: ID Management Architecture



ID Management Architecture

The request takes the center column, widened to `{BRIDGE_ID, id}` in the master adapter and granted with its `bridge_id` in the crossbar. The response climbs the right side: the slave adapter's tracker (CAM or FIFO, dashed because each slave builds exactly one) names the master, and the crossbar and master adapter return B / R with the master's own id bits.

8.2 Parameters

There is no `[bridge.id_management]` table and no `enable_cam`, `cam_type`, `cam_depth` or `max_outstanding_*` key; `config_validator` rejects unknown keys. What decides ID handling:

```
[[bridge.masters]]
name = "cpu"
id_width = 4                    # master-side ID width

[[bridge.slaves]]
name = "ddr"
id_width = 5                    # must be >= widest master id_width +
$clog2(num_masters)
enable_ooo = false              # true forces bridge_cam on this slave
even with one master
```

Derived, not configured (emitted into `<bridge>_pkg.sv`):

Parameters

Constant	Value	bridge_2x2_rw
BRIDGE_ID_WIDTH	$clog2(NUM_MASTERS), minimum 1	1
MASTER_ID_WIDTH	widest master id_width	4
ID_PREFIX_WIDTH	$clog2(NUM_MASTERS), 0 for one master	1
XBAR_ID_WIDTH	MASTER_ID_WIDTH + ID_PREFIX_WIDTH	5

The tracker choice is fixed by the generator (`SlaveAdapterGenerator.use_cam`): `bridge_cam` when the slave sets `enable_ooo`, or when the bridge has more than one master and the slave is a real `axi4` / `axi5` port; an in-order FIFO otherwise (single master, apb/axil/wb shim slaves, the internal subtractive slave).

8.3 Functional Description

8.3.1 Bridge ID Concept

Problem – two masters may use the same AXI ID:

```
Master 0: ARID = 4'h5
Master 1: ARID = 4'h5
A slave returning RID = 4'h5 cannot say which master it answers.
```

Solution (built) – the master adapter prepends its index:

```
fabric ID = {BRIDGE_ID, master ID}

Master 0, ARID = 4'h5 -> 5'b0_0101
Master 1, ARID = 4'h5 -> 5'b1_0101     (bridge_2x2_rw)
```

8.3.2 ID Width Calculation

```
ID_PREFIX_WIDTH = $clog2(NUM_MASTERS)   (0 for a single master)

2 masters:  1 bit      3-4 masters: 2 bits
5-8 masters: 3 bits    9-16 masters: 4 bits

4 masters, 4-bit master IDs: slave ports see 6-bit IDs, masters see 4.
```

8.3.3 Multi-Master OR-Merge Gating

Every slave-side address, ID and `bridge_id` field is an OR of per-master terms, each gated by that master's AW / AR arbiter grant for the slave, so an idle master with a stale address contributes nothing:

```
// bridge_2x2_rw_xbar.sv
assign ddr_axi_bridge_id_aw = (cpu_32b_aw_gnt_ddr ? cpu_bridge_id_aw :
'0) |
                              (dma_32b_aw_gnt_ddr ? dma_bridge_id_aw :
'0);
```

The grant already implies the master is valid and decoded to this slave. An ungated OR let an idle master's zero `bridge_id` corrupt the routing tag and send a B to the wrong master.

8.3.4 ID Injection (Request Path, built)

```
// <master>_adapter.sv
assign xbar_axi_awid = {BRIDGE_ID_WIDTH'(BRIDGE_ID),
MASTER_ID_WIDTH'(fub_axi_awid)};
assign xbar_axi_arid = {BRIDGE_ID_WIDTH'(BRIDGE_ID),
MASTER_ID_WIDTH'(fub_axi_arid)};
```

BRIDGE_ID is the per-instance master index. The concatenation is wiring and adds no cycle. There is no “registered injection” option. A single-master bridge prepends nothing.

8.3.5 ID Stripping and Response Routing (built)

The response is NOT routed by decoding the prefix bits of BID / RID. The slave adapter returns the originating master’s index on a sideband (bid_bridge_id / rid_bridge_id, with bid_valid / rid_valid), and the crossbar’s B / R muxes select on it:

```
// bridge_2x2_rw_xbar.sv (B toward cpu, master index 0)
((ddr_axi_bid_bridge_id == 0) && ddr_axi_bid_valid ? ddr_axi_bid : '0)
| ...
```

The master adapter then returns the low MASTER_ID_WIDTH bits (fub_axi_bid = cpu_32b_b.id[3:0]), so the master sees its own ID.

8.3.6 Content Addressable Memory: bridge_cam (built)

rtl/bridge_cam.sv is hand-written and instantiated twice (u_wr_cam, u_rd_cam) in each CAM-tracked slave adapter:

Content Addressable Memory: bridge_cam

Parameter	Value in the slave adapter
TAG_WIDTH	ID_WIDTH (fabric ID width, 5 in bridge_2x2_rw)
DATA_WIDTH	BRIDGE_ID_WIDTH
DEPTH	16
ALLOW_DUPLICATES	1 (same ID may be outstanding more than once; oldest freed first)
PIPELINE_EVICT	0 (lookup result in the same cycle)

Operation:

```
Allocate:   on the AW / AR handshake from the crossbar
            tag = awid / arid, data = xbar_bridge_id_aw / _ar
Deallocate: write on B handshake; read on the R handshake with RLAST
            tag = bid / rid -> deallocate_data = bid_bridge_id /
rid_bridge_id
Full:       tags_full gates the crossbar-side AWREADY / ARREADY
(bridge BUG-009)
```

An entry holds only the tag, the master index and the duplicate-ordering count. There is no timestamp, transaction-type field or timeout, and only the parallel, distributed-RAM form exists; the sequential-search and BRAM-based CAM variants this chapter once compared were never built.

8.3.7 In-Order FIFO Tracker (built)

Slaves that do not get the CAM keep a `max_outstanding`-entry (default 16) FIFO of `bridge_id`, pushed on the address handshake and popped on B / RLAST. Routing uses the FIFO head, so such a slave must complete in request order; a simulation-only check flags a returned ID that does not match the head (bridge BUG-008). The same not-full gate applies.

8.3.8 Outstanding Transaction Limits

The limit per slave is the tracker depth: 16 writes and 16 reads outstanding at each slave adapter, CAM or FIFO. When the tracker is full the slave's AWREADY / ARREADY toward the crossbar drops until a response frees an entry. There is no per-master outstanding limit key.

8.4 Timing

- **ID injection / stripping**: 0 cycles (concatenation and bit-select).
- **CAM lookup**: 0 added cycles (`PIPELINE_EVICT = 0`; the deallocate data is combinational in the response cycle).
- **FIFO head**: 0 added cycles.

The measured end-to-end latency (2 cycles each way in `bridge_2x2_rw`, HAS Table 5.7) includes whichever tracker the slave uses.

8.5 Design Notes

8.5.1 Resource Utilization

The per-master prefix is wiring. Each CAM-tracked slave adapter carries two `bridge_cam` instances (16 entries at the default `max_outstanding`) built in distributed RAM and flops (no block RAM); the FIFO tracker is two 16-entry FIFOs of `BRIDGE_ID_WIDTH` bits. Measured totals for the fixtures are in HAS 5.3.

8.5.2 Debug and Observability

```
Request:  fub_axi_awid / arid  vs  xbar_axi_awid / arid (prefix added)
Slave:    xbar_bridge_id_aw / _ar at the address handshake
CAM:      u_wr_cam / u_rd_cam tags_count, tags_full, deallocate_valid
Response: bid_bridge_id / rid_bridge_id with bid_valid / rid_valid
```

8.5.3 Common Issues

Response goes to the wrong master - FIFO-tracked slave completing out of order (look for the BUG-008 check message) - Slave truncating the ID: its `id_width` must be at least `XBAR_ID_WIDTH` (the validator enforces this for axi3/axi4/axi5 slaves)

Address channel stalls - Tracker full: 16 outstanding at that slave with no B / RLAST returning - A hung slave stalls its path indefinitely; there is no timeout in the bridge

`deallocate_valid` low on a response - The slave returned an ID it was never sent

8.6 Related Modules

- Section 2.1: Master Adapter (prefix injection and stripping)
- Section 2.3: Crossbar Core (grant-gated OR-merge, B / R muxes)
- Section 2.8: Response Routing
- HAS ch04_interfaces/01_axi4_interface.md (port ID widths)

8.7 Testing

1. **Multi-master ID isolation**: two masters issue the same ID to one slave; each response returns to its own master with the original ID.
2. **Out-of-order at a CAM slave**: the slave answers IDs out of request order; each response still reaches its master (the mix_b / mix_d arbitration tests exercise this).
3. **Tracker capacity**: 16 outstanding at one slave holds off the 17th address until a response frees an entry (bridge BUG-009 tests).
4. **Timeout**: not testable – there is no timeout in the bridge. The system-level answer is a watchdog outside it.

9 2.6 Width Conversion

Masters and slaves rarely agree on data width, and width conversion is how the bridge copes. There is NO fixed internal width — each path carries its port's own width, and converters sit only at the boundaries where two ports disagree. `bridge_4x4_rw` alone instantiates paths at 32, 64, 128 and 256 bits.

9.1 2.6.1 Purpose and Function

Width conversion has five jobs:

1. **Data Path Adaptation**: Converts between different data widths (8, 16, 32, 64, 128, 256 bits)
2. **Burst Splitting**: Divides wide transactions into multiple narrow transactions
3. **Burst Merging**: Combines multiple narrow beats into fewer wide beats
4. **Strobe Mapping**: Translates write strobes across width boundaries
5. **Address Alignment**: Adjusts addresses for width differences

9.2 2.6.2 Internal Data Width

9.2.1 64-Bit Standard

The bridge uses **64-bit (8-byte) internal data width** for the crossbar:

```
Rationale:
- Balance between resource usage and performance
- Common width for modern embedded processors
- Efficient for both 32-bit and 64-bit masters
- Reasonably sized multiplexers in crossbar
```

9.2.2 Width Conversion Locations

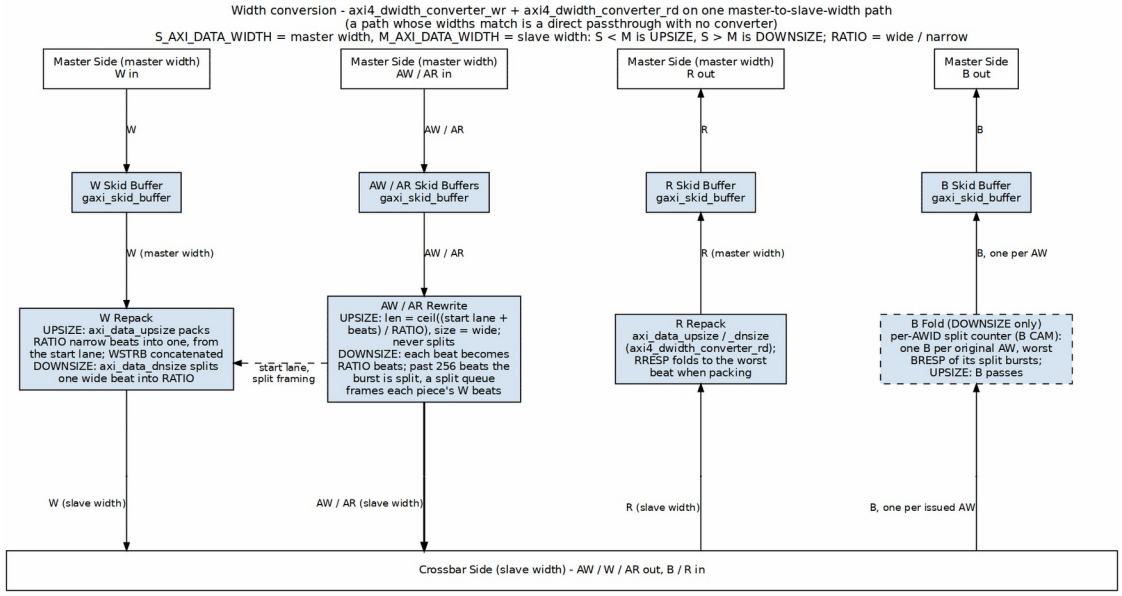
Converters live in the master adapter, one per distinct slave width the master reaches, so a path only pays for conversion when its two ends actually disagree. There is no fixed crossbar width:

- a 64-bit master reaching a 64-bit slave takes a direct passthrough path;
- the same master reaching a 512-bit slave goes through one `axi4_dwidth_converter_wr` / `_rd` pair set to UPSIZE;
- reaching a 32-bit slave, through one pair set to DOWNSIZE.

Each master-slave pair is converted at most once.

9.3 2.6.3 Block Diagram

9.3.1 Figure 2.6: Width Conversion Architecture



Width Conversion Architecture

One converter pair on one master-to-slave-width path. Every channel enters a skid buffer; W and R are repacked by `axi_data_upsize` / `axi_data_dnsize`; AW / AR length and size are rewritten (a DOWNSIZE burst that would exceed 256 beats is split, and a split queue frames its W beats); and in DOWNSIZE the B fold returns one B per original AW.

9.4 2.6.4 Upsizing (Narrow to Wide)

9.4.1 Overview

Upsizing converts narrow data to wide data by buffering multiple narrow beats into a single wide beat:

```
Example: 32-bit master → 64-bit crossbar
  Master beat 0: 32'hAAAA_BBBB (bytes 3:0)
  Master beat 1: 32'hCCCC_DDDD (bytes 7:4)
  → Crossbar beat: 64'hCCCC_DDDD_AAAA_BBBB
```

9.4.2 Write Upsizing

Burst of 4 (32-bit) → Burst of 2 (64-bit) — two master beats pack into each crossbar beat, and the burst length halves:

```
Master AW: AWADDR = 0x1000, AWLEN = 3, AWSIZE = 2 (4 bytes)
Master W beats:
  W[0]: WDATA = 0xAAAA_BBBB, WSTRB = 4'b1111, WLAST = 0
  W[1]: WDATA = 0xCCCC_DDDD, WSTRB = 4'b1111, WLAST = 0
  W[2]: WDATA = 0xEEEE_FFFF, WSTRB = 4'b1111, WLAST = 0
  W[3]: WDATA = 0x1111_2222, WSTRB = 4'b1111, WLAST = 1

Crossbar AW: AWADDR = 0x1000, AWLEN = 1, AWSIZE = 3 (8 bytes)
Crossbar W beats:
  W[0]: WDATA = 0xCCCC_DDDD_AAAA_BBBB, WSTRB = 8'b1111_1111, WLAST = 0
  W[1]: WDATA = 0x1111_2222_EEEE_FFFF, WSTRB = 8'b1111_1111, WLAST = 1
```

9.4.3 Read Upsizing

Burst of 8 (32-bit) → Burst of 4 (64-bit) — the same trick in reverse, where each wide crossbar beat fans back out into two narrow master beats:

```
Master AR: ARADDR = 0x2000, ARLEN = 7, ARSIZE = 2 (4 bytes)

Crossbar AR: ARADDR = 0x2000, ARLEN = 3, ARSIZE = 3 (8 bytes)

Crossbar R beats:
  R[0]: RDATA = 0x1111_2222_3333_4444, RLAST = 0
  R[1]: RDATA = 0x5555_6666_7777_8888, RLAST = 0
  R[2]: RDATA = 0x9999_AAAA_BBBB_CCCC, RLAST = 0
  R[3]: RDATA = 0xDDDD_EEEE_FFFF_0000, RLAST = 1

Master R beats (split from crossbar):
  R[0]: RDATA = 0x3333_4444, RLAST = 0  (from R[0] low)
  R[1]: RDATA = 0x1111_2222, RLAST = 0  (from R[0] high)
  R[2]: RDATA = 0x7777_8888, RLAST = 0  (from R[1] low)
```

```
  R[3]: RDATA = 0x5555_6666, RLAST = 0  (from R[1] high)
  R[4]: RDATA = 0xBBBB_CCCC, RLAST = 0  (from R[2] low)
  R[5]: RDATA = 0x9999_AAAA, RLAST = 0  (from R[2] high)
  R[6]: RDATA = 0xFFFF_0000, RLAST = 0  (from R[3] low)
  R[7]: RDATA = 0xDDDD_EEEE, RLAST = 1  (from R[3] high)
```

9.4.4 Strobe Mapping (Upsizing)

Strobes accumulate right alongside the data — each narrow WSTRB lands in the half of the wide word its beat occupied:

```
32-bit WSTRB → 64-bit WSTRB:

Beat 0 (lower 32 bits):
  32-bit WSTRB = 4'b1010 → 64-bit WSTRB = 8'b0000_1010

Beat 1 (upper 32 bits):
  32-bit WSTRB = 4'b1111 → 64-bit WSTRB = 8'b1111_0000

Combined:
  64-bit WSTRB = 8'b1111_1010
```

9.5 2.6.5 Downsizing (Wide to Narrow)

9.5.1 Overview

Downsizing converts wide data to narrow data by splitting a single wide beat into multiple narrow beats:

```
Example: 128-bit master → 64-bit crossbar
  Master beat: 128'h1111_2222_3333_4444_5555_6666_7777_8888
  → Crossbar beat 0: 64'h5555_6666_7777_8888 (lower)
  → Crossbar beat 1: 64'h1111_2222_3333_4444 (upper)
```

9.5.2 Write Downsizing

Burst of 2 (128-bit) → Burst of 4 (64-bit):

```
Master AW: AWADDR = 0x3000, AWLEN = 1, AWSIZE = 4 (16 bytes)
Master W beats:
  W[0]: WDATA = 0xAAAA...(128 bits), WSTRB = 16'hFFFF, WLAST = 0
  W[1]: WDATA = 0xBBBB...(128 bits), WSTRB = 16'hFFFF, WLAST = 1

Crossbar AW: AWADDR = 0x3000, AWLEN = 3, AWSIZE = 3 (8 bytes)
Crossbar W beats:
  W[0]: WDATA = 0xAAAA_low(64), WSTRB = 8'hFF, WLAST = 0
  W[1]: WDATA = 0xAAAA_high(64), WSTRB = 8'hFF, WLAST = 0
  W[2]: WDATA = 0xBBBB_low(64), WSTRB = 8'hFF, WLAST = 0
  W[3]: WDATA = 0xBBBB_high(64), WSTRB = 8'hFF, WLAST = 1
```

9.5.3 Read Downsizing

Burst of 1 (128-bit) → Burst of 2 (64-bit):

```
Master AR: ARADDR = 0x4000, ARLEN = 0, ARSIZE = 4 (16 bytes)

Crossbar AR: ARADDR = 0x4000, ARLEN = 1, ARSIZE = 3 (8 bytes)

Crossbar R beats:
  R[0]: RDATA = 0x1111_2222_3333_4444, RLAST = 0
  R[1]: RDATA = 0x5555_6666_7777_8888, RLAST = 1

Master R beat (merged):
  R[0]: RDATA = 0x5555_6666_7777_8888_1111_2222_3333_4444, RLAST = 1
```

9.5.4 Strobe Mapping (Downsizing)

A 128-bit bus has 16 strobe bits, one per byte lane; a 64-bit bus has 8. On a 2:1 downsize each wide beat becomes two narrow beats, and the strobes split along the same boundary as the data — the low 8 bits go with beat 0, the high 8 with beat 1. No bit is recomputed; the slice is positional.

```
128-bit WSTRB → 64-bit WSTRB (2:1 split):

  wide WSTRB = 16'b1111_0000_0000_1111   (bytes 15:12 and 3:0 written)

  Beat 0 (bytes  7:0) = wide[7:0]  = 8'b0000_1111
  Beat 1 (bytes 15:8) = wide[15:8] = 8'b1111_0000
```

The example this replaces was arithmetically impossible: it wrote a 32-bit value (`16'hF0F0_0FF0`) into a 16-bit literal, then gave both beats the same result (`8'hF0`) from different bit patterns, one of them an 8-bit literal holding three hex digits (`8'h0F0`). Nothing in the RTL was needed to see it was wrong.

9.6 2.6.6 Address Alignment

9.6.1 Address Adjustment for Width

Change the width and the address has to align to the new beat size:

```
32-bit (4-byte) aligned address: 0x1004
Upsized to 64-bit (8-byte): 0x1000 (round down to 8-byte boundary)

Narrow access at 0x1004 within 64-bit word:
  - Byte offset = 0x1004 & 0x7 = 4
  - Data at bytes [7:4] of 64-bit word
  - Lower bytes [3:0] unused (WSTRB = 8'b1111_0000)
```

9.6.2 Unaligned Access Handling

When a master issues an access that isn't aligned to its own width, there are three ways to play it:

```
Master: 32-bit, unaligned access at 0x1002
Problem: Not aligned to 4-byte boundary

Options:
1. Error response (strict mode)
2. Round down address, use WSTRB for actual bytes
3. Split into multiple aligned accesses

Bridge default: Option 2 (use WSTRB)
```

9.7 2.6.7 Burst Length Adjustment

9.7.1 Length Calculation

The AXI length field is beats-minus-one, so the conversion works out like this:

```
Formula:
  New_Length = (Old_Length + 1) × (Old_Width / New_Width) - 1

Example: Upsizing 32→64
  Old: AWLEN = 7 (8 beats), WIDTH = 32
  New: AWLEN = (7+1) × (32/64) - 1 = 8 × 0.5 - 1 = 3 (4 beats)

Example: Downsizing 128→64
  Old: ARLEN = 3 (4 beats), WIDTH = 128
  New: ARLEN = (3+1) × (128/64) - 1 = 4 × 2 - 1 = 7 (8 beats)
```

9.7.2 Odd Burst Lengths

When the burst doesn't divide evenly, the last wide beat runs partial and the strobes say which bytes are real:

```
Example: 3 beats of 32-bit → 64-bit
  3 beats × 4 bytes = 12 bytes total
  12 bytes / 8 bytes per beat = 1.5 beats

Solution:
  - Beat 0: Full 64-bit (8 bytes)
  - Beat 1: Partial 64-bit (4 bytes, WSTRB = 8'b0000_1111)
  - AWLEN = 1 (2 beats)
```

9.8 2.6.8 Resource Utilization

9.8.1 Per-Converter Resources

32→64 Upsizer:

```
Logic Elements:  ~300 LEs
Registers:       ~100 regs (buffering + FSM)
Block RAM:       0

Breakdown:
- Data buffer (32 bits): ~32 regs
- Strobe buffer: ~4 regs
- Beat counter: ~8 regs
- Control FSM: ~50 LEs, ~20 regs
- MUX logic: ~150 LEs
```

64→32 Downsizer:

```
Logic Elements:  ~250 LEs
Registers:       ~120 regs
Block RAM:       0

Similar structure but includes split logic
```

128→64 Downsizer:

```
Logic Elements:  ~400 LEs
Registers:       ~180 regs
Block RAM:       0

Larger data paths, more complex MUX
```

9.8.2 Scaling

Three things drive the cost:

- **Width ratio**: 2:1 vs. 4:1 vs. 8:1 conversion
- **Data width**: 128-bit vs. 256-bit buffers
- **Buffering depth**: Single vs. multi-beat buffering

9.9 2.6.9 Timing Characteristics

9.9.1 Latency

Upsizing (combining beats): - Buffering latency: 1-2 cycles (wait for N narrow beats) - First beat output: After receiving required narrow beats - Example: 32→64 requires 2 master beats before 1 crossbar beat

Downsizing (splitting beats): - Minimal latency: 1 cycle (register stage) - First beat output: Immediately (split from wide beat) - Example: 128→64 outputs first 64-bit beat immediately

9.9.2 Throughput

Upsizing Throughput:

```
32→64 upsizing:
  Master: 32 bits/cycle = 4 bytes/cycle
  Crossbar: 64 bits per 2 cycles = 4 bytes/cycle (same)
  No throughput loss
```

Downsizing Throughput:

```
128→64 downsizing:
  Master: 128 bits/cycle = 16 bytes/cycle
  Crossbar: 64 bits/cycle = 8 bytes/cycle
  Throughput halved (crossbar becomes bottleneck)
```

Upsizing is free; downsizing costs you the width ratio. Keep that asymmetry in mind when you pick port widths.

9.10 2.6.10 Configuration Parameters

9.10.1 Width Conversion Configuration (TOML)

```toml
[bridge]
# internal_data_width: NOT A KEY -- there is no fixed internal
crossbar width.
enable_width_conversion = true  # Allow width differences

[[bridge.masters]]
name = "cpu_32bit"
data_width = 32                 # Narrower than crossbar (upsizing)
addr_width = 32

[[bridge.masters]]
name = "dma_64bit"
data_width = 64                 # Matches crossbar (no conversion)
addr_width = 32

[[bridge.masters]]
name = "gpu_128bit"
data_width = 128                # Wider than crossbar (downsizing)
addr_width = 36

[[bridge.slaves]]
name = "memory_64bit"
data_width = 64                 # Matches crossbar (no conversion)

[[bridge.slaves]]
name = "periph_32bit"
data_width = 32                 # Narrower than crossbar (downsizing)
```

9.11 2.6.11 Debug and Observability

9.11.1 Recommended Debug Signals

When a conversion path misbehaves, these are the signals to pull into the waveform first:

```
Upsizer:
- Beat accumulator (partial wide beat being assembled)
- Beat counter (position in burst)
- Buffer valid (accumulated beats ready)
- Strobe accumulation

Downsizer:
- Beat splitter state (which sub-beat currently outputting)
- Remaining beats to output
- Source data register
```

9.11.2 Common Issues and Debug

Symptom: Data corruption after width conversion
Check: - Byte ordering (endianness) - Strobe mapping (correct bytes enabled) - Address alignment - Burst length calculation

Symptom: Stalls after width conversion
Check: - Buffer depths (upsizer needs buffering) - Backpressure propagation - LAST signaling

Symptom: Incorrect burst lengths
Check: - Length calculation formula - Odd burst handling - SIZE field matching width

9.12 2.6.12 Verification Considerations

9.12.1 Test Scenarios

1. **Power-of-2 Width Ratios**:

```
- 32→64 (2:1)
- 64→128 (1:2)
- 32→128 (4:1)
```

2. **Various Burst Lengths**:

```
- Single beat (ALEN=0)
- Even bursts (ALEN=3, 7, 15)
- Odd bursts (ALEN=1, 5, 9)
```

3. **Sub-Word Accesses**:

```
- Byte writes (WSTRB with individual bytes)
- Half-word, word accesses
- Unaligned accesses
```

4. **Mixed Widths**:

```
- 32-bit master → 64-bit crossbar → 32-bit slave
- 128-bit master → 64-bit crossbar → 32-bit slave
- All width combinations
```

9.13 2.6.13 Performance Considerations

9.13.1 Conversion Overhead

Upsizing Overhead: - Buffering latency: +1-2 cycles - Throughput maintained - Best for burst-oriented masters

Downsizing Overhead: - Split latency: +1 cycle - Throughput reduced by width ratio - Can bottleneck wide masters

9.13.2 Optimization Strategies

1. **Match Common Widths**: match a master to its slave's width, so no converter is inserted on that path. There is no single internal width to match.
2. **Burst Sizing**: Use appropriate burst lengths for width ratios
3. **Parallel Paths**: Multiple 32-bit masters can aggregate to 64-bit crossbar bandwidth
4. **Selective Conversion**: Only convert where necessary

9.14 2.6.14 Future Enhancements

9.14.1 Planned Features

- **Configurable Internal Width**: Support 32, 64, 128, 256-bit crossbar
- **Multi-Beat Buffering**: Deeper upsizing buffers for smoother flow
- **Byte Rotation**: Handle unaligned access more efficiently
- **Pipelined Conversion**: Multi-stage for high frequency

9.14.2 Under Consideration

- **Asymmetric Widths**: Different widths for read vs. write paths
- **Sparse Data Optimization**: Skip unused bytes in conversions
- **Tagged Data**: Metadata preservation through width conversion
- **Zero-Copy Bypass**: Direct routing for matching widths

Related Sections: - Section 2.1: Master Adapter (upsizing location) - Section 2.3: Crossbar Core (internal data width) - HAS ch06_integration/02_parameters.md (per-port data_width) - HAS ch06_integration/02_parameters.md (per-port data_width)

---

10 2.7 Protocol Conversion

The crossbar speaks AXI4 internally. Protocol conversion is what lets the bridge reach slaves that speak something simpler — APB (Advanced Peripheral Bus) for low-bandwidth peripherals being the common case.

10.1 2.7.1 Purpose and Function

Protocol conversion has five jobs:

1. **Protocol Translation**: Converts AXI4 transactions to target protocol (e.g., APB)
2. **Handshake Mapping**: Translates ready/valid to protocol-specific handshakes
3. **Burst Decomposition**: Breaks AXI bursts into single-beat target transactions
4. **Response Mapping**: Converts protocol-specific responses back to AXI responses
5. **Timing Adaptation**: Handles different timing requirements between protocols

10.2 2.7.2 Supported Protocols

10.2.1 Current Support (Phase 2)

AXI4 (Native): - Full AXI4 protocol - No conversion required - Maximum performance

AXI4-Lite (Master-Side): - Simplified AXI4 subset - Single-beat transactions only (ARLEN=0, AWLEN=0) - Converts to full AXI4 for crossbar - Common for control/status registers

APB (Slave-Side): - APB3 and APB4 support - For low-bandwidth peripherals - Simplified handshaking

APB and APB5 (Master-Side, bridge TASK-004 (was BRIDGE-014)): - The bridge is the APB completer; `apb4_to_axi4` / `apb5_to_axi4` (converters component) turn each transfer into one single-beat AXI4 transaction in front of the ordinary master timing wrapper - SLVERR and DECERR both fold to PSLVERR (APB has one error bit) - APB5: `PAUSER[0]`/`PWUSER[0]` ride the fabric's USER bit; `PWAKEUP` is accepted and terminated

AXI5-Lite (Master-Side, bridge TASK-004 (was BRIDGE-014)): - The AXI4-Lite path plus the AXI5-Lite ID and optional signals on the boundary - `awid`/`arid` -> the AXI4 ID (returned on `bid`/`rid`) and `user` -> the 1-bit USER fields ride the fabric; TRACE and POISON are terminated at the bridge top. AXI5-Lite has no LOCK, LOOP, MPAM, MECID or NSAID (IHI 0022L B2.1.5, GitHub #102 item 3)

Wishbone B4 (both sides, bridge TASK-008 (was BRIDGE-019)): - Slave port: `axi4_to_wb4` (the AXI4-Lite decomposers + `axil4_to_wb4`), one Wishbone transfer per AXI4 beat, ACK -> OKAY, ERR/RTY -> SLVERR - Master port: `wb4_to_axi4` (`wb4_to_axil4` + the single-beat AXI4 promotion) in front of the ordinary timing wrapper; SLVERR/DECERR -> ERR - B4 pipelined; `CTI`/`BTE` carried as CLASSIC/LINEAR, never acted on

AXI3 (both sides, GitHub #97): - Slave port: `axi4_to_axi3` converts the crossbar's AXI4 master face to an AXI3 slave face; same data width only. - Master port: `axi3_to_axi4` converts the external AXI3 master face to an AXI4 slave face, then the ordinary AXI4 timing wrapper takes over. - Long egress bursts are chopped, WID is stamped/stripped, and lock mapping follows the converter's documented encoding — see "AXI3 Boundary Conversion" below and the converters MAS §3.15/§3.16.

10.2.2 Current Limitation: AXI4-Lite Conversion

Superseded. This paragraph said AXIL slaves were treated as full AXI4 internally with no real conversion; the very next paragraph, and the RTL, say otherwise – `axi4_to_axil4_{rd,wr}.sv` perform genuine burst decomposition into single-beat AXI4-Lite transactions. Kept only so the contradiction is not silently deleted. The old text read: - Use the standard AXI4 timing wrapper (same as AXI4 slaves) - Expose full 5-channel AXI4 interface at the bridge boundary - Are documented as "axil" for user reference only

The bridge now emits real AXI4-to-AXIL4 conversion shims (`axi4_to_axil4_{rd,wr}.sv`) at the slave boundary when `protocol = "axil"` is specified in the TOML. These shims downgrade

bursts to single beats and enforce AXIL protocol constraints transparently. See “Generator-Emitted Conversion Shims” below.

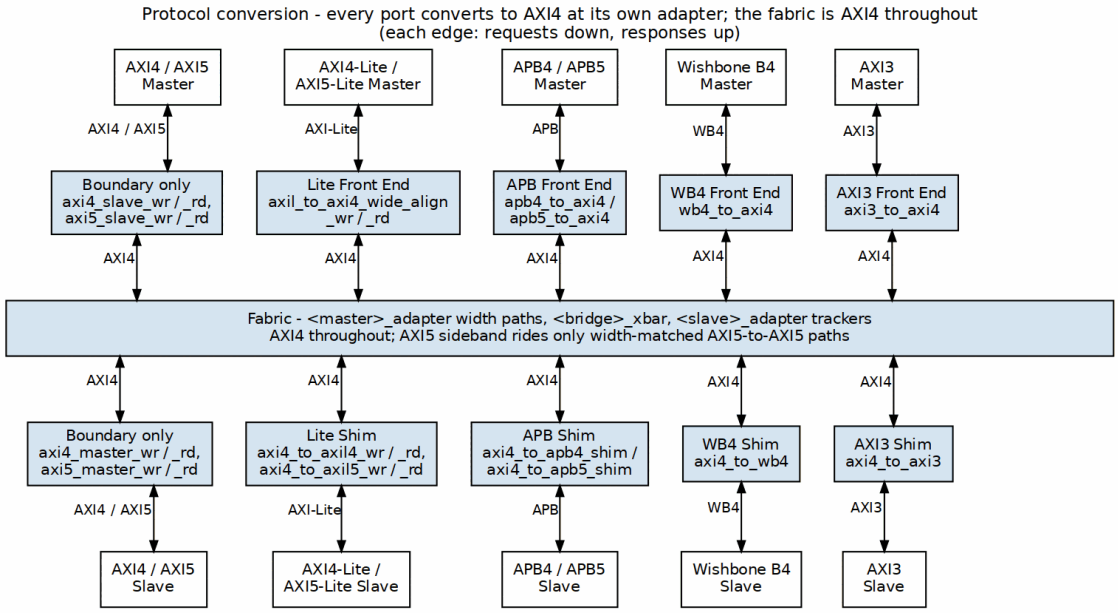
10.2.3 Future Support (Phase 2+)

AXI4-Lite conversion was listed here as future work while the section directly above states it is built and names the shims. It is built; the stale entry has been removed.

- **AHB**: Advanced High-performance Bus
- **Wishbone**: Open-source bus standard
- **Custom**: User-defined protocols

10.3 2.7.3 Conversion Architecture Overview

10.3.1 Figure 2.7: Protocol Conversion Architecture

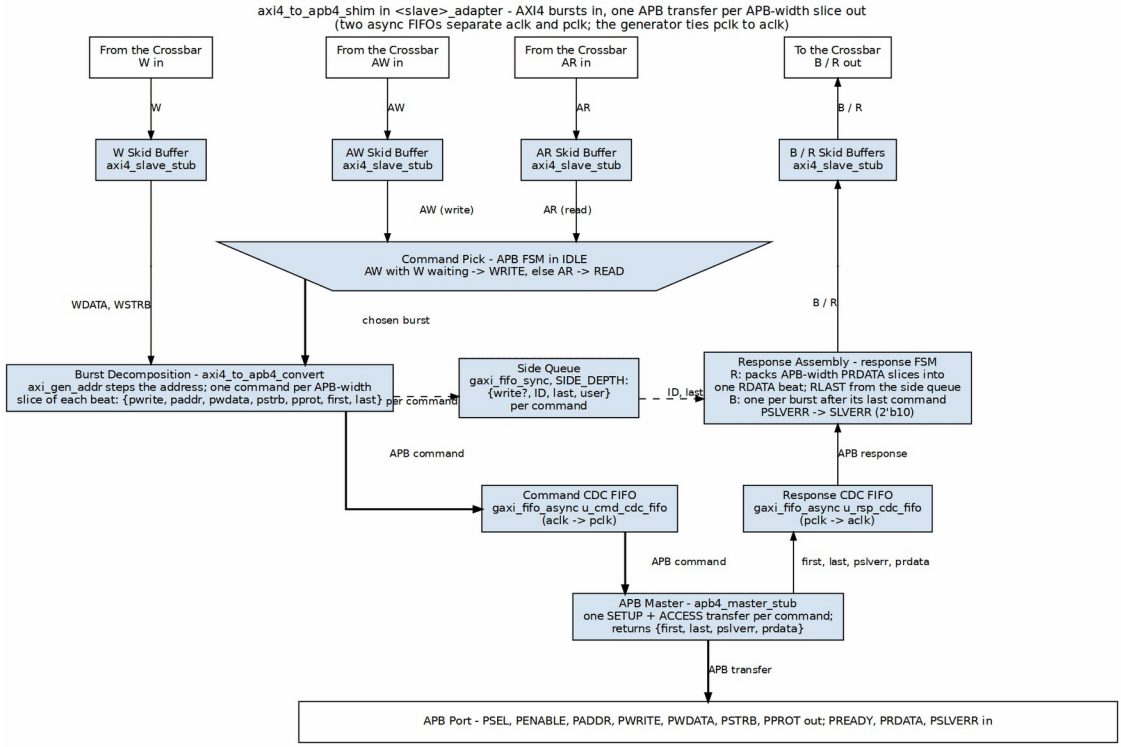


Protocol Conversion Architecture

Every port converts to AXI4 at its own adapter and the fabric is AXI4 throughout: AXI4-Lite, APB, Wishbone and AXI3 masters get a front-end converter in their master adapter, and the matching slaves get a shim in their slave adapter. AXI4 and AXI5 ports need only the boundary skid buffers.

10.4 2.7.4 Block Diagram

10.4.1 Figure 2.8: AXI4 to APB Shim



AXI4 to APB Shim

`axi4_to_apb4_shim` as an APB slave adapter instantiates it. The APB FSM picks a write (AW with W waiting) ahead of a read (the trapezium), `axi_gen_addr` decomposes the burst into one APB command per APB-width slice, and the commands cross to the APB master through an async FIFO. Responses return through a second async FIFO; the response FSM packs PRDATA into RDATA beats, issues one B per burst and maps PSLVERR to SLVERR. The generator ties `pclk` to `aclk`.

10.5 2.7.5 AXI4-Lite to AXI4 Conversion (Master-Side)

10.5.1 AXI4-Lite Protocol Overview

AXI4-Lite is a simplified subset of AXI4 aimed at simple control/status register access:

```
Key Differences from Full AXI4:
- FIXED burst length: ARLEN/AWLEN always = 0 (single beat)
- FIXED burst size: No SIZE field, always full data width
- FIXED burst type: No BURST field, always INCR
- NO exclusive access: No LOCK support
- NO unaligned transfers: Address must be aligned
- Simpler ID: Typically 1-4 bits (fewer outstanding transactions)

Similarities to AXI4:
- Same 5 channels: AR, R, AW, W, B
- Same valid/ready handshaking
- Response codes: OKAY, SLVERR, DECERR. NOT EXOKAY -- AXI4-Lite has no
  exclusive access, so the code has no meaning on this interface.
- Same data widths: 32 or 64 bits typically
```

10.5.2 Conversion Requirements

Adapting an AXI4-Lite master to the full AXI4 crossbar takes three things:

1. **Add Missing Signals**: Provide default values for burst-related signals
2. **Validate Constraints**: Ensure single-beat assumption holds
3. **Pass-Through Simplicity**: Most signals connect directly

10.5.3 Signal Mapping

```
// AXI4-Lite to AXI4 Signal Mapping

// AR Channel (Read Address)
// AXI4-Lite Input          AXI4 Crossbar Output
axi4lite_arvalid      →     axi4_arvalid
axi4lite_arready      ←     axi4_arready
axi4lite_araddr       →     axi4_araddr
axi4lite_arprot       →     axi4_arprot
axi4lite_arid (opt)   →     axi4_arid

// Added by adapter (constants):
                            axi4_arlen    = 8'h00      // Always 1
beat
                            axi4_arsize   = log2(DW/8) // Full width
                            axi4_arburst  = 2'b01      // INCR
                            axi4_arlock   = 1'b0       // No lock
```

```
                            axi4_arcache  = 4'b0000    // Device non-
buf
                            axi4_arqos    = 4'h0       // No QoS
                            axi4_arregion = 4'h0       // Region 0

// R Channel (Read Data)
// AXI4 Crossbar Input      AXI4-Lite Output
axi4_rvalid           →     axi4lite_rvalid
axi4_rready           ←     axi4lite_rready
axi4_rdata            →     axi4lite_rdata
axi4_rresp            →     axi4lite_rresp
axi4_rid (opt)        →     axi4lite_rid (opt)

// Discarded by adapter:
axi4_rlast                  // Always 1 for single beat

// AW Channel (Write Address)
axi4lite_awvalid      →     axi4_awvalid
axi4lite_awready      ←     axi4_awready
axi4lite_awaddr       →     axi4_awaddr
axi4lite_awprot       →     axi4_awprot
axi4lite_awid (opt)   →     axi4_awid

// Added by adapter:
                            axi4_awlen    = 8'h00
                            axi4_awsize   = log2(DW/8)
                            axi4_awburst  = 2'b01
                            axi4_awlock   = 1'b0
                            axi4_awcache  = 4'b0000
                            axi4_awqos    = 4'h0
                            axi4_awregion = 4'h0

// W Channel (Write Data)
axi4lite_wvalid       →     axi4_wvalid
axi4lite_wready       ←     axi4_wready
axi4lite_wdata        →     axi4_wdata
axi4lite_wstrb        →     axi4_wstrb

// Added by adapter:
                            axi4_wlast    = 1'b1       // Always last

// B Channel (Write Response)
axi4_bvalid           →     axi4lite_bvalid
axi4_bready           ←     axi4lite_bready
axi4_bresp            →     axi4lite_bresp
axi4_bid (opt)        →     axi4lite_bid (opt)
```

10.5.4 Implementation

The adapter itself is almost embarrassingly simple — mostly wires, with constants tied off for everything AXI4-Lite doesn't have:

```
// AXI4-Lite to AXI4 Adapter (simplified)
module axi4lite_to_axi4_adapter #(
    parameter ADDR_WIDTH = 32,
    parameter DATA_WIDTH = 64,
    parameter ID_WIDTH = 4          // Optional, often 0 for AXI4-Lite
) (
    input logic clk,
    input logic rst_n,

    // AXI4-Lite Master Interface
    input  logic                    lite_arvalid,
    output logic                    lite_arready,
    input  logic [ADDR_WIDTH-1:0]   lite_araddr,
    input  logic [2:0]              lite_arprot,
    input  logic [ID_WIDTH-1:0]     lite_arid,    // Optional

    output logic                    lite_rvalid,
    input  logic                    lite_rready,
    output logic [DATA_WIDTH-1:0]   lite_rdata,
    output logic [1:0]              lite_rresp,
    output logic [ID_WIDTH-1:0]     lite_rid,     // Optional

    input  logic                    lite_awvalid,
    output logic                    lite_awready,
    input  logic [ADDR_WIDTH-1:0]   lite_awaddr,
    input  logic [2:0]              lite_awprot,
    input  logic [ID_WIDTH-1:0]     lite_awid,    // Optional

    input  logic                    lite_wvalid,
    output logic                    lite_wready,
    input  logic [DATA_WIDTH-1:0]   lite_wdata,
    input  logic [DATA_WIDTH/8-1:0] lite_wstrb,

    output logic                    lite_bvalid,
    input  logic                    lite_bready,
    output logic [1:0]              lite_bresp,
    output logic [ID_WIDTH-1:0]     lite_bid,     // Optional

    // Full AXI4 Crossbar Interface
    output logic                    axi4_arvalid,
    input  logic                    axi4_arready,
    output logic [ADDR_WIDTH-1:0]   axi4_araddr,
    output logic [7:0]              axi4_arlen,
    output logic [2:0]              axi4_arsize,
```

```
    output logic [1:0]              axi4_arburst,
    output logic                    axi4_arlock,
    output logic [3:0]              axi4_arcache,
    output logic [2:0]              axi4_arprot,
    output logic [3:0]              axi4_arqos,
    output logic [3:0]              axi4_arregion,
    output logic [ID_WIDTH-1:0]     axi4_arid,

    input  logic                    axi4_rvalid,
    output logic                    axi4_rready,
    input  logic [DATA_WIDTH-1:0]   axi4_rdata,
    input  logic [1:0]              axi4_rresp,
    input  logic                    axi4_rlast,
    input  logic [ID_WIDTH-1:0]     axi4_rid,

    // ... (AW, W, B channels similar)
);

    // AR Channel: Pass-through with constants
    assign axi4_arvalid  = lite_arvalid;
    assign lite_arready  = axi4_arready;
    assign axi4_araddr   = lite_araddr;
    assign axi4_arprot   = lite_arprot;
    assign axi4_arid     = lite_arid;

    // Constants for single-beat burst
    assign axi4_arlen    = 8'h00;                    // 1 beat
    assign axi4_arsize   = $clog2(DATA_WIDTH/8);     // Full width
    assign axi4_arburst  = 2'b01;                    // INCR
    assign axi4_arlock   = 1'b0;                     // No lock
    assign axi4_arcache  = 4'b0000;                  // Device non-buf
    assign axi4_arqos    = 4'h0;                     // No QoS
    assign axi4_arregion = 4'h0;                     // Region 0

    // R Channel: Pass-through, ignore rlast
    assign lite_rvalid = axi4_rvalid;
    assign axi4_rready = lite_rready;
    assign lite_rdata  = axi4_rdata;
    assign lite_rresp  = axi4_rresp;
    assign lite_rid    = axi4_rid;
    // axi4_rlast ignored (always 1 for single beat)

    // AW Channel: Similar to AR
    assign axi4_awvalid  = lite_awvalid;
    assign lite_awready  = axi4_awready;
    assign axi4_awaddr   = lite_awaddr;
    assign axi4_awprot   = lite_awprot;
    assign axi4_awid     = lite_awid;
```

```
    assign axi4_awlen    = 8'h00;
    assign axi4_awsize   = $clog2(DATA_WIDTH/8);
    assign axi4_awburst  = 2'b01;
    assign axi4_awlock   = 1'b0;
    assign axi4_awcache  = 4'b0000;
    assign axi4_awqos    = 4'h0;
    assign axi4_awregion = 4'h0;

    // W Channel: Pass-through, add wlast
    assign axi4_wvalid = lite_wvalid;
    assign lite_wready = axi4_wready;
    assign axi4_wdata  = lite_wdata;
    assign axi4_wstrb  = lite_wstrb;
    assign axi4_wlast  = 1'b1;                          // Always last

    // B Channel: Pass-through
    assign lite_bvalid = axi4_bvalid;
    assign axi4_bready = lite_bready;
    assign lite_bresp  = axi4_bresp;
    assign lite_bid    = axi4_bid;

endmodule
```

10.5.5 Resource Utilization

AXI4-Lite Adapter Resources:

```
Logic Elements:  ~50-100 LEs (minimal, mostly wiring)
Registers:       ~50 regs (if skid buffers added)
Block RAM:       0

Breakdown:
- Signal pass-through: ~20 LEs (buffering)
- Constant generation: ~10 LEs
- Optional skid buffers: ~50 regs (for timing)

Note: Most implementations are purely combinatorial wire
      assignments with optional pipeline registers.
```

10.5.6 Performance Impact

Latency: - **Zero-latency** (combinatorial) if no pipeline stages - **1-2 cycles** if skid buffers added for timing - No protocol conversion overhead

Throughput: - **1 transaction per cycle** (same as native AXI4) - No degradation for single-beat transactions - Limited by AXI4-Lite's single-beat constraint

10.5.7 Configuration

```
[[bridge.masters]]
name       = "control_processor"
prefix     = "cpu_axil_"       # required -- prefixes every external
signal
protocol   = "axil"            # NOT "axi4lite"; see the protocol
table in the HAS
channels   = "rw"              # "rw" | "rd" | "wr"
id_width   = 0                 # AXI4-Lite carries no ID
addr_width = 32
data_width = 32
user_width = 1
```

Mind the field names: they are `id_width` / `addr_width` / `data_width` – one `id_width` per port, not the `arid_width` / `awid_width` pair an earlier revision of this page showed, which the loader does not read. The table is `[[bridge.masters]]`, not `[[masters]]`. Compare `bin/test_configs/bridge_1x5_wr_axil.toml` for a config that actually generates.

10.5.8 Common Issues and Debug

Issue 1: Burst Detected on AXI4-Lite

```
Symptom: ARLEN/AWLEN != 0 on AXI4-Lite interface
Cause: Master not properly configured as AXI4-Lite
Check: Verify master only issues single-beat transactions
```

Issue 2: Unaligned Addresses

```
Symptom: ARADDR/AWADDR not aligned to data width
Cause: AXI4-Lite requires full-width aligned access
Check: Address[log2(DW/8)-1:0] should be zero
```

Issue 3: rlast/wlast Handling

```
Symptom: Master expects rlast/wlast but doesn't have them
Cause: True AXI4-Lite interface omits these signals
Solution: Adapter provides wlast=1 to crossbar, strips rlast
```

10.6 2.7.6 AXI4 to APB Conversion (Slave-Side)

10.6.1 APB Protocol Overview

APB is a simple, low-power bus protocol — it earns its keep on area and power, not speed:

```
Characteristics:
- Single address phase
- Single data phase
- No burst support (one transfer per operation)
- Minimal logic
- Low power consumption
- Suitable for peripherals: UARTs, timers, GPIOs
```

10.6.2 APB Signals

```
Address Phase:
  PADDR[N-1:0]  - Address bus
  PSEL          - Slave select
  PENABLE       - Enable (2nd cycle of transfer)
  PWRITE        - Write direction (1=write, 0=read)

Data Phase (Write):
  PWDATA[N-1:0] - Write data
  PSTRB[N/8-1:0]- Write strobes (APB4 only)

Data Phase (Read):
  PRDATA[N-1:0] - Read data

Response:
  PREADY        - Slave ready (can extend transfer)
  PSLVERR       - Slave error (APB3+)
```

10.6.3 APB State Machine

Every APB transfer is a 2-phase handshake — SETUP, then ACCESS:

```
IDLE:
  - Wait for AXI request (ARVALID or AWVALID)
  - PSEL = 0, PENABLE = 0

SETUP:
  - Assert PSEL = 1
  - Drive PADDR, PWRITE, PWDATA (if write)
  - PENABLE = 0
  - Duration: 1 cycle

ACCESS:
  - Assert PENABLE = 1
```

```
  - Wait for PREADY = 1
  - Capture PRDATA (if read) or PSLVERR
  - Can extend multiple cycles if PREADY = 0

Complete:
  - De-assert PSEL, PENABLE
  - Return to IDLE or SETUP (if more beats)
```

10.6.4 Read Transaction Conversion

AXI4 Read:

```
Cycle 0: ARVALID=1, ARADDR=0x100, ARLEN=3 (4 beats)
Cycle 1: ARREADY=1
Cycles 2-5: R beats returning
```

Converted to APB (4 separate APB reads):

```
Beat 0:
  Cycle 0: PSEL=1, PENABLE=0, PADDR=0x100, PWRITE=0 (SETUP)
  Cycle 1: PSEL=1, PENABLE=1, wait PREADY (ACCESS)
  Cycle 2: PREADY=1, capture PRDATA → First R beat

Beat 1:
  Cycle 3: PSEL=1, PENABLE=0, PADDR=0x104 (SETUP)
  Cycle 4: PSEL=1, PENABLE=1, wait PREADY (ACCESS)
  Cycle 5: PREADY=1, capture PRDATA → Second R beat

... (beats 2 and 3 similar)
```

Latency: 2-3 cycles per beat (SETUP + ACCESS + ready)

10.6.5 Write Transaction Conversion

AXI4 Write:

```
Cycle 0: AWVALID=1, AWADDR=0x200, AWLEN=1 (2 beats)
Cycle 1: AWREADY=1
Cycle 1: WVALID=1, WDATA=0xAAAA_BBBB, WLAST=0
Cycle 2: WREADY=1
Cycle 2: WVALID=1, WDATA=0xCCCC_DDDD, WLAST=1
Cycle 3: WREADY=1
Cycle 4: BVALID=1, BRESP=OKAY
```

Converted to APB (2 separate APB writes):

```
Beat 0:
  Cycle 0: PSEL=1, PENABLE=0, PADDR=0x200, PWRITE=1,
PWDATA=0xAAAA_BBBB
  Cycle 1: PSEL=1, PENABLE=1, wait PREADY
```

```
  Cycle 2: PREADY=1 → First write complete

Beat 1:
  Cycle 3: PSEL=1, PENABLE=0, PADDR=0x204, PWRITE=1,
PWDATA=0xCCCC_DDDD
  Cycle 4: PSEL=1, PENABLE=1, wait PREADY
  Cycle 5: PREADY=1 → Second write complete, return B
```

10.6.6 Burst Handling

APB has no burst concept, so the converter decomposes:

```
AXI Burst: AWLEN = 15 (16 beats)
→ 16 separate APB transfers
→ Address increments per AWBURST type:
   - INCR: Addr += SIZE each beat
   - WRAP: Wrapping within boundary
   - FIXED: Same address each beat
```

10.6.7 Response Mapping

```
APB → AXI Response Translation:

PREADY=1, PSLVERR=0 → RRESP/BRESP = 2'b00 (OKAY)
PREADY=1, PSLVERR=1 → RRESP/BRESP = 2'b10 (SLVERR)

PREADY stuck at 0:
  The converter WAITS. There is no timeout and no DECERR.
```

There is no PREADY timeout anywhere in the path. `apb4_master` waits unconditionally in ACCESS (`if (m_apb_PREADY) ...` with no counter), and neither `axi4_to_apb4_convert`, `axi4_to_apb4_shim` nor `axi4_to_apb5_shim` contains a threshold register or cycle counter. A slave that never asserts PREADY stalls that path indefinitely, and the stall propagates back through the crossbar as backpressure. Budget for it in the system, or put a watchdog outside the bridge; the bridge will not manufacture an error response.

10.7 2.7.7 Implementation

10.7.1 AXI4-to-APB Converter FSM

```
// Simplified AXI4-to-APB converter
typedef enum logic [2:0] {
    IDLE,
    AR_SETUP,
    AR_ACCESS,
    AW_SETUP,
    W_ACCESS,
    B_RESPONSE
} state_t;

state_t state, next_state;

always_ff @(posedge clk) begin
    if (!rst_n) state <= IDLE;
    else state <= next_state;
end

always_comb begin
    next_state = state;

    case (state)
        IDLE: begin
            if (arvalid) next_state = AR_SETUP;
            else if (awvalid) next_state = AW_SETUP;
        end

        AR_SETUP: begin
            next_state = AR_ACCESS;  // 1 cycle SETUP
        end

        AR_ACCESS: begin
            if (pready) begin
                if (more_beats) next_state = AR_SETUP;  // Next beat
                else next_state = IDLE;
            end
        end

        AW_SETUP: begin
            if (wvalid) next_state = W_ACCESS;
        end

        W_ACCESS: begin
            if (pready) begin
                if (!wlast) next_state = AW_SETUP;  // Next beat
```

```
                else next_state = B_RESPONSE;
            end
        end

        B_RESPONSE: begin
            if (bready) next_state = IDLE;
        end
    endcase
end

// APB signal generation
assign psel = (state != IDLE);
assign penable = (state == AR_ACCESS || state == W_ACCESS);
assign pwrite = (state == AW_SETUP || state == W_ACCESS);
```

10.7.2 Address Generation

```
// Address increment for burst
logic [ADDR_WIDTH-1:0] current_addr;
logic [7:0] beat_count;

always_ff @(posedge clk) begin
    if (state == IDLE) begin
        current_addr <= arvalid ? araddr : awaddr;
        beat_count <= 0;
    end else if ((state == AR_ACCESS || state == W_ACCESS) && pready)
begin
        beat_count <= beat_count + 1;

        case (burst_type)
            2'b01: current_addr <= current_addr + (1 << size);  //
INCR
            2'b10: current_addr <= wrap_address(current_addr);  //
WRAP
            2'b00: current_addr <= current_addr;                //
FIXED
        endcase
    end
end

assign paddr = current_addr;
```

10.8 2.7.8 Resource Utilization

10.8.1 APB Converter Resources

Per APB Slave Interface:

```
Logic Elements:  ~400 LEs
Registers:       ~150 regs
Block RAM:       0

Breakdown:
- FSM control:           ~100 LEs, ~20 regs
- Address generation:    ~80 LEs, ~40 regs
- Burst counter:         ~50 LEs, ~20 regs
- Response accumulation: ~80 LEs, ~30 regs
- Data path MUX:         ~90 LEs, ~40 regs
```

10.8.2 Scaling

Adding APB slaves: - Linear scaling: +~400 LEs per APB slave - Shared address decoder logic - Independent per-slave FSMs

10.9 2.7.9 Timing Characteristics

10.9.1 Latency

APB Read Latency (per beat):

```
Best case: 2 cycles (SETUP + ACCESS with PREADY=1)
Typical: 3-5 cycles (if slave extends with PREADY=0)
Worst case: unbounded -- the converter waits for PREADY with no
timeout

For 8-beat AXI burst:
  Total: 8 × 3 = 24 cycles typical
```

APB Write Latency (per beat):

```
Similar to read: 2-5 cycles per beat
```

10.9.2 Throughput

Severely Limited:

```
APB: ~0.3-0.5 transactions/cycle (due to 2-3 cycle protocol)
AXI4: 1 transaction/cycle (burst mode)

APB suitable only for low-bandwidth peripherals
```

No amount of buffering fixes that — the 2-3 cycle protocol itself is the bottleneck.

10.10 2.7.10 Configuration Parameters

10.10.1 Protocol Conversion Configuration (TOML)

```toml
[[bridge.slaves]]
name = "uart_peripheral"
protocol = "apb"             # "axi4", "apb", "ahb" (future)
base_address = 0xF000_0000
size = 0x1000
data_width = 32

[[bridge.slaves]]
name = "ddr_memory"
protocol = "axi4"            # Native, no conversion
base_address = 0x8000_0000
size = 0x4000_0000
data_width = 64
```

10.11 2.7.11 Debug and Observability

10.11.1 Recommended Debug Signals

```
APB Converter:
- FSM state
- APB phase (SETUP, ACCESS)
- Beat counter (progress through burst)
- Response accumulation (for burst)

APB Bus:
- PSEL, PENABLE, PWRITE
- PADDR, PWDATA, PRDATA
- PREADY, PSLVERR
```

10.11.2 Common Issues and Debug

Symptom: APB slave not responding (timeout)
Check: - PREADY signal (stuck at 0?) - APB slave clock/reset - PSEL assertion (There is no timeout threshold to check – the converter waits forever.)

Symptom: Data corruption on APB
Check: - SETUP phase duration (should be 1 cycle) - PENABLE assertion timing - Data sampling on correct cycle

Symptom: Burst to APB takes too long
Check: - Burst length (consider limiting ARLEN/AWLEN) - APB slave response time (PREADY) - Alternative: Use AXI4 slave instead

10.12 2.7.12 Verification Considerations

10.12.1 Test Scenarios

1. **Single APB Transfer**:

```
- AXI ARLEN=0 (1 beat) → 1 APB read
- Verify SETUP → ACCESS sequence
- Check PREADY handling
```

2. **APB Burst Decomposition**:

```
- AXI AWLEN=7 (8 beats) → 8 APB writes
- Verify address increment
- Check each beat completes before next
```

3. **APB PREADY Extension**:

```
- Slave holds PREADY=0 for N cycles
- Verify converter waits
- Check no data corruption
```

4. **APB Error Response**:

```
- Slave asserts PSLVERR
- Verify mapped to AXI SLVERR
- Check error propagated to master
```

5. **APB stall (no timeout to test)**:

```
- Slave never asserts PREADY
- Verify the path stalls and backpressures cleanly, with no lost or
  duplicated beats once PREADY finally arrives
- There is NO timeout and NO DECERR to check for -- see 2.7.6
```

10.13 2.7.13 Performance Considerations

10.13.1 When to Use APB

Good Use Cases: - Low-speed peripherals (UART, GPIO, timers) - Infrequent accesses - Simple register interfaces - Power-sensitive designs

Poor Use Cases: - High-bandwidth devices - Burst-intensive masters - Performance-critical paths - Memory interfaces

10.13.2 APB vs. AXI4 Comparison

Feature	APB	AXI4
Complexity	Simple	Complex
Throughput	Low (~0.3/cyc)	High (1/cyc burst)
Latency/beat	2-5 cycles	1 cycle
Burst Support	No	Yes (up to 256)
Resources	~400 LEs	Native (no converter)
Power	Very low	Moderate
Use Case	Peripherals	Memory, DMA

10.14 2.7.14 Mixed Protocol Bridges

10.14.1 Example Configuration

```toml
# Bridge with mixed protocols
[bridge]
num_masters = 2
num_slaves = 3

[[bridge.masters]]
name = "cpu"
protocol = "axi4"

[[bridge.masters]]
name = "dma"
protocol = "axi4"

[[bridge.slaves]]
name = "ddr_memory"
protocol = "axi4"          # High bandwidth

[[bridge.slaves]]
name = "sram"
protocol = "axi4"          # Medium bandwidth

[[bridge.slaves]]
name = "peripherals"
protocol = "apb"           # Low bandwidth, simple
```

10.14.2 Routing Optimization

Walk the routes and you can see exactly where the conversion cost lands:

```
CPU → DDR Memory: AXI4-to-AXI4 (native, fast)
CPU → Peripherals: AXI4-to-APB (converted, slower)
DMA → SRAM: AXI4-to-AXI4 (native, fast)
DMA → Peripherals: AXI4-to-APB (rare, acceptable slowdown)
```

10.15 2.7.15 Master-Side vs Slave-Side Conversion

10.15.1 Comparison

Feature	Master-Side (AXI4-Lite)	Slave-Side (APB)
Complexity	Very Simple	Complex
Resource Usage	~50-100 LEs	~400 LEs
Latency Added	0-1 cycles	2-5 cycles/beat
Throughput Impact	None	Severe (3x slower)
Burst Handling	Single beat only	Decompose to singles
State Machine	None (combinatorial)	Multi-state FSM
Buffering Required	Optional (timing)	Essential
Use Case	Control registers	Peripherals

10.15.2 When to Use Each

AXI4-Lite Master Adapter: - Simple control/status register interfaces - Low-complexity masters (MCUs, simple CPUs) - Minimal resource overhead acceptable - No burst performance needed

APB Master Front End (`apb{4,5}_to_axi4`, bridge TASK-004 (was BRIDGE-014)): - An APB requester that needs to reach AXI4 or AXI4-Lite completers - One-outstanding by nature of APB: one transfer per fabric round trip - Same lane behaviour as an AXI4-Lite master toward wider slaves (the aligner is shared)

APB Slave Converter: - Legacy peripheral integration - Very simple slave devices (GPIO, timers) - Low-bandwidth acceptable - Power optimization critical

10.16 2.7.16 Generator-Emitted Conversion Shims

You don't instantiate any of these converters by hand. The bridge generator automatically emits protocol conversion shims at the slave boundary based on the TOML configuration. These shims are instantiated between the crossbar core (uniformly AXI4 internally) and the external slave port.

10.16.1 AXI4 to AXI4-Lite Conversion

When `protocol = "axil"` is specified in the slave TOML: - Generator emits `axi4_to_axil4_rd.sv` (read path) and `axi4_to_axil4_wr.sv` (write path) - Shims downgrade bursts to single beats (set ARLEN/AWLEN = 0 on output) - Enforce single-beat semantics transparently - Data width remains unchanged (upsizing/downsizing done separately) - Shims are inserted **between crossbar and slave port** in the generated top-level module

Modules:
`projects/components/utility-ip/converters/rtl/axi4_to_axil4_{rd,wr}.sv`

10.16.2 AXI4 to APB Conversion

When `protocol = "apb"` is specified in the slave TOML: - Generator emits ONE shim, `axi4_to_apb4_shim` – a direct full-AXI4-to-APB4 converter. There is no AXI4-Lite stage: the shim does its own burst decomposition (`r_burst_count` + `axi_gen_addr`), so nothing upstream has to reduce the burst first. - `protocol = "apb5"` emits `axi4_to_apb5_shim`, a sideband wrapper over that same shim – see AMBA5 Boundary. - Slave port externally presents APB signals; internally the crossbar is AXI4

Modules: `projects/components/utility-ip/converters/rtl/axi4_to_apb4_shim.sv` (`axi4_to_apb5_shim.sv` for `apb5`)

An earlier revision of this page described the path as an `axi4_to_axil4` shim followed by an internal AXIL-to-APB bridge chain. No such chain exists, and no generated APB slave adapter instantiates `axi4_to_axil4`.

10.16.3 Master-Side APB Front End

When `protocol = "apb"` or `"apb5"` is specified on a **master** (bridge TASK-004, was BRIDGE-014): - The master adapter's external surface is the APB completer set (the requester's `PSEL/PENABLE/PADDR/PWRITE/PWDATA/PSTRB/PPROT` are inputs; `PREADY/PRDATA/PSLVERR` outputs; `apb5` adds `PAUSER/PWUSER/PWAKEUP` in and `PRUSER/PBUSER` out). - The adapter instantiates `apb4_to_axi4` / `apb5_to_axi4` on an internal AXI4 face (`apbx_axi_*`) and feeds that to the same `axi4_slave_{wr,rd}` timing wrapper – `_mon` in the monitored variant – an AXI4 master port gets. From the wrapper onward the port is an AXI4-Lite-shaped single-beat requester: decode, the width converters and the wide-slave aligner, and the response mux are untouched. - The fabric ID is the master index alone (`id_width = 0`, bridge TASK-005 (was BRIDGE-016)).

Modules: `projects/components/utility-ip/converters/rtl/apb4_to_axi4.sv`, `apb5_to_axi4.sv`, `apb_cmdrsp_to_axi4.sv` – see the converters MAS.

10.16.4 Wishbone B4 at Either Boundary

When `protocol = "wb4"` is specified (bridge TASK-008, was BRIDGE-019): - On a **slave**, the adapter instantiates `axi4_to_wb4` through the `Axi4ToWb4Shim` component – the same channel wiring, bridge BUG-009 (was BRIDGE-011) not-full gate and `converter_*` response intercepts as the APB shim, and the same `axi4_master_*_mon` sandwich in the monitored variant. The external surface is the thirteen-signal B4 requester set from `bridge_pkg/wb4_signals.py`, the one table the bridge top, the adapter and the instance component all read. - On a **master**, the adapter's external surface is the B4 completer set (directions reversed) and `wb4_to_axi4` drives an internal `wbx_axi_*` AXI4 face into the same `axi4_slave_{wr,rd}` wrapper an AXI4 master gets. From there the port is an AXI4-Lite-shaped single-beat requester and takes the wide-slave aligner toward wider slaves.

Modules: `projects/components/utility-ip/converters/rtl/axi4_to_wb4.sv`, `wb4_to_axi4.sv` – see the converters MAS.

10.16.5 AXI3 Boundary Conversion

When `protocol = "axi3"` is specified, a same-width protocol converter sits at the boundary and the crossbar behind it stays AXI4:

- **Slave boundary:** the generated adapter instantiates `axi4_to_axi3` between the crossbar core and the external AXI3 slave port. Egress AXI4 bursts longer than 16 beats are chopped into sequential AXI3 bursts; `WID` is stamped per beat; `QoS`/`REGION` are dropped; lock is mapped as the converter documents it.
- **Master boundary:** the adapter exposes an AXI3 completer surface and instantiates `axi3_to_axi4` in front of the ordinary AXI4 timing wrapper. `WID` is stripped, `QoS`/`REGION` are driven to zero, and lock is mapped as the converter documents it.

The bridge does not add width conversion inside the protocol converter. When an AXI3 port's external width differs from a target slave width, the existing internal width-adaptation path (`dwidth_converter` or the wide-slave aligner) operates on the AXI4 side behind the converter.

Converter-sourced obligations (for full wording see converters MAS §3.15 *AXI4 to AXI3 Converter* and §3.16 *AXI3 to AXI4 Converter*):

1. An AXI3 master must not interleave W data across IDs. `axi3_to_axi4_wr` asserts `ASSERT_A3_WID_STABLE` if `WID` changes mid-burst or differs from the active AWID.
2. Egress bursts longer than 16 beats are chopped by address regeneration; `INCR` and `FIXED` are supported, `WRAP` longer than 16 beats is rejected.
3. `QoS` and `REGION` are absent on AXI3 pins.

4. The converters are same-width only; width adaptation is the bridge's internal concern.
5. Lock mapping follows the ARM AXI3 encoding (ARM IHI 0022: `00` normal, `01` exclusive, `10` locked), as fixed by bridge GitHub #98:
   - `axi4_to_axi3`: `1'b0 → 2'b00`, `1'b1 → 2'b01`.
   - `axi3_to_axi4`: `2'b01 → 1'b1`; `2'b00`, `2'b10` (locked — AXI4 dropped locked accesses), and the reserved `2'b11 → 1'b0`.

Validation fixtures: `bridge_1x2_axi3`, `bridge_2x2_axi3_masters`, `bridge_2x2_axi3_slaves`, `bridge_mix_e`.

Modules: `projects/components/utility-ip/converters/rtl/axi4_to_axi3.sv`, `axi4_to_axi3_rd.sv`, `axi4_to_axi3_wr.sv`, `axi3_to_axi4.sv`, `axi3_to_axi4_rd.sv`, `axi3_to_axi4_wr.sv`.

10.16.6 Master-Side AXI5-Lite Sideband

When `protocol = "axil5"` is specified on a **master** (bridge TASK-004, was BRIDGE-014), the bridge top exposes the AXI4-Lite set plus the AXI5-Lite ID and every optional group (from `bridge_pkg/axil5_sideband.py`, the one table the slave side also reads, with the directions flipped for a requester). The enabled forwardable groups and the ID join the Lite surface and are wired into the adapter's AXI4 face (`awid`/`bid`/`arid`/`rid` always, `user` -> `aw/w/ar user`); the rest is terminated at the top – requester-driven inputs consumed by a reduction into an `_unused_<master>_axil5_sb` wire, completer-driven outputs driven `'0`. Response-side USER (`buser`/`ruser`) is connected but reads 0, because the master adapters tie response USER (PRD).

10.16.7 Master-Side AXIL→Wider-Slave Alignment

When an AXI4-Lite master interfaces with a wider AXI4 slave (e.g., 32-bit AXIL master to 64-bit AXI4 slave) – and likewise an AXI5-Lite or APB master, which present the same single-beat stream: - Generator emits `axil_to_axi4_wide_align_rd.sv` (read) and `axil_to_axi4_wide_align_wr.sv` (write) at the **master adapter output** (before crossbar core) - These modules handle **width alignment** (not protocol conversion — that's done at the slave boundary if needed) - Preserves AXIL's single-beat constraint on the master side - Properly aligns partial-word reads/writes on the wide slave side

Modules: `projects/components/utility-ip/converters/rtl/axil_to_axi4_wide_align_{rd,wr}.sv`

Example: 32-bit AXIL master → 64-bit AXI4 slave - Master writes to addr 0x04 with data 0xAABBCCDD. The shim selects the lane from `addr[2]` (= 1), so the data lands on the UPPER half: the slave sees `wdata[63:32] = 0xAABBCCDD`, `WSTRB = 0xF0`, at row-aligned address `0x00` – the lane bits move out of the address and into the strobe. Worked through in Width Converters. - Shim is inserted after master adapter, before crossbar

10.17 2.7.17 Future Protocol Support

10.17.1 Planned Features

AXI4-Lite – NOT future work; BUILT. `axi4_to_axil4_{rd,wr}` shims are emitted when `protocol = "axil"`, and `bridge_1x2_rw_axil5` and the `mix_*` configurations ship AXIL slaves with tests in the FULL regression. It was listed here as planned while the same page describes the shims that implement it.

AHB (AMBA High-performance Bus): - More capable than APB - Pipeline support - Burst support - Suitable for moderate-bandwidth peripherals

Wishbone – NOT future work; BUILT (bridge TASK-008 (was BRIDGE-019), 2026-09-11). `wb4` is a legal `protocol` on master and slave ports alike; see “Wishbone B4 at Either Boundary” above. Not built: B4 standard (classic) mode from the TOML, and registered-feedback bursts (`CTI`/`BTE` are carried, not generated).

10.17.2 Under Consideration

- **Custom Protocol**: User-defined through configuration
- **Stream Interface**: AXI4-Stream for data streaming
- **PCIe TLP**: For PCIe endpoint integration
- **CHI**: ARM’s Coherent Hub Interface

10.18 2.7.18 Best Practices

10.18.1 Design Recommendations

1. **Limit APB Burst Lengths**: Configure masters to use short bursts to APB slaves
2. **Watchdog Outside the Bridge**: the converter has no PREADY timeout, so a wedged APB slave stalls its path forever. If the system needs to survive that, the watchdog belongs upstream.
3. **Protocol Matching**: Use native AXI4 where possible, APB only when necessary
4. **Address Map Planning**: Group APB peripherals together for efficient decoding
5. **Width Matching**: Match APB data width to peripheral requirements

10.18.2 Performance Tips

```
Inefficient: 256-beat AXI burst → APB
  256 beats × 3 cyc/beat = 768 cycles

Better: Limit to 4-beat bursts → APB
  64 bursts of 4 beats each
  Still long, but more manageable

Best: Use AXI4-Lite slave for registers
  No burst, but native protocol
```

Related Sections: - Section 2.3: Crossbar Core (protocol integration point) - HAS ch04_interfaces/01_axi4_interface.md (port signals) - Chapter 4: Programming (configuring protocol conversion) - Appendix A: Generator Deep Dive (protocol converter generation)

---

11 2.8 Response Routing

Requests are easy to route — the address says where they go. Responses are harder: read data and write acknowledgments have to find their way back to whichever master asked, and by then the address that launched the transaction is ancient history. Response routing does that with Bridge IDs, demultiplexing logic, and optional CAM structures.

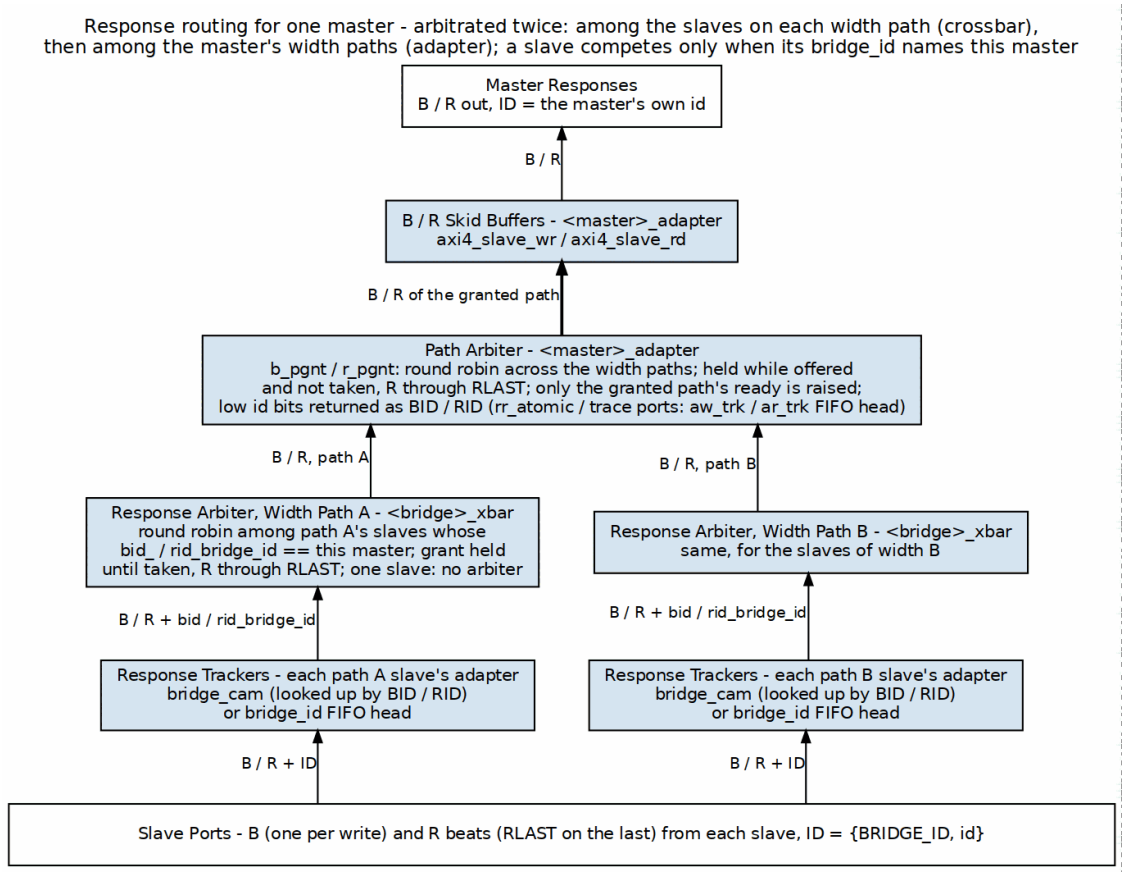
11.1 2.8.1 Purpose and Function

Response routing does five things:

1. **Response Direction**: Routes R and B channel responses to correct master
2. **Tracked Routing**: each slave adapter names the master to answer – `bridge_cam` looked up with the returned BID/RID on a multi-master AXI4/AXI5 slave, an in-order `bridge_id` FIFO elsewhere
3. **Multi-Slave Merging**: Combines responses from multiple slaves to single master
4. **Flow Control**: Manages backpressure from master on response channels
5. **Error Propagation**: Ensures error responses reach originating master

11.2 2.8.2 Block Diagram

11.2.1 Figure 2.9: Response Routing Architecture



Response Routing Architecture

Responses for one master climb from the slaves. Each slave adapter's tracker supplies the `bridge_id` of the master to answer; the crossbar presents that slave's B / R on this master's width path only when the `bridge_id` matches; and the master adapter takes the path at the head of its `aw_trk` / `ar_trk` FIFO. Nothing here arbitrates, so there is no trapezium.

11.3 2.8.3 Stable Response Routing via FIFO-Tracked Slave Select

11.3.1 Architecture: FIFO-Tracked Master Routing

> **Scope since the one-target limitation was fixed (GitHub #119 L1, 2026-10-15):** this FIFO steering is used only on master ports with a read-return atomic or a trace echo. Every other port has per-ID tracking and a path arbiter instead (MAS 2.1, and Response Arbitration below).

For multi-slave masters, responses are routed based on a **stable slave-select value** captured at request time, not the current address decode:

```
// At AW/AR time: Capture slave index
assign aw_slave_select = address_decode(fub_axi_awaddr);
always_ff @(posedge aclk) begin
    if (fub_axi_awvalid && fub_axi_awready) begin
        aw_trk_fifo <= aw_slave_select;  // Capture now
    end
end

// At B time: Use captured slave from FIFO (not current decode)
assign b_slave_select = aw_trk_fifo_out;  // Stable value
logic [NUM_MASTERS-1:0] master_bvalid;
always_comb begin
    master_bvalid = '0;
    master_bvalid[extracted_bid] = slave_bvalid && (b_slave_select ==
current_slave);
end
```

Benefits: - B-responses route to correct master even after address reverts - Handles multi-slave masters spanning different widths - Prevents response loss due to stale address decodes

11.3.2 BID Extraction

The master is not chosen by decoding the prefix bits of the returned ID. With more than one master the slave-side ID is `{BRIDGE_ID, id}` (bridge TASK-005), but the crossbar steers B / R by the master index the slave adapter's tracker returns beside the response (`bid_bridge_id` / `rid_bridge_id`). For a CAM-tracked slave that index is looked up BY the returned BID/RID; for a FIFO-tracked slave it is the FIFO head. The master adapter then strips the prefix:

```
// bridge_2x2_rw_xbar.sv: B toward cpu (master index 0)
((ddr_axi_bid_bridge_id == 0) && ddr_axi_bid_valid ? ddr_axi_bid : '0)
| ...

// cpu_adapter.sv: return the master's own ID bits
fub_axi_bid = cpu_32b_b.id[3:0];
```

11.3.3 CAM-Based Extraction

Built (bridge BUG-012 (was BRIDGE-015)/016, 2026-09-10). Every real AXI slave of a multi-master bridge, and any slave declared `enable_ooo = true`, gets `bridge_cam` for both channels and routes responses by ID; single-master bridges, shim slaves and the subtractive slave keep the in-order per-slave FIFO. Per-ID tracking is sound across masters because fabric IDs are `{master index, master id}`; the FIFO would require the slave to complete in request order across ALL IDs, which a compliant AXI slave need not do. Fixture `bridge_2x2_ooo` exercises reordering slaves with two masters sharing IDs; `test_bridge_cam.py` covers the CAM alone. The mode had been unbuildable since c64660f47. See ch04 `02_id_tracking.md`.

For complex configurations with OOO or ID reordering:

```
// CAM lookup for response routing
logic [TOTAL_RID_WIDTH-1:0] response_id;
logic [BID_WIDTH-1:0] master_index;
logic cam_hit;

// Search CAM for matching transaction
assign {cam_hit, master_index} = cam_lookup(response_id);

// Route response to found master
if (cam_hit) begin
    master_rvalid[master_index] = slave_rvalid;
end else begin
    // Error: No matching transaction (protocol violation)
    error_response();
end
```

11.4 2.8.4 Response Demultiplexing

11.4.1 Per-Master Response Paths

Each master has dedicated response channels from the demultiplexer:

```
Master 0 Response:
  - M0_RVALID, M0_RREADY, M0_RDATA, M0_RID, M0_RRESP, M0_RLAST
  - M0_BVALID, M0_BREADY, M0_BID, M0_BRESP

Source: Responses from any slave with BID=0
```

11.4.2 Demux Implementation

```
// Response demultiplexer (4 masters, 3 slaves)
// Combines responses from all slaves, routes to correct master

// Slave responses (multiple sources)
logic s0_rvalid, s1_rvalid, s2_rvalid;
logic [63:0] s0_rdata, s1_rdata, s2_rdata;
logic [5:0] s0_rid, s1_rid, s2_rid;  // Includes BID

// Master responses (multiple destinations)
logic m0_rvalid, m1_rvalid, m2_rvalid, m3_rvalid;
logic [63:0] m0_rdata, m1_rdata, m2_rdata, m3_rdata;
logic [3:0] m0_rid, m1_rid, m2_rid, m3_rid;  // BID stripped

// Demux logic per slave
for (genvar s = 0; s < 3; s++) begin
    logic [1:0] bid;
    assign bid = slave_rid[s][5:4];  // Extract BID

    always_comb begin
        case (bid)
            2'b00: begin
                if (slave_rvalid[s] && !m0_busy) begin
                    m0_rvalid = 1'b1;
                    m0_rdata = slave_rdata[s];
                    m0_rid = slave_rid[s][3:0];  // Strip BID
                end
            end
            2'b01: /* Route to M1 */
            2'b10: /* Route to M2 */
            2'b11: /* Route to M3 */
        endcase
    end
end
```

11.5 2.8.5 Multi-Slave Response Merging

11.5.1 Problem Statement

When multiple slaves can respond to the same master in the same cycle:

```
Scenario:
  Master 0 has outstanding transactions to Slave 0 and Slave 1
  Both slaves return R responses in same cycle

Problem: Master 0 can only accept one R response per cycle
Solution: Arbitrate between slave responses
```

11.5.2 Response Arbitration

Built 2026-10-15, fixing the one-target-per-master limitation (GitHub #119 L1). Responses are arbitrated at two levels:

1. **Crossbar, per master width path** (`<master>_<W>b_{b,r}_rarb_*`): the slaves on that path whose tracker names this master request. Round robin grants one. A grant offered and not taken holds, and an R grant holds through RLAST. The mux toward the master and the slave's BREADY / RREADY both follow the grant (`_rgnt_<slave>`). A path with one slave keeps the plain ownership gate. Excerpt in MAS 2.3.
2. **Master adapter, across width paths** (`b_pgnt` / `r_pgnt`): the same rules over the master's width paths. Only the granted path's ready is raised (MAS 2.1).

There are no response FIFOs. A response that loses arbitration waits in the slave's (or the slave adapter's skid buffer's) valid/ready hold.

11.5.3 Response Buffering

Not built. The option would add FIFOs to buffer pending responses:

```
Purpose:
  - Prevent response loss during arbitration conflicts
  - Allow slaves to return responses even if master busy
  - Smooth out bursty response patterns

Configuration:
  per_master_response_fifo_depth = 4-16 entries

Trade-off:
  + No response loss
  + Better throughput
```

```
- Additional resources (~200 LEs per FIFO)
- +1-2 cycle latency
```

11.6 2.8.6 Independent Write-Tracker FIFO (W-Channel Deadlock Fix)

Prior to commit `d24bd617`, the AW and W ready signals shared a single multiplexer at the write-address arbiter output. Under interleaved master traffic patterns, this could cause deadlock: a master's W channel could back up waiting for a non-granted write-address slot, preventing the arbiter from servicing other masters' AW transactions.

The corrected design (current) uses an **independent W-tracker FIFO** alongside the AW tracker:

```
// AW tracker FIFO (per master)
always_ff @(posedge aclk) begin
    if (master_awvalid && master_awready) begin
        aw_trk_fifo <= address_decode(master_awaddr);  // Capture
slave
    end
end

// Independent W tracker and ready MUX
logic aw_ready_mux, w_ready_mux;  // Separate paths (prior: shared)

assign master_awready = aw_ready_mux;   // AW gets its own ready
assign master_wready = w_ready_mux;      // W gets independent ready

// W-tracker: Gate W valid against tracked slave select
always_comb begin
    w_ready_mux = (aw_trk_fifo != 0) && slave_wready;  // W
independent
end
```

Result: AW and W channels can assert independently without blocking each other, preventing deadlock during interleaved multi-master write bursts.

11.7 2.8.7 Backpressure Management

11.7.1 Ready Signal Routing

The master's RREADY/BREADY has to find its way back to the correct slave:

```
// Route master ready back to slaves
logic m0_rready;  // From master
logic [2:0] slave_rready;  // To slaves

// Only selected slave sees master's ready
for (genvar s = 0; s < 3; s++) begin
    assign slave_rready[s] = (selected_slave == s) ? m0_rready : 1'b0;
end

// Non-selected slaves see ready = 0 (backpressure)
```

11.7.2 Stall Conditions

Response path can stall when:

```
1. Master not ready (M_RREADY = 0)
   → Selected slave stalls (S_RREADY = 0)
   → Other slaves buffer or stall

2. Arbitration conflict (multiple slaves ready)
   → Non-selected slaves stalled
   → Selected slave proceeds

3. bridge_id tracking FIFO full -- the address handshake is gated on
it (bridge BUG-009, was BRIDGE-011); there is no response DATA FIFO
   → Slave stalled until space available
```

11.8 2.8.8 Response Path Latency

11.8.1 End-to-End R Channel

```
Slave R response → Bridge → Master R channel

Components:
1. Slave response valid
2. FIFO head read (0 cycles, combinatorial) -- no BID extraction
3. Demux routing (0-1 cycles)
4. No response arbitration: B and R follow the address-phase selection
5. Optional FIFO (0-1 cycles)
6. Master adapter (1 cycle, skid buffer)

Total: 2-4 cycles typical
```

11.8.2 End-to-End B Channel

```
Similar to R channel:
  Slave B → Extraction → Demux → Arbitration → Master B
  Total: 2-4 cycles
```

11.9 2.8.9 Error Response Routing

11.9.1 Slave Error Responses

When slave returns error:

```
RRESP = 2'b10 (SLVERR) or 2'b11 (DECERR)
BRESP = 2'b10 (SLVERR) or 2'b11 (DECERR)

Routing:
  - Extract BID normally
  - Route to originating master
  - Preserve error code
  - Master sees error response
```

11.9.2 Out-of-Range Error Responses

When router generates error for OOR address:

```
Process:
1. Router captures OOR request
2. Generates internal error response
   - Uses cached request ID (with BID)
   - Sets RRESP/BRESP = DECERR
3. Injects into response path
4. Routes to master using BID
```

11.10 2.8.10 Resource Utilization

11.10.1 Response Router Resources

4 masters, 3 slaves (64-bit data):

```
Logic Elements:  ~1500-2000 LEs
Registers:       ~400-600 regs
Block RAM:       0 (unless FIFOs enabled)

Breakdown:
- BID extraction (3 slaves):     ~150 LEs
- Demux logic (4 masters):        ~600 LEs, ~150 regs
- Response arbitration:           ~300 LEs, ~100 regs
- Backpressure routing:           ~200 LEs, ~50 regs
- Control FSMs:                   ~250 LEs, ~100 regs

Optional FIFOs (4 × 8 entries):  +800 LEs, +2KB BRAM
```

11.10.2 Scaling

```
Resource usage scales with:
- Number of masters: Linear (one demux path per master)
- Number of slaves: Linear (one extraction per slave)
- Data width: Linear (wider data = wider demux)
- FIFO depth: Linear (deeper = more BRAM)
```

11.11 2.8.11 Configuration Parameters

11.11.1 Response Routing Configuration (TOML)

```toml
[bridge]
num_masters = 4
num_slaves = 3

[bridge.response_routing]
enable_response_fifos = false       # Buffer responses per master
fifo_depth = 8                      # Entries per FIFO
response_arbiter_type = "round_robin"  # "round_robin",
"fixed_priority"
registered_demux = false            # Register demux output (+1 cycle)

# Per-master response credits (optional)
[[bridge.masters]]
name = "cpu"
max_response_credits = 16           # Outstanding responses allowed
```

11.12 2.8.12 Debug and Observability

11.12.1 Recommended Debug Signals

```
BID Extraction:
- Slave RID/BID (from each slave)
- Extracted BID (master index)
- External RID/BID (to master, BID stripped)

Response Demux:
- Demux select signals (which master per slave)
- Response valid per master
- Response valid per slave

Arbitration:
- Conflict detection (multiple responses for same master)
- Arbiter grant (which slave wins)
- Stall counters

FIFOs (if enabled):
- FIFO occupancy per master
- FIFO full/empty flags
- FIFO overflow errors
```

11.12.2 Performance Counters

```
- Responses routed per master
- Arbitration conflicts (cycles with multiple responses to same
master)
- Response stall cycles (master not ready)
- Average response latency per master
- FIFO overflow counts
- bridge_id FIFO push/pop counts
```

11.13 2.8.13 Common Issues and Debug

Symptom: Response not reaching master
Check: - BID extraction (correct bit slicing?) - Master index (BID → master mapping) - Demux select logic - Backpressure (is master READY?)

Symptom: Response goes to wrong master
Check: - BID values (verify each master has unique BID) - BID width calculation (clog2 correct?) - bridge_id FIFO contents (wr_fifo/rd_fifo) - ID corruption in transit

Symptom: Response ordering incorrect
Check: - FIFO head (routing is by position, so order IS the mechanism) - Arbitration fairness (round-robin working?) - Multiple outstanding transactions per master

Symptom: Response loss
Check: - FIFO overflows (enable FIFOs if needed) - Dropped responses during arbitration conflicts - Protocol violations (slave not following AXI rules)

11.14 2.8.14 Verification Considerations

11.14.1 Test Scenarios

1. **Single Master, Single Slave**:

```
- Basic routing test
- Verify BID extraction and stripping
- Check response reaches correct master
```

2. **Multiple Masters, Single Slave**:

```
- Interleaved responses
- Verify each response routes to correct master
- Check BID uniqueness prevents collisions
```

3. **Single Master, Multiple Slaves**:

```
- Simultaneous responses from multiple slaves
- Verify arbitration fairness
- Check no response loss
```

4. **All Masters, All Slaves**:

```
- Maximum stress test
- All masters issuing to all slaves
- Responses returning in various orders
- Verify correct routing under heavy load
```

5. **OOO Responses**:

```
- Issue transactions: ID=1, ID=2, ID=3
- Return responses: ID=3, ID=1, ID=2
- Verify the FIFO head routes each
- Check response order at master
```

6. **Backpressure Test**:

```
- Master asserts RREADY=0
- Verify slave stalls (RREADY=0 propagated)
- Master asserts RREADY=1
- Verify responses flow
```

11.15 2.8.15 Performance Optimization

11.15.1 Techniques

1. Registered Demux:

```
Trade-off: +1 cycle latency for timing closure
Best for: High-frequency designs, many masters
```

2. Response FIFOs:

```
Trade-off: +2KB BRAM, +1-2 cycle latency
Benefit: Prevent response loss, smooth conflicts
Best for: High-throughput systems
```

3. Distributed Arbitration:

```
Implementation: Per-master arbiters instead of global
Benefit: Reduced logic depth, better timing
Best for: >8 masters
```

4. Priority Response Routing:

```
Implementation: High-priority masters get preference in conflicts
Benefit: Guaranteed low latency for critical masters
Best for: Real-time systems
```

11.16 2.8.16 Advanced Features

11.16.1 Response Reordering (Future)

Built (bridge BUG-012 (was BRIDGE-015)/016, 2026-09-10). Every real AXI slave of a multi-master bridge, and any slave declared `enable_ooo = true`, gets `bridge_cam` for both channels and routes responses by ID; single-master bridges, shim slaves and the subtractive slave keep the in-order per-slave FIFO. Per-ID tracking is sound across masters because fabric IDs are `{master index, master id}`; the FIFO would require the slave to complete in request order across ALL IDs, which a compliant AXI slave need not do. Fixture `bridge_2x2_ooo` exercises reordering slaves with two masters sharing IDs; `test_bridge_cam.py` covers the CAM alone. The mode had been unbuildable since c64660f47. See ch04 `02_id_tracking.md`.

Allow bridge to reorder responses for efficiency:

```
Scenario:
  Master issues: Req A (slow slave), Req B (fast slave)
  Fast slave responds first
  Bridge can deliver B before A (if IDs different)

Benefit: Reduced latency for fast responses
Requirement: CAM to track outstanding in-order constraints
```

11.16.2 Response Coalescing (Future)

Combine multiple responses into fewer beats:

```
Scenario:
  Multiple single-beat reads to narrow slave
  Bridge coaleses into fewer wide beats

Benefit: Reduced overhead
Complexity: High (requires buffering and alignment)
```

11.16.3 Response QoS (Future)

Prioritize responses based on master QoS:

```
Feature: High-QoS masters get response priority
Implementation: Weighted arbitration in response path
Use case: RT masters need bounded response latency
```

11.17 2.8.17 Timing Considerations

11.17.1 Critical Paths

Common critical paths in response routing:

```
1. BID extraction → Demux select → Master RDATA
   Depth: ~8-12 logic levels

2. Slave RVALID → Arbitration → Master RVALID
   Depth: ~6-10 logic levels

3. Master RREADY → Backpressure → Slave RREADY
   Depth: ~5-8 logic levels
```

11.17.2 Optimization Strategies

```
- Register demux outputs (+1 cycle, breaks path)
- Pipeline arbitration (+1-2 cycles, multi-stage)
- Complex ID scenarios are unsupported: routing is positional, not ID-
keyed
- Limit response buffering depth (less MUX levels)
```

11.18 2.8.18 Future Enhancements

11.18.1 Planned Features

- **Dynamic Response Buffering**: Adjust FIFO depth based on utilization
- **Response Ordering Hints**: Masters specify ordering requirements
- **Multi-Cycle Arbitration**: Pipelined for >16 masters
- **Response Compression**: Pack narrow responses into wide beats

11.18.2 Under Consideration

- **Response Speculation**: Predictive routing before full ID available
- **Virtual Response Channels**: Separate paths for different QoS classes
- **Response Mirroring**: Duplicate responses for redundancy
- **Cross-Clock Response**: Async response paths with CDC

Related Sections: - Section 2.3: Crossbar Core (response path architecture) - Section 2.5: ID Management (BID extraction details) - Section 2.4: Arbitration (response arbitration algorithms) - HAS ch04_interfaces/01_axi4_interface.md (response channel signals)

---

12 2.9 Error Handling

What error handling this bridge actually has. Measured against `rtl/generated/`, not designed on paper. The complete inventory:

Mechanism	Where	What it does
`DECERR` on unmapped access	`axi4_subtractive_slave`	answers, so an unmapped address cannot hang the master
`o_hit_irq` / `o_hit_count`	same	sticky flag and saturating count, both cleared by `i_hit_clear`
`o_hit_addr`	same	FIRST fault address; deliberately RETAINED across a clear so the evidence survives
monbus error packet	`*_mon` builds only	reports the fault to a monitor bus
response-ordering check	simulation only	`$error` on a BID/RID that does not match the FIFO head (bridge BUG-008, was BRIDGE-010)

Everything else this chapter describes – a protocol checker, a per-transaction watchdog, an error status register, an error history buffer, error interrupt logic, and a `[bridge.error_handling]` TOML table – is NOT BUILT. Searches for `protocol_check`, `error_status`, `error_history` and `error_irq` across every generated file return nothing, and the config loader does not know the `error_handling` table name, so such a section is silently ignored. Those parts are retained as design intent and each is marked.

Error handling is everything the bridge does when something goes wrong: detecting the problem, answering with the right AXI error response, logging what happened, and keeping the rest of the system running. The covered failure modes are protocol violations, out-of-range addresses, timeout conditions, and configuration errors.

12.1 2.9.1 Purpose and Function

Error handling does five things:

1. **Error Detection**: Identifies violations and abnormal conditions
2. **Error Response Generation**: Creates appropriate AXI error responses
3. **Error Reporting**: Logs and signals errors for debug
4. **Graceful Degradation**: Maintains system stability during errors
5. **Recovery Support**: Enables system recovery after errors

12.2 2.9.2 Error Categories

12.2.1 Transaction Errors

Out-of-Range (OOR) Addresses:

```
Master requests address not mapped to any slave
Response: DECERR (Decode Error)
Action: Bridge generates error response internally
```

Protocol Violations:

```
Master violates AXI4 protocol rules
Examples:
  - VALID deasserted while READY=0
  - Incorrect burst parameters
  - ID width mismatches
Response: Configurable (error log, SLVERR, or ignore)
```

Slave Errors:

```
Slave returns SLVERR or DECERR response
Action: Bridge forwards error to master unchanged
```

12.2.2 System Errors

Timeout Conditions: there are none. The bridge has NO transaction timeout – not on the AXI path and not in the APB shim, where 2.7 states it three times (“There is no PREADY timeout anywhere in the path”). A slave that never responds stalls that path indefinitely; nothing in the fabric detects or breaks the stall.

This block previously described a configurable timeout returning DECERR. No such mechanism was ever built, and believing in it is worse than knowing it is absent – an integrator counting on the fabric to time out will not add the watchdog that actually protects the system.

```
(retained for contrast -- NOT implemented)
Slave fails to respond within configured timeout
Response: DECERR after timeout expires
Action: Force transaction completion
```

CAM Overflow:

> **Built as backpressure, not as an error.** Each slave adapter’s response tracker (`bridge_cam` for a multi-master AXI4/AXI5 slave, an in-order FIFO otherwise) is the slave’s `max_outstanding` deep (default 16); when it is full the slave’s AWREADY / ARREADY toward the crossbar drops (bridge BUG-009). No error response is generated. See ch04.

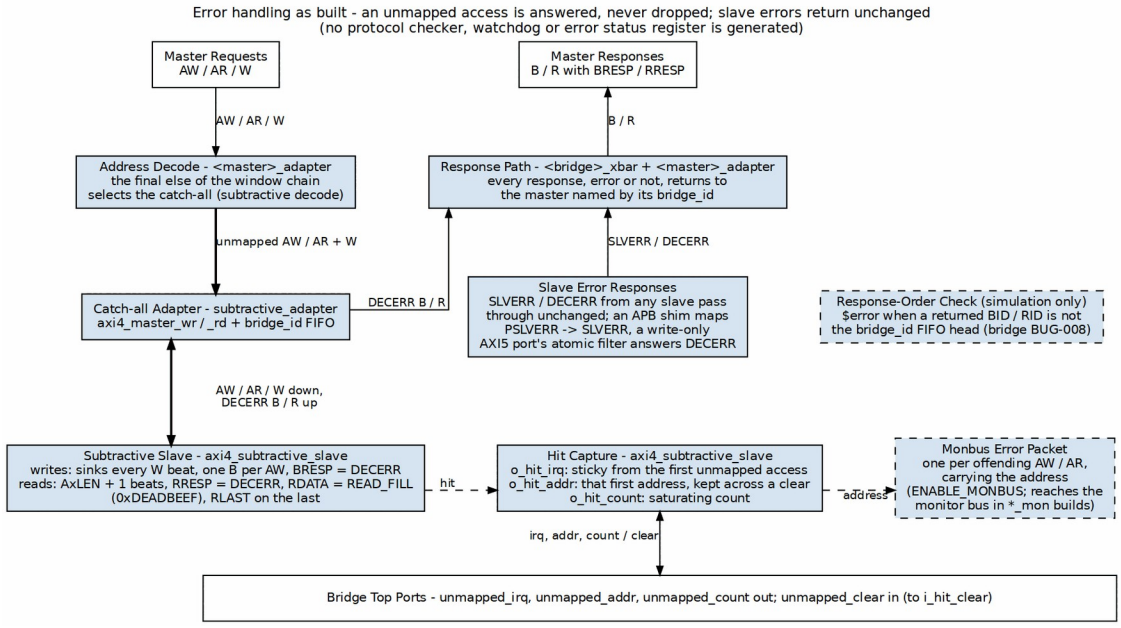
```
Too many outstanding transactions (CAM full)
Response: Backpressure master (ARREADY/AWREADY=0)
Action: Wait for responses to free entries
```

Configuration Errors:

```
Invalid bridge configuration detected at generation time
Examples:
  - Overlapping slave address ranges
  - Invalid data width combinations
  - Illegal parameter values
Response: Generation error, fail early
```

12.3 2.9.3 Block Diagram

12.3.1 Figure 2.10: Error Handling Architecture



Error Handling System

Only what the generated RTL contains: the catch-all path that answers an unmapped access with DECERR (and 0xDEADBEEF read data), the sticky hit capture behind the bridge's `unmapped_*` ports, the monitor-bus error packet, slave error responses returning unchanged, and the simulation-only response-order check.

12.4 2.9.4 Out-of-Range Address Handling

12.4.1 Detection

Router identifies OOR when no slave address range matches:

```
// Address decode with OOR detection
logic [NUM_SLAVES-1:0] slave_match;
logic oor_detected;

for (genvar i = 0; i < NUM_SLAVES; i++) begin
    assign slave_match[i] = (addr >= SLAVE_BASE[i]) &&
                            (addr <= SLAVE_END[i]);
end

assign oor_detected = ~(|slave_match) && ~default_slave_en;
```

12.4.2 Error Response Generation

Read OOR Response:

```
// Generate R response for OOR read
always_ff @(posedge clk) begin
    if (ar_oor_detected && arvalid && arready) begin
        // Capture transaction details
        oor_arid <= arid;
        oor_arlen <= arlen;
        oor_burst_count <= 0;
        oor_generating <= 1'b1;
    end else if (oor_generating) begin
        // Generate R beats
        rvalid <= 1'b1;
        rdata <= OOR_DATA_PATTERN;  // Configurable pattern
        rid <= oor_arid;
        rresp <= 2'b11;  // DECERR
        rlast <= (oor_burst_count == oor_arlen);

        if (rready) begin
            oor_burst_count <= oor_burst_count + 1;
            if (rlast) oor_generating <= 1'b0;
        end
    end
end
```

Write OOR Response:

```
// Generate B response for OOR write
always_ff @(posedge clk) begin
    if (aw_oor_detected && awvalid && awready) begin
```

```
            // Capture write ID
            oor_awid <= awid;
            oor_w_count <= 0;
            oor_sinking_w <= 1'b1;
        end else if (oor_sinking_w) begin
            // Sink W data (accept and discard)
            wready <= 1'b1;

            if (wvalid && wlast) begin
                oor_sinking_w <= 1'b0;
                oor_gen_bresp <= 1'b1;
            end
        end else if (oor_gen_bresp) begin
            // Generate B response
            bvalid <= 1'b1;
            bid <= oor_awid;
            bresp <= 2'b11;  // DECERR

            if (bready) begin
                oor_gen_bresp <= 1'b0;
            end
        end
    end
```

12.4.3 Configurable Data Patterns

Not built. The read fill value is a module PARAMETER of `axi4_subtractive_slave` (`READ_FILL`, default `32'hDEAD_BEEF`). The generator does not override it – `bridge_2x2_rw.sv` passes only the width, `UNIT_ID` and `AGENT_ID` parameters – and no TOML key sets it. There is no `[bridge.error_handling]` table; the loader does not know that name, so a config that contains one is silently ignored. To change the pattern today, override the parameter at instantiation.

```
[bridge.error_handling]
oor_read_data_pattern = 0xDEADCAFE  # Pattern for OOR reads
oor_response_latency = 2            # Cycles to generate response
```

12.5 2.9.5 Protocol Violation Handling

12.5.1 Common Violations

VALID Dropped While READY=0:

```
AXI4 Rule: Once VALID asserted, must remain until READY
Violation: Master deasserts VALID before handshake
Detection: Track VALID history per channel
Action: Log error, optionally assert error signal
```

Incorrect Burst Parameters:

```
Violations:
  - SIZE > DATA_WIDTH
  - LEN > 255
  - Address not aligned to SIZE
  - Burst crosses 4KB boundary
Detection: Parameter checking logic
Action: SLVERR response or reject transaction
```

ID Width Mismatch:

```
Violation: Master uses ID wider than configured
Detection: Check ID values against expected width
Action: Truncate or error (configurable)
```

12.5.2 Protocol Checker Implementation

> **Not built.** No generated bridge contains a protocol checker: a search for `protocol_check` / `proto_check` / `protocol_viol` across every file in `rtl/generated/` returns nothing. The section below describes hardware that was never implemented. Protocol violations by an attached master or slave are not detected – with one exception, the bridge BUG-008 (was BRIDGE-010) response-ordering check, which is simulation-only and cannot fire in silicon.

```
// AXI4 protocol checker (simplified)
module axi_protocol_checker #(
    parameter ID_WIDTH = 4,
    parameter ADDR_WIDTH = 32,
    parameter DATA_WIDTH = 64
) (
    input logic clk, rst_n,
    // AXI signals
    input logic arvalid, arready,
    input logic [ADDR_WIDTH-1:0] araddr,
    input logic [7:0] arlen,
```

```
    input logic [2:0] arsize,
    // Error outputs
    output logic protocol_error,
    output logic [7:0] error_code
);

    // Check: VALID held until READY
    logic arvalid_prev;
    always_ff @(posedge clk) begin
        arvalid_prev <= arvalid;
        if (arvalid_prev && !arvalid && !aready) begin
            protocol_error <= 1'b1;
            error_code <= 8'h01;  // VALID_DROPPED
        end
    end

    // Check: SIZE <= DATA_WIDTH
    always_comb begin
        if (arvalid && (arsize > $clog2(DATA_WIDTH/8))) begin
            protocol_error = 1'b1;
            error_code = 8'h02;  // INVALID_SIZE
        end
    end

    // Check: Address alignment
    always_comb begin
        if (arvalid && (araddr[arsize-1:0] != 0)) begin
            protocol_error = 1'b1;
            error_code = 8'h03;  // UNALIGNED_ADDR
        end
    end

    // Additional checks...
endmodule
```

12.6 2.9.6 Timeout Detection

12.6.1 Per-Transaction Watchdog

Not built. As described, The bridge has NO per-transaction watchdog: it never terminates a stuck transaction and never manufactures a response for one. Non-monitor builds contain zero timeout logic – `bridge_2x2_rw` and `bridge_mix_a` have no `timeout` signal at all.

What DOES exist, and only in `*_mon` builds, is observability: `cfg_wr_timeout_enable` / `cfg_wr_timeout_cycles` / `cfg_wr_axi_timeout_mask` (and the `rd` equivalents) are inputs to the AXI MONITOR, which emits a timeout EVENT PACKET on monbus when a transaction outlives the programmed cycle count. It reports; it does not intervene. A hung slave still hangs the bridge – the monitor just tells you it happened.

```
// Timeout detector for outstanding transactions
typedef struct packed {
    logic valid;
    logic [TOTAL_ID_WIDTH-1:0] id;
    logic [15:0] timestamp;
    logic is_read;  // 1=read, 0=write
} timeout_entry_t;

timeout_entry_t watchdog [0:15];  // Track up to 16 transactions
logic [15:0] global_timer;

always_ff @(posedge clk) begin
    if (!rst_n) begin
        global_timer <= 0;
    end else begin
        global_timer <= global_timer + 1;

        // Check for timeouts
        for (int i = 0; i < 16; i++) begin
            if (watchdog[i].valid) begin
                logic [15:0] elapsed;
                elapsed = global_timer - watchdog[i].timestamp;

                if (elapsed > TIMEOUT_THRESHOLD) begin
                    // Timeout detected
                    timeout_error <= 1'b1;
                    timeout_id <= watchdog[i].id;
                    timeout_type <= watchdog[i].is_read ? "READ" :
"WRITE";
```

```
                    // Force completion with DECERR
                    force_error_response(watchdog[i]);
                    watchdog[i].valid <= 1'b0;
                end
            end
        end
    end
end
```

12.6.2 Timeout Configuration

> **Not built.** No watchdog exists to configure. In `*_mon` builds the `cfg_*_timeout_*` inputs configure the AXI MONITOR's reporting threshold, not any bridge behaviour.

There is none. `[bridge.error_handling]` is not a table the loader knows, and `enable_timeout` / `timeout_cycles` / `timeout_action` / `per_slave_timeout` are not keys.

What the loader actually does with a key it does not know – measured by feeding it a config carrying `cam_depth`, `pipeline_depth` and `internal_data_width`, not inferred from reading it. Three different behaviours, which is why other pages disagree:

Timeout Configuration

Where	Behaviour
`[bridge.mon_group]`	HARD ERROR – `ValueError` naming the offending keys and listing the valid ones
`[bridge.defaults]`	WARNS: "`[bridge.defaults]` is not implemented – section ignored"
anywhere else	SILENTLY ACCEPTED and ignored; nothing warns

So a TOML written from the block this section used to contain loads fine and does nothing. An earlier revision of this paragraph claimed `config_validator` rejects unknown keys and the config "does not load at all"; that is true only of `mon_group`.

A hung slave stalls its path indefinitely. If the system needs to survive that, the watchdog belongs outside the bridge.

12.7 2.9.7 Error Logging and Reporting

12.7.1 Error Status Register

Not built. No generated file contains an error status register. The nearest real thing is the subtractive slave's `o_hit_irq` / `o_hit_addr` / `o_hit_count`, which are module PORTS, not a memory-mapped register.

The register as originally proposed (read/clear), kept for reference:

Error Status Register

Bits	Description
0	OOR Read Error
1	OOR Write Error
2	Protocol Violation
3	Timeout Error
4	bridge_id FIFO overflow (gated off)
5	Slave Error (SLVERR received)
6	Decode Error (DECERR received)
7	ID Match Error
15:8	Reserved
31:16	Error Count (saturating)

12.7.2 Error History Buffer

Not built. There is no history buffer. Only the FIRST fault address is retained (`o_hit_addr`), plus a saturating count.

```
// Circular buffer for error history
typedef struct packed {
    logic [7:0] error_type;
    logic [31:0] address;
    logic [TOTAL_ID_WIDTH-1:0] id;
    logic [31:0] timestamp;
    logic [7:0] master_index;
    logic [7:0] slave_index;
} error_entry_t;

error_entry_t error_log [0:15];  // 16-entry history
logic [3:0] error_wr_ptr;
```

```
always_ff @(posedge clk) begin
    if (error_detected) begin
        error_log[error_wr_ptr] <= {
            error_type,
            error_addr,
            error_id,
            global_timer,
            error_master,
            error_slave
        };
        error_wr_ptr <= error_wr_ptr + 1;
    end
end
```

12.7.3 Error Interrupts

Not built as a block. The only interrupt is `o_hit_irq` from the subtractive slave: one sticky bit, cleared by `i_hit_clear`. There is no interrupt controller, mask register or priority logic.

```
Optional interrupt generation:

Interrupt Enable Register:
  - OOR_INT_EN
  - TIMEOUT_INT_EN
  - PROTOCOL_INT_EN
  - etc.

Interrupt when enabled error occurs
Clear on status register read or explicit clear
```

12.8 2.9.8 Resource Utilization

12.8.1 Error Handling Resources

Not built. These figures price a protocol checker, watchdog and error registers that do not exist. The real cost is the subtractive slave plus a few status flops.

```
Logic Elements:  ~800-1200 LEs
Registers:       ~400-600 regs
Block RAM:       0-2 KB (if error logging enabled)

Breakdown:
- OOR detection/response:     ~300 LEs, ~100 regs
- Protocol checkers:           ~250 LEs, ~50 regs
- Timeout watchers:            ~200 LEs, ~150 regs
- Error logging:               ~150 LEs, ~100 regs
- Status registers:            ~100 LEs, ~50 regs

Optional error log (16 entries): +2KB BRAM
```

12.9 2.9.9 Configuration Parameters

12.9.1 Error Handling Configuration (TOML)

Not built. The loader does not know a `[bridge.error_handling]` table. A config containing one is silently ignored – nothing warns. The read fill pattern is the `READ_FILL` parameter of `axi4_subtractive_slave`, which the generator never overrides.

```toml
[bridge.error_handling]
# OOR handling
# NOT REAL KEYS. Out-of-range handling is fixed, not configured: the
# subtractive slave always answers DECERR with READ_FILL (0xDEADBEEF),
there
# is no timeout anywhere in the bridge, and there is no protocol
checker.
# Kept visible rather than deleted because several pages once
described these
# as configuration, and a reader who saw them elsewhere should find
out here
# that they do not exist.
#
#   enable_oor_errors / oor_response_type / oor_read_pattern
#   enable_timeout / timeout_cycles / timeout_response
#   enable_protocol_checker
protocol_checker_mode = "log"     # "log", "error", "ignore"

# Error logging
enable_error_log = true
error_log_depth = 16              # Entries in circular buffer

# Error reporting
enable_error_interrupts = false
error_status_register_addr = 0xFFFF_FFF0  # Optional mapped register
```

12.10 2.9.10 Debug and Observability

12.10.1 Recommended Debug Signals

```
Error Detection:
- OOR flags (per master)
- Protocol violation flags
- Timeout counters
- bridge_id FIFO occupancy

Error Response:
- Error response generation FSM states
- Active error responses
- Error data patterns

Error Logging:
- Error count
- Last error type
- Last error address
- Error log write pointer
```

12.11 2.9.11 Common Issues and Debug

Symptom: Unexpected DECERR responses
Check: - Address map configuration (gaps?) - Slave address ranges (overlaps?) - Default slave configuration - Timeout thresholds (too short?)

Symptom: System hangs on certain addresses
Check: - Timeout detection enabled? - Slave responsiveness - Protocol violations upstream - bridge_id FIFO overflow (prevented by the not-full gate, bridge BUG-009 (was BRIDGE-011))

Symptom: Error log filling quickly
Check: - What errors are occurring? - Master behavior (bad addresses?) - Slave configuration - Timeout thresholds

12.12 2.9.12 Verification Considerations

12.12.1 Test Scenarios

1. **OOR Address Tests**:

```
- Read from unmapped address
- Write to unmapped address
- Burst to partially mapped range
- Verify DECERR response
```

2. **Timeout Tests**:

```
- Slave never responds
- Verify timeout after N cycles
- Check error logged
- Verify forced completion
```

3. **Protocol Violation Tests**:

```
- Drop VALID before READY
- Invalid burst parameters
- Misaligned addresses
- Verify detection and response
```

4. **Error Recovery Tests**:

```
- Error occurs
- System continues operating
- Subsequent transactions succeed
- Error status readable
```

5. **Error Logging Tests**:

```
- Generate multiple errors
- Verify all logged
- Check circular buffer wrap
- Verify timestamps
```

12.13 2.9.13 Recovery Mechanisms

12.13.1 Automatic Recovery

```
OOR Errors:
  - Generate error response
  - Continue normal operation
  - No intervention needed

Timeout Errors:
  - Force transaction completion
  - Pop the bridge_id FIFO
  - Allow new transactions

Protocol Violations:
  - Log error
  - Complete/reject transaction
  - Continue with next transaction
```

12.13.2 Manual Recovery

```
bridge_id FIFO stuck:
  - Software reset via control register
  - Reset the bridge_id FIFO pointers
  - Restart affected masters

Persistent Errors:
  - Read error log
  - Identify root cause
  - Reconfigure or reset subsystem
```

12.14 2.9.14 Safety and Reliability

12.14.1 Error Containment

```
Goal: Prevent error propagation

Mechanisms:
- Isolate error to originating master
- Don't corrupt other masters' transactions
- Maintain crossbar integrity
- Log for post-mortem analysis
```

12.14.2 Fail-Safe Behavior

```
On Critical Error:
1. Generate safe error response (DECERR)
2. Log error details
3. Assert error signal (optional)
4. Continue operation if possible
5. Halt only if configured (safety-critical mode)
```

12.14.3 Error Injection (Debug/Test)

> **Not built.** The loader knows no `bridge.debug` table and no `error_injection` – searching `config_loader.py` for either returns nothing. A config containing this section is accepted and silently ignored, which is worse than an error: the build looks configured and injects nothing.

```toml
[bridge.debug]
enable_error_injection = true

[[bridge.debug.error_injection]]
type = "oor"
trigger_address = 0xBAD_ADDR
action = "decerr"

[[bridge.debug.error_injection]]
type = "timeout"
trigger_after_n_cycles = 100
target_slave = 2
```

12.15 2.9.15 Future Enhancements

12.15.1 Planned Features

- **Error Recovery FSM**: Automatic recovery from certain error conditions
- **Error Rate Limiting**: Prevent error storm from misbehaving master
- **Detailed Error Telemetry**: Capture full transaction context on error
- **Error Prediction**: Detect patterns indicating impending failures

12.15.2 Under Consideration

- **ECC Protection**: Error correction for internal state
- **Redundant Checking**: Duplicate checkers for safety-critical
- **Watchdog Timers**: System-level health monitoring
- **Error Severity Levels**: Classify errors by impact

Related Sections: - Section 2.2: Slave Router (OOR detection) - Section 2.5: ID Management (bridge_id FIFO depth) - Section 2.8: Response Routing (error response paths) - Chapter 5: Verification (error testing strategies)

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

13 AMBA5 Boundary and Native Sideband

How AXI5/APB5 ports ride the AXI4 fabric (bridge TASK-002 (was BRIDGE-002) phases A5-1 through A5-3). One spec table drives every generator: `bin/bridge_pkg/sideband.py::SIDEBAND_FIELDS` maps each feature to its per-channel struct fields, widths, and wrapper port bases — package, adapter, crossbar, and slave-adapter emission all iterate it in the same order, so struct layout and wiring cannot drift apart.

13.1 Boundary Wrappers

- AXI5 **master** port → `axi5_slave_{wr,rd}[_mon]` at the master adapter (the bridge is a slave to the external master).
- AXI5 **slave** port → `axi5_master_{wr,rd}[_mon]` at the slave adapter.
- Both families carry every feature signal through their skid path, gated by `ENABLE_<FEATURE>` parameters; the generator binds every pin (enabled features connect through, the rest tie/open).

13.2 Native Sideband Through the Structs (A5-2)

The per-bridge `_pkg` channel structs gain feature fields as the **union** of features on any AXI5 port. Pure-AXI4 bridges emit no fields, so their RTL stays byte-identical — the zero-drift invariant.

- **Master adapter:** packs its own enabled fields from the wrapper's `fub_axi_*` sideband on the **direct (width-matched) arm only**; converter arms pack `'0` — per-beat and per-transaction sideband cannot traverse the dwidth-converter IP. Because non-qualifying sources are guaranteed `'0`, the crossbar forwards request fields unconditionally.
- **Crossbar:** explodes struct fields into discrete `<slave>_axi_<sig>` nets for feature-enabled AXI5 slaves; response fields (`b.trace`, `r.trace`, `r.poison`) mux from qualifying slaves and default `'0` (they must be driven for every master since union fields exist in every struct).
- **Slave adapter:** rides `xbar_<slave>_axi_<sig>` nets into the wrapper via the component's `native_sideband` flag, which stops terminating enabled features' fub ports (fall-through to the connector prefix).

The struct field for AWIDUNQ/ARIDUNQ is named `idunq` (IHI 0022L A5.2; the library called the indicator `awunique`/`arunique` until GitHub #102 item 5, but AWUNIQUE is an unrelated ACE signal). `BIDUNQ`/`RIDUNQ` are deliberately **not** struct fields: the master adapter regenerates them from a per-ID bit table (`r_awidunq_by_id` / `r_aridunq_by_id`, written at the AW/AR handshake on the wrapper's fub face, and for a read-return atomic at its AW), read by `BID`/`RID` (`AdapterGenerator._idunq_echo_lines`). A5.2 makes the response indicator a function of the request and forbids mixing indicators among outstanding same-ID transactions, so the table is exact under any response order, and it is right on every path the request indicator was dropped on. The slave adapter leaves the slave's `BIDUNQ`/`RIDUNQ` unconnected.

13.2.1 AWAKEUP (A14.1.1)

`wakeup` is a port feature but not struct sideband. Both boundary wrappers of a port carry an `awakeup` pin (one signal covers both request channels). On a master port both `axi5_slave_{wr,rd}` take the top-level `AWAKEUP`; their fub-side copies (the input OR'd with requests held in that wrapper's skid buffers) are OR'd into the adapter output `wakeup_req`. The top ORs the `wakeup_req` of every connected wake-capable master into each wake-capable slave adapter's `wakeup_req` input (`1'b0` when none), which feeds both `axi5_master_{wr,rd}` fub `awakeup` pins; each wrapper ORs in its own held requests and the slave adapter ORs the two `m_axi_awakeup` copies onto the port. Every term is a flop or a top-level input, so the tree is glitch-free. The bridge never waits for a master's `AWAKEUP`.

13.2.2 Data-scaled fields: Memory Tagging and chunking (bridge TASK-007, was BRIDGE-018)

The last two wrapper features, `mte` and `chunking`, joined the table on 2026-09-13 and closed the native-AXI5 gap. Their fields are the only ones whose width follows the data bus: a tag bus is 4 bits per 16 bytes (`wtag`, `rtag`; IHI 0022L A12.2 has no `AWTAG` or `BTAG`, and the library's `awtag`/`btag`/`rtagmatch` went with GitHub #102 item 5), `wtagupdate` and `rchunkstrb` one bit per 16 bytes / 128-bit chunk. `sideband.py` carries them as symbolic widths (`WIDTH_TAGS`, `WIDTH_NTAGS`, `WIDTH_CHUNKSTRB`) and resolves them with `field_width(width, data_width)`:

- the per-width `w`/`r` structs size them for their own width;
- the width-independent `aw`/`ar`/`b` structs size them for the **widest port in the bridge**, and the adapters and crossbar fit a narrower port's value with an explicit zero-extension or slice (`fit_expr`) – widths decided in Python, so the emitted RTL carries no casts;
- slave ports and adapter pins are sized for that port's own width.

The fabric carries these fields and never interprets them: tags are payload routed with their beat, the 2-bit `btagmatch` (Table A12.14) returns through the B mux by bridge id like `bresp`, and a chunked burst's R beats are routed by ID and released on `RLAST` whatever order the completer sends the chunks in. Both features require a port of 128 bits or more (`validate_axi5_wide_features`).

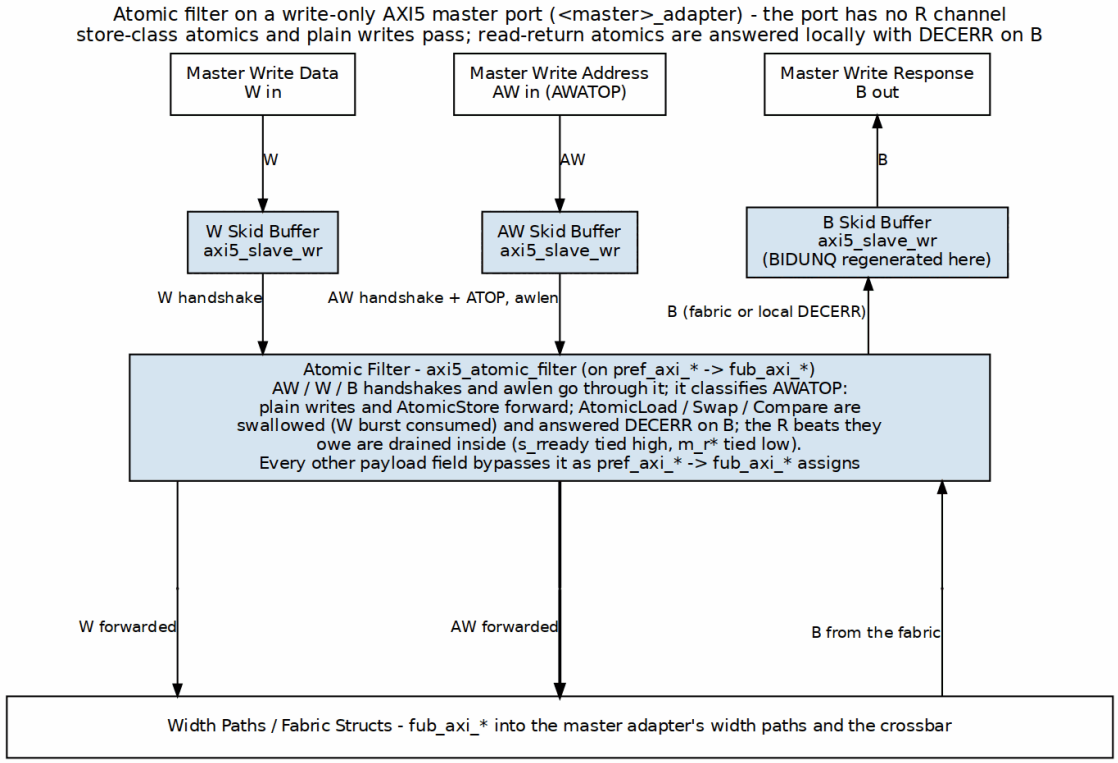
13.3 Connectivity Gating (poison, atomic, mte) and Droppable Chunking

`AXI5_CONNECTIVITY_GATED_FEATURES` in the validator: these features are legal only when **every** connected path is AXI5-both-ends, feature-enabled, and width-matched — otherwise a config error naming the offending pair. `mte` is in the set because a dropped tag operation changes what the write means. Droppable sideband that terminates mid-path is legal but prints a generation-time warning per (master, slave, feature); `chunking` is droppable because `ARCHUNKEN` is permission, not demand – a slave that cannot chunk answers with ordered data and `RCHUNKV` low, which every chunking requester must accept.

13.4 Atomic Filter (A5-3a, write-only masters)

A **write-only** atomic-enabled master's wr path inserts `axi5_atomic_filter` between the boundary wrapper and the fabric:

13.4.1 Figure 2.11: Atomic Filter on a Write-Only Port



Atomic Filter on a Write-Only Port

Handshakes (AW/W/B) and the B payload (`bid`/`bresp`) go **through** the filter, and `awlen` is connected so the filter can count the read data it owes; all other payload gets `pref → fub` pass-through assigns. Store- class ATOP and plain writes forward; read-return classes are swallowed (W burst consumed) and answered with a local DECERR B.

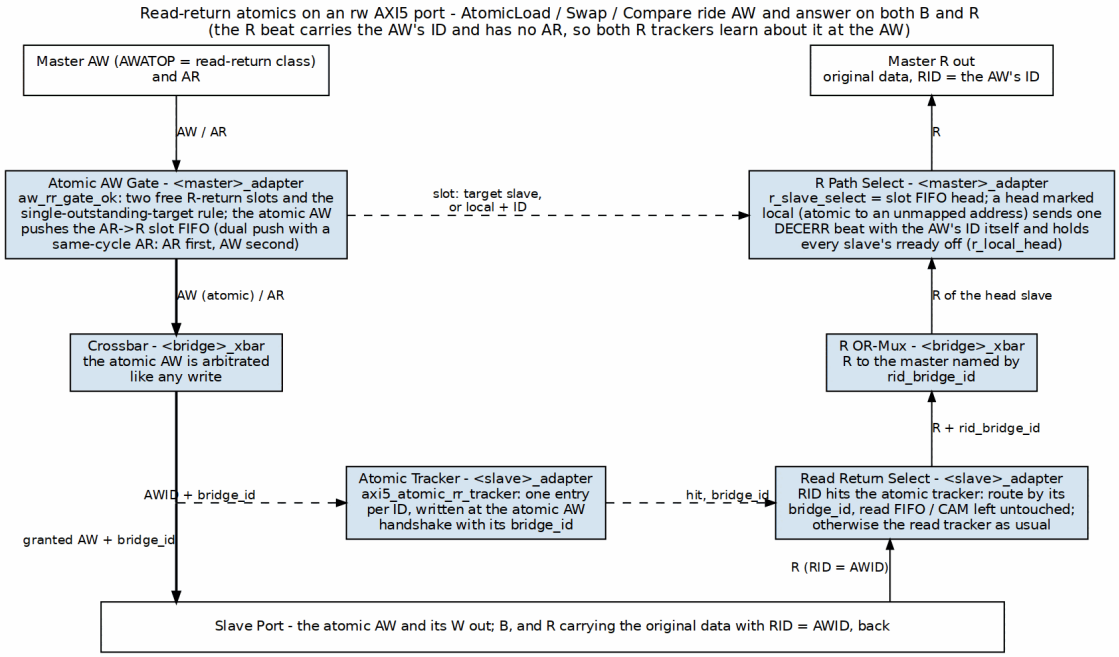
Since GitHub #102 item 2 the filter also returns the read data transfers a swallowed atomic owes (IHI 0022L A6.4.8: AWLEN+1 for Load/Swap, half that for Compare, each DECERR, RLAST on the last), merged into its `s_r*` output with downstream priority and no burst interleave. On the bridge's write-only port there is no R channel upstream to merge into and no downstream R, so the generator ties the filter's `s_rready` high and leaves `s_rvalid/s_rid/ s_rresp/s_rlast` open (the beats are accepted and discarded), and ties `m_rvalid`/`m_rid`/`m_rresp`/`m_rlast` low. Draining matters: an undrained read-return queue fills after `2**DEPTH_LG2` (8) swallowed atomics and then stalls AW for good. The atomic completes with DECERR on B only; HAS 4.4 states the resulting rule – managers must not issue load-class atomics on a write-only port. When the port has `unique`, `BIDUNQ` is regenerated on the `pref` side, so the local DECERR carries its own `AWIDUNQ`. See the module doc: `docs/markdown/rtl-amba/axi5/axi5_atomic_filter.md`. The

filter exists because a port with no read path has nowhere to deliver the R beat those classes answer with. A port that has one gets the next section instead.

13.5 Read-Return Atomics (A5-3b, rw masters)

An **rw** atomic-enabled master has no filter. AtomicLoad (`10xxxx`), AtomicSwap and AtomicCompare (`11000x`) ride the AW path to the slave like any write; the slave performs the operation and answers on **both** B and R, the R beat carrying the location's original data under the **AW's ID**. Two things in this fabric had to learn about that beat, because every R-return tracker in it is fed by AR handshakes:

13.5.1 Figure 2.12: Read-Return Atomics



Read-Return Atomics

Master adapter. The AR->R tracking FIFO that drives `r_slave_select` takes an entry at the atomic AW handshake as well as at AR. An AR and an atomic AW can handshake in the same cycle, so the FIFO has a dual push: AR takes the first slot, the AW the second, and the AW's own gate (`aw_rr_gate_ok`) demands two free slots and the same single-outstanding-target rule the read side already applies. The AW is the last entry pushed, so it becomes the active target. The B/W paths are unchanged. The trace tracker mirrors the dual push so the R beat echoes the AW's trace (bridge BUG-010, was BRIDGE-012).

Out of range. A read-return atomic that decodes to the subtractive slave would otherwise wedge the port: that slave answers DECERR on B and knows nothing about R, while the master adapter is holding an R-return slot for the beat. The tracker entry therefore carries a `local` flag and the AW's ID, and when it reaches the head the R mux presents a single DECERR beat with that ID itself, holding every slave's `rready` off (`r_path_active && !r_local_head`) for that cycle. B and R both report DECERR, and the port keeps working. This is bridge BUG-007 (was BRIDGE-009)'s rule applied to the atomic path.

The invariant underneath. This port keeps the one-target-at-a-time FIFO path (MAS 2.1): the R-return FIFO entry carries the atomic's local flag and ID, which the per-ID table of the other ports does not. The atomic AW's gate therefore yields when an AR to a different slave handshakes in the same cycle, the one way a dual push could hold two targets. The crossbar's response mux no longer depends on this: since the one-target limitation was fixed (GitHub #119 L1) it is a grant-driven mux behind a per-path response arbiter, not an OR-merge, so the old simulation-only `$countones` guard on the select vector is gone.

Slave adapter. `axi5_atomic_rr_tracker` (module doc: `docs/markdown/rtl-amba/axi5/axi5_atomic_rr_tracker.md`) records (AWID -> requester index) at every AW handshake with `AWATOP[5]` set, and answers combinationally from `RID`. An R beat it claims is routed by its tag and does **not** pop the in-order read FIFO; every other beat routes by FIFO position as before. The atomic AW's `awready` is held while the tracker is full. The bridge BUG-008 (was BRIDGE-010) in-order check skips tracked beats and reports, in simulation, an R beat that matches both a tracked atomic and the FIFO head: that is two requesters aliasing one ID at this slave, which this fabric does not disambiguate.

Validator. An rw atomic master's connected atomic slaves must be `rw` (a write-only slave cannot return read data). The return tracker sits beside whichever read tracker the slave uses, the in-order FIFO or the `enable_ooo` CAM (bridge BUG-012 (was BRIDGE-015) repaired that mode). A write-only atomic master is not subject to the rule; it keeps the filter.

Filelist. `axi5_atomic_filter.f` is pulled only when a write-only atomic master exists; `axi5_atomic_rr_tracker.f` only when an rw atomic slave exists. Pure-AXI4 bridges and the A5-3a fixture are byte-identical to before this change.

13.6 APB5 Slaves (A5-3c)

`protocol = "apb5"` reuses the entire APB4 path with two deltas: the slave adapter instantiates `axi4_to_apb5_shim` (a sideband wrapper over the APB4 shim — see the converters MAS), and the external surface adds `PAUSER/PWUSER` out (driven `'0`), `PWAKEUP` out and `PRUSER/PBUSER` in (terminated). `PWAKEUP` is a requester output (IHI 0024E 3.7.2; it was an input until GitHub #102 item 7): the shim drives it from a flop of `PSEL`, so it is glitch-free and held through the PREADY cycle. The generated TB drives the port with the APB5 BFM, which observes `PWAKEUP`: APB5 keeps the APB4 transfer protocol.

13.7 Lite and APB Requesters (bridge TASK-004, was BRIDGE-014)

Every protocol value is legal on a master port. Two of them needed work beyond the AXI4-Lite master path that already existed:

- **`axil5` master.** The AXI4-Lite promotion (AXI4 extras tied at the top, `axi4_slave_*` wrappers, the wide-slave aligner toward wider slaves) plus the AXI5-Lite ID and optional signals on the boundary, directions flipped for a requester. The ID (`awid`/`bid`/`arid`/`rid`) and `user` – the one optional group with an AXI4 destination – join the Lite surface and reach the adapter's AXI4 face; TRACE and POISON are terminated at the top. AXI5-Lite has no LOCK, LOOP, MPAM, MECID or NSAID (IHI 0022L B2.1.5, GitHub #102 item 3). `validate_axil5_features` applies to masters and slaves alike, so a config cannot name a group the design drops.
- **`apb` / `apb5` master.** The bridge is the APB completer. `apb4_to_axi4` / `apb5_to_axi4` (converters: `apb{4,5}_slave` + `apb_cmdrsp_to_axi4`) sit inside the master adapter in front of the ordinary timing wrapper, so monitoring, decode, width adaptation and the response mux are untouched. One transfer -> one single-beat AXI4 transaction; SLVERR and DECERR fold to PSLVERR; `PAUSER[0]`/`PWUSER[0]` ride the fabric USER bit (visible at an `axil5` slave with `user`). `PWAKEUP` is requester-driven and terminated.

Validator rules: AXI4-Lite masters `id_width = 0`, AXI5-Lite masters `id_width >= 1`; APB masters `addr_width = 32` (a requester addresses the whole fabric, unlike an APB slave port's window offset). The generated TB drives `axil5` masters with the AXIL5 BFMs and `apb`/`apb5` masters with `APBMaster`/`APB5Master`; `master_read`/ `master_write` unwrap the APB transaction and turn `PSLVERR` into `AxiResponseError(resp=2)` – APB cannot tell SLVERR from DECERR, so 2 is the honest code.

13.8 Verification Anchors

- Generator unit tests: `bin/tests/test_generator_pkg.py` (feature gating, poison/atomic connectivity rules, generation smoke).
- Sideband **values** end-to-end, driven per transaction by the AXI5 master BFM: `dv/tests/test_bridge_1x2_{rd,wr}_axi5n_sideband.py` (native path and drop path; the AXI5 slave BFM echoes trace on B/R).
- Sideband **through the arbiter**: `dv/tests/test_bridge_2x2_axi5_sideband_arb.py` – two AXI5 masters with distinct NSAIDs contend for the AXI5 slave; every slave-side AW/AR NSAID must belong to its issuing master and the counts must match.
- Sideband **across a width converter**: `dv/tests/test_bridge_1x2_rd_axi5w_sideband.py` – 32b AXI5 master into a 64b AXI4 slave; data round-trips, trace returns 0.
- Atomics, filtered: `dv/tests/test_bridge_1x2_wr_axi5a_atomics.py` (store forwards; load, swap AND compare answered DECERR by the filter, via `atomic_operation`; a burst of 12 load-class atomics – more than the filter's 8-deep read-return queue – all complete and the port stays usable; BIDUNQ follows AWIDUNQ on the local DECERRs) and `val/amba/test_axi5_atomic_filter.py`.
- Unique-ID indicators: `test_bridge_2x2_axi5_sideband_arb.py`, `test_bridge_1x2_rd_axi5{n,w}_sideband.py`, `test_bridge_1x2_rd_axi5_bfm5.py` (BIDUNQ/RIDUNQ follow the request on native, AXI4, converter paths) and `test_bridge_1x2_rw_axi5a_atomics.py` (an atomic's RIDUNQ and BIDUNQ).
- AWAKEUP: `dv/tests/test_bridge_2x2_axi5_wakeup.py` (forwarding ahead of any request, held on every request cycle including traffic from a master without AWAKEUP, low when idle).
- Atomics, read-return: `dv/tests/test_bridge_1x2_rw_axi5a_atomics.py` – load ADD/SET/UMAX, swap, compare (match and mismatch) on both slaves, each checked three ways (R data is the pre-op value, memory holds the post-op value, a plain read agrees), then reads and atomics in flight together across and within slaves. The slave BFM performs the operation; the AXI5 checker treats a read-return atomic as an outstanding read and flags any R beat nobody requested. `val/amba/test_axi5_atomic_rr_tracker.py` covers the tracker alone.

Mutation-checked: with the tracker's `hit` removed from `rid_valid`, the R beat is never routed and the test fails on a read-return timeout.

- Lite requesters: `dv/tests/test_bridge_2x2_lite_req_sideband.py` on `bridge_2x2_lite_req` (AXI4-Lite + AXI5-Lite masters, 64b AXI4 + AXI5-Lite slaves) – `user`/`lock` from the AXI5-Lite master seen at the AXI5-Lite slave and absent from the AXI4-Lite master's traffic, terminated groups reading 0, both halves of a 64-bit row from both Lite masters (the aligner), and both ID-less masters in flight at one slave.
- APB requesters: `dv/tests/test_bridge_2x3_apb_req_paths.py` on `bridge_2x3_apb_req` (APB4 + APB5 masters, AXI4 + AXI5-Lite + APB4 slaves) – PSLVERR for unmapped addresses with the port working after, PPROT at the AXI4 slave, the APB5 USER bit at the AXI5-Lite slave and 0 from the APB4 master, APB in / APB out through the fabric from both requesters interleaved. The converters alone: `projects/components/utility-ip/converters/dv/tests/test_apb{4,5}_to_axi4.py`.
- AXI5 compliance at the boundary: every generated TB arms an `AXI5ComplianceChecker` on each AXI5 master port and every generated test asserts zero violations before PASSED; `dv/tests/test_bridge_1x2_rd_axi5_bfm5.py` is the read-side sign-off.

14 Bridge Arbiter Finite State Machines

14.1 Overview

The Bridge AXI4 crossbar arbitrates per slave, not per master — each slave owns two dedicated arbiters:

- **AW Arbiter** - Write Address channel arbitration
- **AR Arbiter** - Read Address channel arbitration

Total FSM Count: `2 × NUM_SLAVES`

Each arbiter is a simple 2-state FSM. Round-robin access keeps any master from starving, and two states keep the behavior easy to reason about — there's real value in an arbiter you can hold in your head.

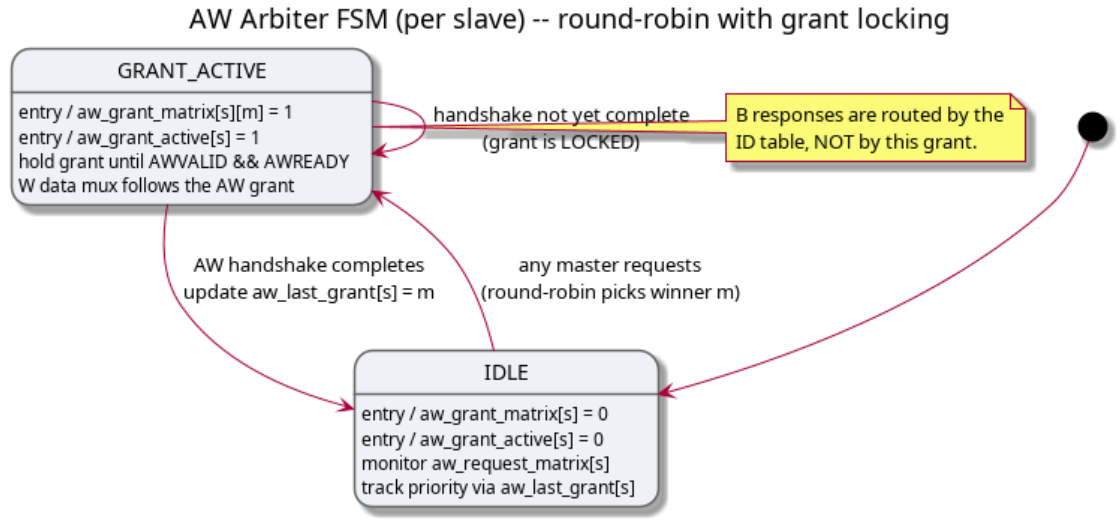
14.2 AW Channel Arbiter FSM

Purpose: Arbitrate write address requests from multiple masters to a single slave

States: 2 (IDLE, GRANT_ACTIVE)

Algorithm: Round-robin with grant locking

14.2.1 Figure 3.1: AW Arbiter FSM



AW Arbiter FSM

State Descriptions:

IDLE State: - **Entry Actions:** - `aw_grant_matrix[s] = 0` - No grant issued - `aw_grant_active[s] = 0` - Arbiter inactive - **Behavior:** - Monitor `aw_request_matrix[s]` for master requests - Track round-robin priority via `aw_last_grant[s]` - Remain in IDLE until at least one master requests access

GRANT_ACTIVE State: - **Entry Actions:** - `aw_grant_matrix[s][m] = 1` - Issue grant to winning master - `aw_grant_active[s] = 1` - Mark arbiter as active - **Behavior:** - Hold grant until AW handshake completes (`AWVALID && AWREADY`) - W channel data multiplexing follows AW grant - B channel response routes by bridge_id FIFO position (no ID table, not grant-based)

State Transitions:

AW arbiter FSM transitions

From	To	Condition	Description
IDLE	GRANT_ACTIVE	`| aw_request_matrix[s]| > 0`	At least one master requesting access

From	To	Condition	Description
GRANT_ACTIVE	IDLE	`m_axi_awvalid[s] && m_axi_awready[s]`	AW handshake complete

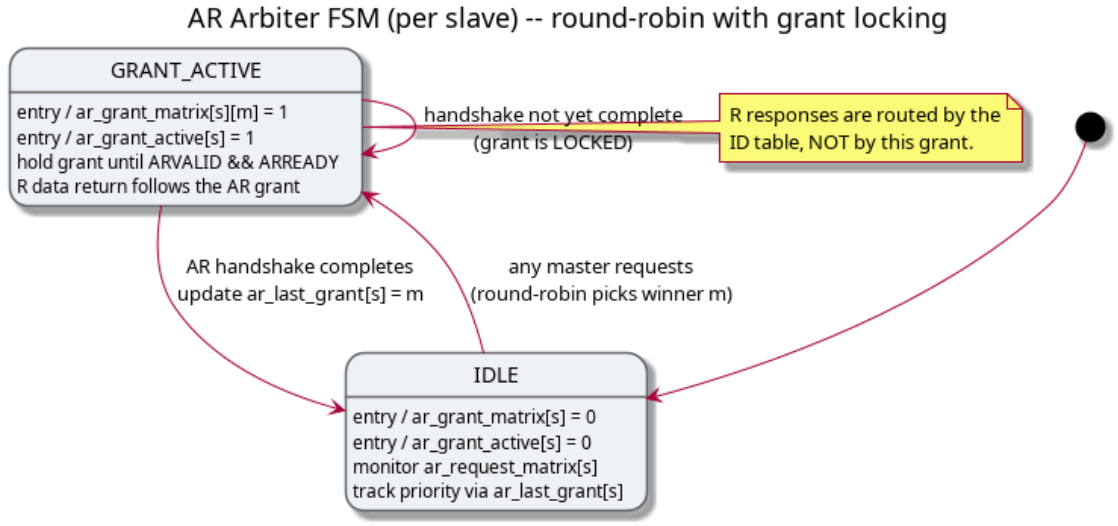
14.3 AR Channel Arbiter FSM

Purpose: Arbitrate read address requests from multiple masters to a single slave

States: 2 (IDLE, GRANT_ACTIVE)

Algorithm: Round-robin with grant locking (same as AW)

14.3.1 Figure 3.2: AR Arbiter FSM



AR Arbiter FSM

State Descriptions:

IDLE State: - **Entry Actions:** - `ar_grant_matrix[s] = 0` - No grant issued - `ar_grant_active[s] = 0` - Arbiter inactive - **Behavior:** - Monitor `ar_request_matrix[s]` for master requests - Track round-robin priority via `ar_last_grant[s]` - Independent from AW arbiter state

GRANT_ACTIVE State: - **Entry Actions:** - `ar_grant_matrix[s][m] = 1` - Issue grant to winning master - `ar_grant_active[s] = 1` - Mark arbiter as active - **Behavior:** - Hold grant until AR handshake completes (`ARVALID && ARREADY`) - R channel data multiplexing follows AR grant - R channel response routes by bridge_id FIFO position (no ID table)

State Transitions:

AR arbiter FSM transitions

From	To	Condition	Description
IDLE	GRANT_ACTIVE	`| ar_request_matrix[s]| > 0`	At least one master requesting

From	To	Condition	Description
GRANT_ACTIVE	IDLE	`m_axi_arvalid[s] && m_axi_arready[s]`	read AR handshake complete

14.4 Round-Robin Arbitration Algorithm

Algorithm Description:

Both AW and AR arbiters use the same fair round-robin arbitration algorithm — learn it once and you've learned both:

```
Pseudocode:
1. Start search from (last_grant[s] + 1) % NUM_MASTERS
2. Search cyclically through all masters
3. Select first master with pending request
4. Update last_grant[s] = winning_master
5. Issue grant
```

Example with 4 Masters:

```
Initial state: last_grant[0] = 0

Request Pattern:
  Master 0: request
  Master 1: request
  Master 2: no request
  Master 3: request

Arbitration Sequence:
  Grant 1: Master 1 (start from master 1, first requester)
          last_grant = 1

  Grant 2: Master 3 (start from master 2, find master 3)
          last_grant = 3

  Grant 3: Master 0 (start from master 0, first requester)
          last_grant = 0

  Grant 4: Master 1 (start from master 1, first requester)
          last_grant = 1
```

Fairness Properties:

1. **No Starvation** - Every requesting master will eventually receive grant
2. **Priority Rotation** - Priority shifts after each grant
3. **Predictable Latency** - Maximum wait time = (NUM_MASTERS - 1) × grant_duration
4. **Equal Opportunity** - All masters treated equally

14.5 Grant Locking Mechanism

Purpose: Prevent grant changes mid-transaction

AW Grant Locking:

```
// AW grant locked until address handshake completes
always_ff @(posedge aclk or negedge aresetn) begin
    if (!aresetn) begin
        aw_grant_active[s] <= 1'b0;
        aw_grant_matrix[s] <= '0;
    end else begin
        if (aw_grant_active[s]) begin
            // GRANT_ACTIVE state: Hold grant until handshake
            if (m_axi_awvalid[s] && m_axi_awready[s]) begin
                aw_grant_active[s] <= 1'b0;  // Return to IDLE
                aw_grant_matrix[s] <= '0;
            end
        end else if (|aw_request_matrix[s]) begin
            // IDLE state: Arbitrate if requests pending
            aw_grant_matrix[s] <=
round_robin_select(aw_request_matrix[s]);
            aw_grant_active[s] <= 1'b1;  // Enter GRANT_ACTIVE
        end
    end
end
```

AR Grant Locking:

```
// AR grant locked until address handshake completes
always_ff @(posedge aclk or negedge aresetn) begin
    if (!aresetn) begin
        ar_grant_active[s] <= 1'b0;
        ar_grant_matrix[s] <= '0;
    end else begin
        if (ar_grant_active[s]) begin
            // GRANT_ACTIVE state: Hold grant until handshake
            if (m_axi_arvalid[s] && m_axi_arready[s]) begin
                ar_grant_active[s] <= 1'b0;  // Return to IDLE
                ar_grant_matrix[s] <= '0;
            end
        end else if (|ar_request_matrix[s]) begin
            // IDLE state: Arbitrate if requests pending
            ar_grant_matrix[s] <=
round_robin_select(ar_request_matrix[s]);
            ar_grant_active[s] <= 1'b1;  // Enter GRANT_ACTIVE
        end
    end
end
```

Why Grant Locking Matters:

1. **Protocol Compliance** - AXI4 spec requires stable AWID/ARID during handshake
2. **Data Integrity** - W channel must follow granted master's AW request
3. **Response Routing** - B/R channels need consistent transaction tracking
4. **Simplicity** - Single grant active per slave prevents mux conflicts

14.6 Independent Read/Write Arbitration

Here's the design decision that pays for itself: the AW and AR arbiters operate completely independently.

Benefits:

1. **No Head-of-Line Blocking:**
   - Read requests don't wait for write completion
   - Write requests don't wait for read completion
2. **Parallel Operation:**
   - Master 0 can write to Slave 0 (AW path)
   - Master 1 can read from Slave 0 (AR path)
   - Both transactions proceed simultaneously
3. **Resource Efficiency:**
   - Full utilization of read/write bandwidth
   - No artificial serialization

Example Scenario:

```
Time T0: Master 0 issues write to Slave 0
         - AW arbiter grants to Master 0
         - AW_GRANT_ACTIVE = 1
         - W data follows

Time T1: Master 1 issues read to Slave 0 (during Master 0 write)
         - AR arbiter grants to Master 1 (independent!)
         - AR_GRANT_ACTIVE = 1
         - R data returns

Result: Read and write happen in parallel - no blocking
```

14.7 FSM Instance Breakdown by Configuration

2×2 Configuration (2 masters, 2 slaves): - Slave 0 AW Arbiter: 1 FSM - Slave 0 AR Arbiter: 1 FSM - Slave 1 AW Arbiter: 1 FSM - Slave 1 AR Arbiter: 1 FSM - **Total: 4 FSMs**

4×4 Configuration (4 masters, 4 slaves): - 4 slaves × 2 arbiters per slave = **8 FSMs**

8×8 Configuration (8 masters, 8 slaves): - 8 slaves × 2 arbiters per slave = **16 FSMs**

Scalability: - FSM count scales linearly with NUM_SLAVES - Arbitration complexity scales with NUM_MASTERS (search time) - Synthesis impact minimal for up to 16×16 configurations

14.8 Performance Characteristics

Latency:

- **Best Case:** 1 clock cycle (IDLE → GRANT_ACTIVE → IDLE with immediate handshake)
- **Worst Case:** (NUM_MASTERS) clock cycles (round-robin search + contention)
- **Average Case:** ~(NUM_MASTERS / 2) clock cycles

Throughput:

- **Back-to-Back Grants:** Possible if different masters (no wait)
- **Same Master Repeated:** 1 cycle minimum between grants (FSM state change)

Fairness:

- **Guaranteed:** Every master gets grant within (NUM_MASTERS - 1) arbitration cycles
- **No Priority:** All masters treated equally (can be extended for QoS)

14.9 Comparison with Other Crossbar Arbiters

Arbiter comparison

Feature	Bridge Arbiter	APB Crossbar	Commercial Tools
States	2 (IDLE, GRANT_ACTIVE)	1 (passthrough)	3-5 (complex)
Algorithm	Round-robin	N/A (APB is 1:1)	Priority, QoS, weighted
Grant Locking	Yes (until handshake)	N/A	Burst-aware locking
Read/Write Indep	Yes (2 FSMs/slave)	N/A	Yes
Complexity	Low	Minimal	High
Predictability	High (round-robin)	N/A	Medium (priority-based)

Simplicity Trade-off:

- **Advantages:**
  - Easy to verify (2-state FSM)
  - Predictable latency
  - Fair resource allocation
- **Limitations (future enhancements):**
  - No Quality-of-Service (QoS) support
  - No priority levels
  - No weighted arbitration

14.10 Future Enhancements (Phase 3+)

QoS Support:

```
// Master QoS priority field (AXI4 optional)
input logic [3:0] s_axi_awqos [NUM_MASTERS];

// Arbitration considers QoS priority
function automatic [NUM_MASTERS-1:0] qos_arbitrate(
    input logic [NUM_MASTERS-1:0] requests,
    input logic [3:0] qos [NUM_MASTERS]
);
    // Higher QoS value = higher priority
    // Round-robin among same QoS level
endfunction
```

Weighted Arbitration:

```
// Master weight configuration
parameter int MASTER_WEIGHTS [NUM_MASTERS] = '{1, 2, 1, 4};

// Grant frequency proportional to weight
// Master 3 gets 4× more grants than Master 0
```

Burst-Aware Locking:

```
// Lock grant until entire burst completes
// Currently: Lock until address handshake
// Enhanced: Lock until WLAST (write) or RLAST (read)
```

14.11 Related Documentation

See Also: - 3.1 - Module Structure - Generated RTL organization - 3.3 - Crossbar Core - Internal crossbar instantiation

Reference: - ARM AMBA AXI4 Specification (IHI 0022E) - Section A7 (Arbitration)

Next: 3.3 - Crossbar Core

---

15 Transaction Tracking FSMs

15.1 Overview

Despite the chapter title, the generated bridge tracks outstanding transactions with FIFOs and a CAM, not state machines: the W destination and W owner FIFOs in the crossbar, the `aw_trk` / `ar_trk` / `w_trk` slave-select FIFOs in each master adapter, and the per-slave `bridge_id` tracker (`bridge_cam` for multi-master AXI4/AXI5 slaves, an in-order FIFO otherwise). The FSM designs this chapter used to show are kept below only where they are marked as not built.

15.2 Write Data Tracking (FIFO, not an FSM)

15.2.1 Purpose

The W channel carries no address of its own, so the bridge remembers which slave each AW selected and steers the data beats to match.

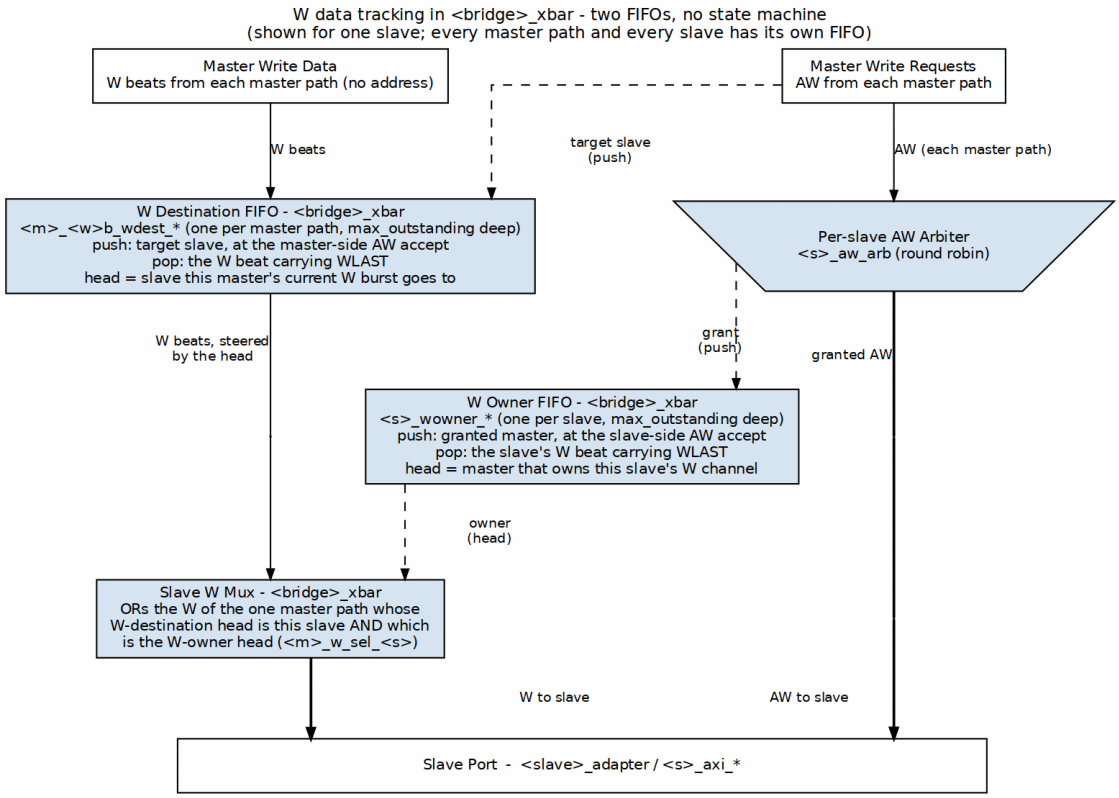
15.2.2 Implementation

There is no IDLE/WRITING state machine. Each master width-path has a `max_outstanding`-entry W destination FIFO in the crossbar (`<master>_<w>b_wdest_*`), and each master adapter has a matching `w_trk` FIFO of slave selects:

```
// bridge_2x2_rw_xbar.sv
wire cpu_32b_wdest_push = cpu_32b_awvalid && cpu_32b_awready;
// record target at AW
wire cpu_32b_wdest_pop  = cpu_32b_wvalid && cpu_32b_wready &&
cpu_32b_w.last;  // retire at WLAST
assign cpu_32b_w_to_ddr = cpu_32b_wdest_valid && (cpu_32b_wdest_head
== 2'd0);
```

Because every accepted AW is queued, a master may issue further AWs before the first burst's W beats finish; each W burst follows its own AW in order.

15.2.3 Figure 3.3: W Data Tracking FIFOs



W Data Tracking FIFOs

W beats run down the left: each master path's W destination FIFO names the slave its current burst goes to, and the slave's W mux takes the beats of the master that is also at the head of that slave's W owner FIFO. The AW requests run down the center through the per-slave arbiter. The dashed pushes come from the AW side: the master-side AW accept pushes the target slave, and the slave-side AW accept pushes the granted master. Both FIFOs pop on the W beat carrying WLAST, and several bursts can be queued at once.

15.3 Response Pending Counter

Not built. There is no per-master outstanding-transaction counter and no flow control derived from one. Outstanding depth is bounded by the per-slave bridge_id tracker (CAM or FIFO, `max_outstanding` deep, default 16), whose fullness gates the address handshake (bridge BUG-009, was BRIDGE-011). Searching the generated RTL for a response-pending counter returns nothing.

15.3.1 Purpose

Track number of outstanding transactions per master for flow control.

15.3.2 Implementation

```
logic [7:0] r_outstanding [NUM_MASTERS];

// Increment on AR/AW acceptance
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        for (int i = 0; i < NUM_MASTERS; i++)
            r_outstanding[i] <= '0;
    end else begin
        for (int m = 0; m < NUM_MASTERS; m++) begin
            logic inc = (ar_accepted[m] || aw_accepted[m]);
            logic dec = (r_delivered[m] || b_delivered[m]);

            case ({inc, dec})
                2'b10: r_outstanding[m] <= r_outstanding[m] + 1;
                2'b01: r_outstanding[m] <= r_outstanding[m] - 1;
                default: r_outstanding[m] <= r_outstanding[m];
            endcase
        end
    end
end

// Backpressure when outstanding limit reached
assign accept_ar[m] = (r_outstanding[m] < MAX_OUTSTANDING);
assign accept_aw[m] = (r_outstanding[m] < MAX_OUTSTANDING);
```

15.4 Burst Counter FSM

Not built. No generated file contains a burst-beat counter; the read tracker frees its entry on the R beat with RLAST, so no count is needed.

15.4.1 Purpose

Track burst beat count for responses spanning multiple cycles.

15.4.2 States

Transaction tracking states

State	Description
IDLE	No burst in progress
COUNTING	Tracking burst beats

15.4.3 Implementation

```
logic [7:0] r_burst_count;
logic r_burst_active;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_burst_count <= '0;
        r_burst_active <= 1'b0;
    end else begin
        if (!r_burst_active && rvalid && rready) begin
            // Start new burst
            r_burst_active <= !rlast;
            r_burst_count <= rlast ? '0 : 8'd1;
        end else if (r_burst_active && rvalid && rready) begin
            // Continue burst
            r_burst_count <= r_burst_count + 1;
            r_burst_active <= !rlast;
        end
    end
end
```

15.5 Related Documentation

- Arbiter FSMs - Address channel arbitration
- ID Management - the per-slave response tracker (`bridge_cam` or in-order FIFO)

16 CAM Architecture

16.1 Overview

`bridge_cam` (`rtl/bridge_cam.sv`, hand-written) is the response tracker in every slave adapter that has to route by ID: an AXI4/AXI5 slave of a bridge with more than one master, or any slave with `enable_ooo = true` (`SlaveAdapterGenerator.use_cam`). Each such adapter instantiates two, `u_wr_cam` and `u_rd_cam`. Single-master bridges, protocol-shim slaves and the internal subtractive slave use an in-order FIFO instead (see ID Tracking).

> An earlier revision of this chapter said the CAM was never built, that it was instantiated in zero bridges, and that IDs passed through at equal width. Since bridge TASK-005 the master adapter widens IDs to `{BRIDGE_ID, id}` (`cpu_m_axi_awid` 4 bits, `ddr_s_axi_awid` 5 bits in `bridge_2x2_rw`) and multi-master AXI4/AXI5 slaves are tracked here. The sizing formula and the block-RAM sequential search below were never built and are marked as such.

16.2 Functional Description

16.2.1 What the CAM Does

Transaction tracking. Each entry stores:

- the fabric transaction ID `{BRIDGE_ID, id}` (tag, `TAG_WIDTH = ID_WIDTH`)
- the originating master index (`DATA_WIDTH = BRIDGE_ID_WIDTH`)
- an ordering count, so one ID may be outstanding more than once

Response routing. When a response arrives:

1. The returned BID / RID is the deallocate tag
2. `deallocate_data` returns the originating master index, in the same cycle
3. The crossbar steers the B / R beat by that index (`bid_bridge_id` / `rid_bridge_id`)

The slave may therefore complete different IDs in any order.

16.2.2 Figure 4.1: CAM Entry Format

bridge_cam entry - one per outstanding transaction, DEPTH = the slave's max_outstanding (default 16) per CAM (u_wr_cam and u_rd_cam in each CAM-tracked slave adapter)

valid	tag	data	count
1 bit	TAG_WIDTH = ID_WIDTH (XBAR_ID_WIDTH)	DATA_WIDTH = BRIDGE_ID_WIDTH	$clog2(DEPTH) + 1
r_valid[i]	r_tag_array[i]	r_data_array[i]	r_count_array[i]
entry occupied	fabric AXI ID {BRIDGE_ID, master id} (AWID / ARID)	issuing master's index (xbar_bridge_id_aw / _ar); returned as bid_ / rid_bridge_id	age among entries with the same tag (0 = oldest, retired first; ALLOW_DUPLICATES = 1)
1 bit	5 bits	1 bit	5 bits

Grey row: bridge_2x2_rw (4-bit master IDs, 2 masters) -> 12 bits per entry.

Allocate on the crossbar-side AW / AR handshake; look up and free on B, or on R with RLAST.

No timestamp, transaction-type or metadata field exists.

CAM Entry Format

16.2.3 CAM Sizing

As generated, every CAM is the slave's `max_outstanding` entries deep (default 16, power of two 2..64, bridge GitHub #119 L5) regardless of master count:

```
bridge_cam #(.TAG_WIDTH(ID_WIDTH),          // XBAR_ID_WIDTH, 5 in
bridge_2x2_rw
             .DATA_WIDTH(BRIDGE_ID_WIDTH),  // 1 in bridge_2x2_rw
             .DEPTH(16),
             .ALLOW_DUPLICATES(1),
             .PIPELINE_EVICT(0))
```

The formula once proposed here (`DEPTH = MAX_OUTSTANDING x NUM_MASTERS`, with a metadata field in each entry) was never built; there is no `max_outstanding` key and no metadata field.

16.2.4 Insertion (AW / AR Acceptance)

On the crossbar-side address handshake the slave adapter allocates the fabric ID against the granted master's index (from the generated `ddr_adapter.sv`):

```
.allocate     (xbar_ddr_axi_awvalid && xbar_ddr_axi_awready),
.allocate_tag (xbar_ddr_axi_awid),     // {BRIDGE_ID, id}
.allocate_data(xbar_bridge_id_aw),     // master index sideband
```

16.2.5 Lookup and Deletion (B / last R)

Lookup and deletion are one operation: the response handshake deallocates by the returned ID and returns the stored master index combinationally (`PIPELINE_EVICT = 0`):

```
// write side: every B; read side: the R beat with RLAST
.deallocate     (xbar_ddr_axi_bvalid && xbar_ddr_axi_bready),
.deallocate_tag (xbar_ddr_axi_bid),
.deallocate_valid(bid_valid),          // tag found
.deallocate_data(bid_bridge_id),       // routes B in the crossbar
```

A response whose ID is not in the CAM leaves `deallocate_valid` low and routes nowhere; it means the slave returned an ID it was never sent.

16.2.6 Duplicate IDs: the Ordering Counter (Mode 2)

`bridge_cam` with `ALLOW_DUPLICATES = 1` accepts several outstanding transactions under one ID and retires them in order. Every entry carries a small ordering counter, and the entries holding one tag always carry the counts `0..k-1`: the oldest is 0, a newcomer takes `k`, and when the oldest retires the survivors step down by one. Two consequences shape the logic:

- The count a newcomer needs is `k`, which is simply how many entries match its tag: a popcount of the match vector, a few LUT levels. It is not a search for the largest existing count. That scan (`DEPTH` chained comparators, hanging off the crossbar's arbitrated ARID) was the bridge's critical path when first synthesized: 40 LUT levels and a 32 ns data path on an Artix-7 -1 (HAS 6.4), fixed 2026-09-11 by counting instead.

- The entry to retire is the one match with count 0, which is one-hot by construction, so its index and its stored master come out of OR reductions rather than a priority chain.

An allocate and a same-tag retire in one cycle take `k - 1`, because the survivors step down in that same cycle (MAS 2.3).

16.2.7 Implementation

Only one form exists: a parallel comparator array in distributed RAM and flops (no block RAM), every entry compared at once:

```
// bridge_cam.sv (simplified): one match vector per lookup port
w_evict_matches[i] = r_valid[i] && (r_tag_array[i] == deallocate_tag);
w_entry_matches[i] = r_valid[i] && (r_tag_array[i] == allocate_tag);
```

The block-RAM sequential-search variant that an earlier revision proposed for deeper tables was never built.

16.2.8 CAM Full

The CAM never overflows; it backpressures first. Its `tags_full` masks the slave adapter's ready toward the crossbar (bridge BUG-009, was BRIDGE-011):

```
assign xbar_ddr_axi_awready = w_sub_awready && !wr_trk_full;  //
wr_trk_full = u_wr_cam.tags_full
```

The address phase waits until a B or last R frees an entry. Nothing is dropped and no error response is generated.

16.3 Related Modules

- ID Tracking - which slaves use the CAM and which use the in-order FIFO
- Response Routing - how the crossbar uses the returned master index

17 Response Tracking

17.1 Overview

A multi-master fabric has to know which master a response belongs to. The bridge does two things:

1. **Widens the ID.** With more than one master, each master adapter sends `{BRIDGE_ID, id}` into the fabric (bridge TASK-005), so IDs are unique across masters at every slave. In `bridge_2x2_rw` the 4-bit `cpu_m_axi_awid` becomes the 5-bit `ddr_s_axi_awid`. On B / R the master adapter returns the low bits, so the master sees its own ID.
2. **Records the issuing master per slave.** The granted master's index rides beside the request as `xbar_bridge_id_aw` / `_ar` and is stored by the slave adapter's tracker at the address handshake; the crossbar steers B / R by the index the tracker returns (`bid_bridge_id` / `rid_bridge_id`).

The tracker comes in two forms, chosen at generation:

Slave response tracker selection

Slave	Tracker	Completion order allowed
AXI4 / AXI5 slave, more than one master	`bridge_cam` by fabric ID (CAM Architecture)	any order across IDs
any slave with `enable_ooo = true`	`bridge_cam`	any order across IDs
single-master bridge, apb / axil / wb shim slave, subtractive slave	in-order `bridge_id` FIFO	request order across all IDs

```
// FIFO form
wr_fifo[wr_ptr[...]] <= xbar_bridge_id_aw;      // push on AW accept
assign bid_bridge_id  = wr_fifo[rd_ptr[...]];   // route by the HEAD
```

A FIFO-tracked slave must return B/R in request order across ALL IDs; the slaves that get the FIFO are in order by construction or have one master. Every FIFO-tracked adapter carries a SIMULATION-ONLY check that compares the returned BID/RID against the head and $error()s on a mismatch (bridge BUG-008, was BRIDGE-010); it cannot fire in silicon.

17.2 Functional Description

17.2.1 Figure 4.2: ID Table Structure



ID Table Structure

The figure shows the contents of one slave's CAM in a 4-master bridge: the 6-bit fabric ID (2 master-index bits over a 4-bit master ID), the stored master index and the valid bit. `bridge_cam` also keeps a per-entry ordering count for duplicate IDs.

17.2.2 Sizing

Every tracker, CAM or FIFO, is the slave's `max_outstanding` entries (default 16; configurable since bridge GitHub #119 L5) per direction per slave adapter:

```
CAM:   TAG_WIDTH = XBAR_ID_WIDTH, DATA_WIDTH = BRIDGE_ID_WIDTH, DEPTH
= 16
FIFO:  BRIDGE_ID_WIDTH bits x 16 (WR_FIFO_DEPTH / RD_FIFO_DEPTH)
```

When it is full the slave adapter drops its AWREADY / ARREADY toward the crossbar (bridge BUG-009). There is no `max_outstanding` key.

17.2.3 ID Extension (Master Adapter)

```
// <master>_adapter.sv
assign xbar_axi_awid = {BRIDGE_ID_WIDTH'(BRIDGE_ID),
MASTER_ID_WIDTH'(fub_axi_awid)};
assign xbar_axi_arid = {BRIDGE_ID_WIDTH'(BRIDGE_ID),
MASTER_ID_WIDTH'(fub_axi_arid)};

// Example, 4 masters: master 2, ID 4'hA -> fabric ID 6'b10_1010
```

The slave port is declared at `XBAR_ID_WIDTH`; the config validator rejects an AXI3/AXI4/AXI5 slave whose `id_width` is narrower. A single-master bridge prepends nothing.

17.2.4 ID Stripping and Routing

The prefix bits are not decoded to choose the master; the tracker's returned index is. The master adapter only narrows the ID:

```
fub_axi_bid = cpu_32b_b.id[3:0];   // bridge_2x2_rw: drop the 1-bit
prefix
```

17.2.5 Tracker Operations

Tracker operations by event

Event	CAM form	FIFO form
AW / AR accepted from crossbar	allocate tag = fabric ID, data = `bridge_id`	push `bridge_id`
B handshake	deallocate by BID, return master index	pop head
R handshake with RLAST	deallocate by RID, return	pop head

Event	CAM form	FIFO form
	master index	
full	`tags_full` holds AWREADY / ARREADY low	not-full gate holds AWREADY / ARREADY low

17.2.6 Multi-ID Considerations

17.2.6.1 Same Master ID, Different Masters

```
Master 0 issues ID=5 -> fabric ID 00_0101
Master 1 issues ID=5 -> fabric ID 01_0101
Master 2 issues ID=5 -> fabric ID 10_0101
```

All three can be outstanding at one slave at once; the prefix keeps them distinct, and a CAM-tracked slave may answer them in any order.

17.2.6.2 Same Fabric ID Twice

One master may reuse an ID while an earlier transaction with it is still outstanding (AXI allows this; same-ID responses come back in order). The CAM is built with `ALLOW_DUPLICATES = 1` and retires same-tag entries oldest first using the per-entry ordering count.

17.3 Design Notes

17.3.1 Resource Utilization

Per CAM-tracked slave adapter (default depth): two 16-entry CAMs of `XBAR_ID_WIDTH + BRIDGE_ID_WIDTH` bits plus valid and ordering count, in distributed RAM and flops (no block RAM). Per FIFO-tracked slave adapter: two 16-entry FIFOs of `BRIDGE_ID_WIDTH` bits. Measured totals per fixture are in HAS 5.3.

17.4 Related Modules

- CAM Architecture - `bridge_cam` details
- Response Routing - the crossbar's B / R muxes

18 Width Converters

18.1 Overview

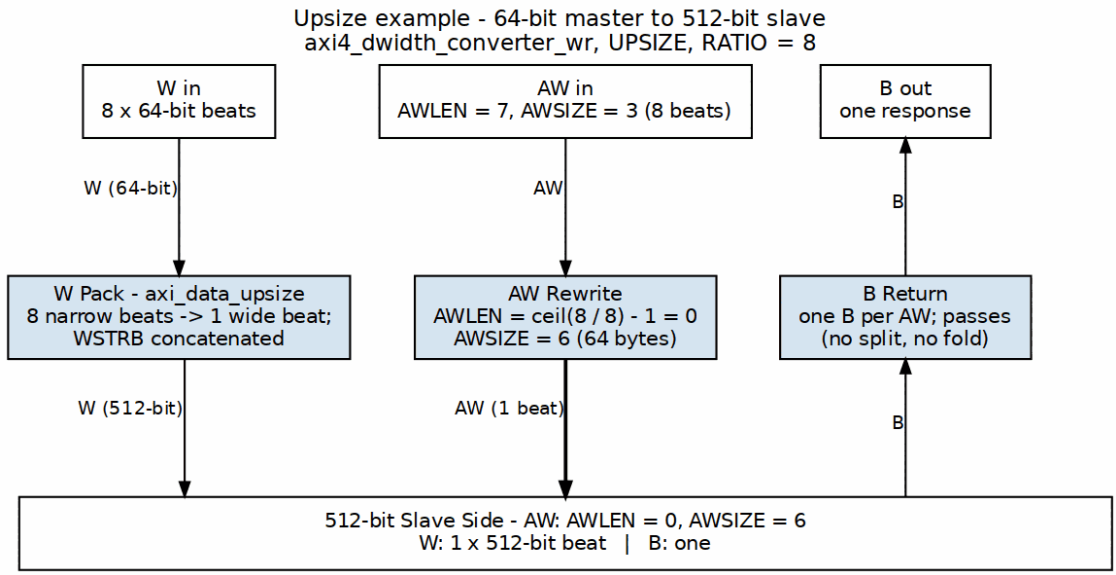
When a master and slave disagree on data width, a converter sits between them: an upsizer for narrow-to-wide, a downsizer for wide-to-narrow. The generator inserts them automatically wherever the configuration needs one — you never instantiate these by hand.

18.2 Functional Description

18.2.1 Upsize Converter

Purpose. Convert narrow master data to wide slave interface.

18.2.2 Figure 5.1: Upsize Converter (64-bit to 512-bit)



Upsize Converter

18.2.3 Implementation

The packing loop is simple in principle: fill a wide buffer one narrow beat at a time, then drain it. One thing before you read the sketch — there is no `width_upsize` module in the tree. The real converters are `axi_data_upsize` / `axi_data_dnsize` (beat packing) wrapped by `axi4_dwidth_converter_rd` / `axi4_dwidth_converter_wr`:

```
// NOTE: there is no `width_upsize` module. The real converters are
// `axi_data_upsize` / `axi_data_dnsize` (beat packing) wrapped by
// `axi4_dwidth_converter_rd` / `axi4_dwidth_converter_wr`. Sketch
only:
module width_upsize #(
    parameter IN_WIDTH = 64,
    parameter OUT_WIDTH = 512
) (
    input  logic clk, rst_n,

    // Narrow input
    input  logic                    s_valid,
    output logic                    s_ready,
```

```
    input  logic [IN_WIDTH-1:0]     s_data,
    input  logic                    s_last,

    // Wide output
    output logic                    m_valid,
    input  logic                    m_ready,
    output logic [OUT_WIDTH-1:0]    m_data,
    output logic                    m_last
);

    localparam RATIO = OUT_WIDTH / IN_WIDTH;

    logic [OUT_WIDTH-1:0] r_buffer;
    logic [$clog2(RATIO)-1:0] r_count;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_count <= '0;
            r_buffer <= '0;
        end else if (s_valid && s_ready) begin
            // Pack input into buffer
            r_buffer[r_count * IN_WIDTH +: IN_WIDTH] <= s_data;

            if (s_last || r_count == RATIO - 1) begin
                r_count <= '0;
            end else begin
                r_count <= r_count + 1;
            end
        end
    end

    // Output when buffer full or last beat
    assign m_valid = (r_count == RATIO - 1) || s_last;
    assign m_data = r_buffer;
    assign s_ready = !m_valid || m_ready;

endmodule
```

18.2.4 Burst Length Conversion

The burst math follows directly — eight narrow beats pack into one wide beat:

Burst Length Conversion Examples

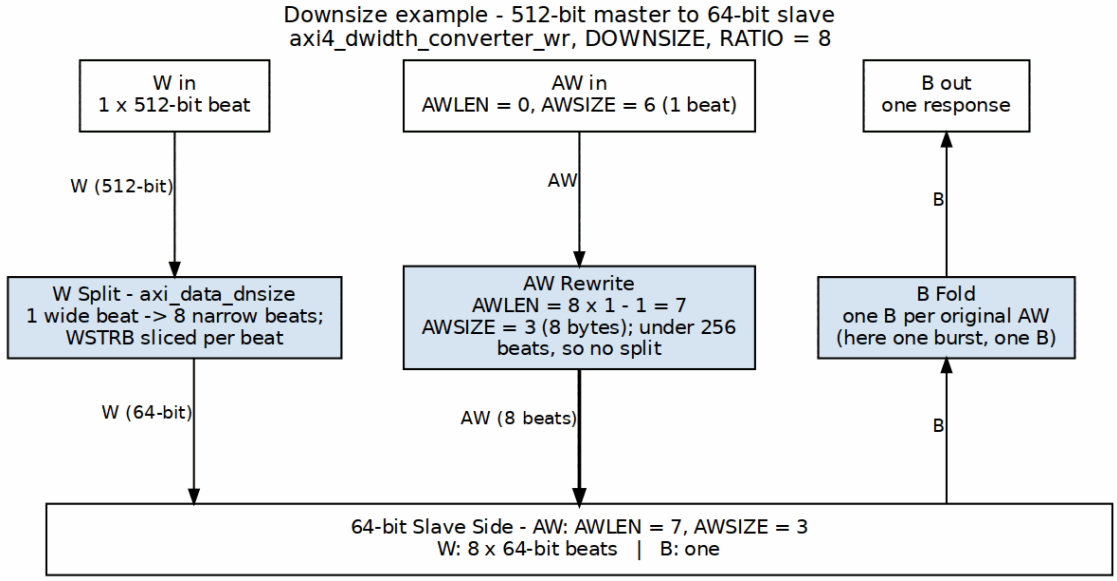
Master AWLEN	Master Beats	Slave AWLEN	Slave Beats
7 (8 beats)	8 × 64b	0 (1 beat)	1 × 512b

Master AWLEN	Master Beats	Slave AWLEN	Slave Beats
15 (16 beats)	16 × 64b	1 (2 beats)	2 × 512b
255 (256 beats)	256 × 64b	31 (32 beats)	32 × 512b

18.2.5 Downsize Converter

Purpose. Convert wide master data to narrow slave interface.

18.2.6 Figure 5.2: Downsize Converter (512-bit to 64-bit)



Downsize Converter

18.2.7 Implementation

The downsizer is the mirror image: latch one wide word, then slice it out one narrow beat at a time.

```
module width_downsize #(
    parameter IN_WIDTH = 512,
    parameter OUT_WIDTH = 64
) (
    input  logic clk, rst_n,

    // Wide input
    input  logic                    s_valid,
    output logic                    s_ready,
    input  logic [IN_WIDTH-1:0]     s_data,
    input  logic                    s_last,
```

```
    // Narrow output
    output logic                    m_valid,
    input  logic                    m_ready,
    output logic [OUT_WIDTH-1:0]    m_data,
    output logic                    m_last
);

    localparam RATIO = IN_WIDTH / OUT_WIDTH;

    logic [IN_WIDTH-1:0] r_buffer;
    logic [$clog2(RATIO)-1:0] r_count;
    logic r_active;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_count <= '0;
            r_buffer <= '0;
            r_active <= 1'b0;
        end else begin
            if (!r_active && s_valid) begin
                // Load new wide word
                r_buffer <= s_data;
                r_active <= 1'b1;
                r_count <= '0;
            end else if (r_active && m_ready) begin
                if (r_count == RATIO - 1) begin
                    r_active <= 1'b0;
                end else begin
                    r_count <= r_count + 1;
                end
            end
        end
    end

    // Output from buffer slice
    assign m_valid = r_active;
    assign m_data = r_buffer[r_count * OUT_WIDTH +: OUT_WIDTH];
    assign m_last = r_active && (r_count == RATIO - 1);
    assign s_ready = !r_active;

endmodule
```

18.2.8 Strobe Handling

Data isn't the only thing that rescales — the byte strobes pack and split right alongside it.

18.2.8.1 Upsize Strobe Packing

```
// Pack 8-byte strobes into 64-byte strobes
logic [7:0] in_strb;   // 64-bit input
logic [63:0] out_strb; // 512-bit output

always_ff @(posedge clk) begin
    if (s_valid && s_ready) begin
        out_strb[r_count * 8 +: 8] <= in_strb;
    end
end
```

18.2.8.2 Downsize Strobe Splitting

```
// Split 64-byte strobes into 8-byte strobes
logic [63:0] in_strb;  // 512-bit input
logic [7:0] out_strb;  // 64-bit output

assign out_strb = in_strb[r_count * 8 +: 8];
```

18.2.9 AXIL-to-Wider-Slave Master-Side Alignment Converters

An AXI4-Lite master talking to a wider AXI4 slave through the bridge has an alignment problem: AXIL single-beat transactions don't know where they land in a wide word. The bridge generator emits master-side alignment converters (`axil_to_axi4_wide_align_{rd,wr}.sv`) to handle partial-word alignment on the AXIL side while producing properly-aligned transactions on the wide slave side.

18.2.9.1 Use Case Example

A concrete write makes the lane math clear:

```
32-bit AXIL Master → Bridge → 64-bit AXI4 Slave

Master writes 32-bit word to offset 0x04:
  Data = 0xAABBCCDD (on bits [31:0])

The converter picks the lane from the address, then ROW-ALIGNS the
address:
  slot      = addr[2]                 = 1      (SLOT_LSB=$clog2(4)=2,
SLOT_W=$clog2(2)=1)
  m_axi_wdata[63:32] = 0xAABBCCDD             (slot 1 -> the UPPER
half)
  m_axi_wstrb        = 0xF0                   (byte lanes 4..7)
  m_axi_awaddr       = 0x04 & ~0x7 = 0x00     (row_addr_mask clears
the lane bits)

  Slave sees: 0xAABBCCDD_00000000 at address 0x00, WSTRB 0xF0
```

```
The address the slave sees is 0x00, NOT 0x04: the lane bits move out
of the
address and into WSTRB. Byte lanes outside the strobe are don't-care,
shown
here as zero.
```

18.2.9.2 Architecture

Read Path: - Master issues AXIL read (no burst, single beat) - Converter passes address to slave (unchanged) - Slave returns 64-bit word - Converter extracts the relevant 32-bit slice for the master

Write Path: - Master issues AXIL write with address offset and 32-bit data - Converter computes byte-lane position within 64-bit word - Converter expands 32-bit strobe to 64-bit strobe (8 lanes) - Converter sends full 64-bit write to slave - Slave performs a 64-bit write with byte strobes

18.2.9.3 Modules

- **`axil_to_axi4_wide_align_rd.sv`**: Read path (address → data conversion)
- **`axil_to_axi4_wide_align_wr.sv`**: Write path (strobe expansion, data placement)

Note: These converters handle **width alignment**, not **protocol bridging**. Protocol conversion (AXIL→AXI4 burst restriction) is applied separately at the slave boundary (via `axi4_to_axil4_*` shims if needed).

18.3 Design Notes

18.3.1 Resource Utilization

Upsize Converter (64 to 512):

```
Registers: ~520 (buffer + control)
Logic: ~200 LEs (packing + control)
```

Downsize Converter (512 to 64):

```
Registers: ~520 (buffer + control)
Logic: ~150 LEs (selection + control)
```

The register counts are dominated by the data buffer — 512 bits of storage plus control, in either direction.

18.4 Related Modules

- Width Conversion Block - Block-level description
- APB Converters - Protocol conversion
- Generator-Emitted Conversion Shims - Master-side vs. slave-side shim placement

---

19 APB Converters

19.1 Overview

APB converters turn AXI4 transactions into APB transfers — the price a high-performance master pays to talk to a low-speed peripheral.

19.2 Functional Description

19.2.1 Conversion Requirements

19.2.1.1 Protocol Differences

The two protocols differ in ways that force real work on the converter:

AXI4 vs APB Protocol Comparison

Feature	AXI4	APB
Channels	5 (AW, W, B, AR, R)	1 (combined)
Bursts	Up to 256 beats	None (single)
Pipelining	Yes	No
Min cycles	1 per beat	2 per transfer

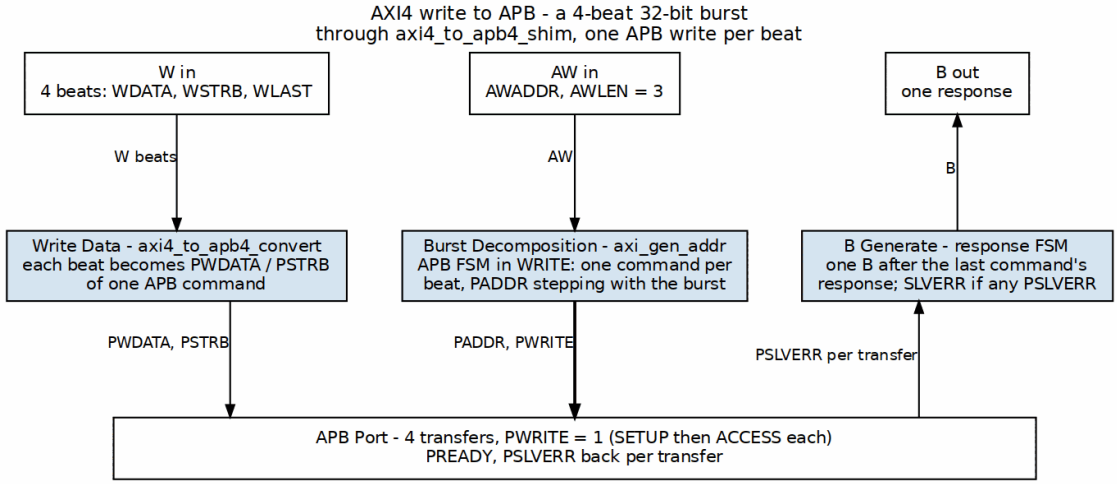
19.2.1.2 Conversion Strategy

The recipe, then:

1. Split AXI4 bursts into individual APB transfers
2. Serialize AW+W into APB write sequence
3. Convert AR into APB read sequence
4. Generate AXI4 responses from APB completions

19.2.2 Write Conversion

19.2.3 Figure 5.3: AXI4 Write to APB Write



AXI4 to APB Write

19.2.3.1 State Machine

The write path walks each beat through the two-cycle APB handshake:

```
typedef enum logic [2:0] {
    IDLE,
    AW_ACCEPT,
    W_ACCEPT,
    APB_SETUP,
    APB_ACCESS,
    B_GENERATE
} apb_wr_state_t;

apb_wr_state_t r_state;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_state <= IDLE;
    end else begin
        case (r_state)
            IDLE: if (awvalid) r_state <= AW_ACCEPT;

            AW_ACCEPT: begin
                if (awready) r_state <= W_ACCEPT;
            end

            W_ACCEPT: begin
                if (wvalid && wready) r_state <= APB_SETUP;
            end

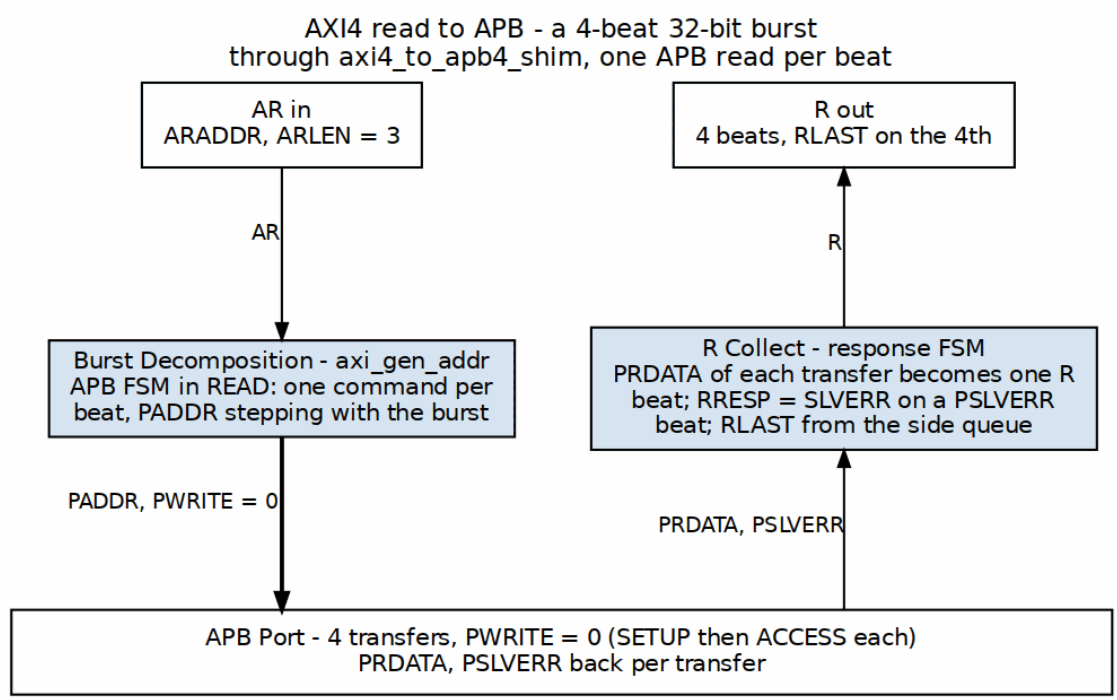
            APB_SETUP: r_state <= APB_ACCESS;

            APB_ACCESS: begin
                if (pready) begin
                    if (r_beat_count == r_awlen)
                        r_state <= B_GENERATE;
                    else
                        r_state <= W_ACCEPT;  // Next beat
                end
            end

            B_GENERATE: begin
                if (bready) r_state <= IDLE;
            end
        endcase
    end
end
```

19.2.4 Read Conversion

19.2.5 Figure 5.4: AXI4 Read to APB Read



AXI4 to APB Read

19.2.5.1 Read State Machine

The read path is the same shape, minus the W channel:

```
typedef enum logic [2:0] {
    IDLE,
    AR_ACCEPT,
    APB_SETUP,
    APB_ACCESS,
    R_GENERATE
} apb_rd_state_t;
```

19.2.6 Burst Handling

19.2.6.1 Burst to Single Conversion

Bursts don't exist on APB, so each beat becomes its own transfer with its own address:

```
AXI4 Burst (AWLEN=7, 8 beats):
  Beat 0: AWADDR + 0×SIZE
  Beat 1: AWADDR + 1×SIZE
  Beat 2: AWADDR + 2×SIZE
```

```
  ...
  Beat 7: AWADDR + 7×SIZE

APB Sequence:
  Transfer 0: PADDR = AWADDR + 0×SIZE
  Transfer 1: PADDR = AWADDR + 1×SIZE
  ...
  Transfer 7: PADDR = AWADDR + 7×SIZE
```

19.2.6.2 Address Calculation

Address generation honors all three burst types:

```
// Calculate APB address for each beat
logic [ADDR_WIDTH-1:0] apb_addr;
logic [7:0] beat_count;

always_comb begin
    case (r_burst_type)
        FIXED: apb_addr = r_base_addr;
        INCR:  apb_addr = r_base_addr + (beat_count << r_size);
        WRAP:  apb_addr = wrap_address(r_base_addr, beat_count,
r_size, r_len);
    endcase
end
```

19.2.7 Error Handling

19.2.7.1 APB Error to AXI4 Response

One error bit in, one of two response codes out:

APB to AXI4 Error Mapping

APB PSLVERR	AXI4 Response
0 (OK)	OKAY (2'b00)
1 (Error)	SLVERR (2'b10)

19.2.7.2 Error Propagation

Errors latch on first sight and ride along to the final response — one bad beat fails the burst:

```
// Latch first error in burst
logic r_error_seen;

always_ff @(posedge clk) begin
    if (r_state == APB_ACCESS && pready && pslverr)
        r_error_seen <= 1'b1;
```

```
    if (r_state == IDLE)
        r_error_seen <= 1'b0;
end

// Final response reflects any errors
assign bresp = r_error_seen ? 2'b10 : 2'b00;
```

19.3 Timing

APB costs you on both axes, and it's inherent to the two-cycle transfer — not something you can pipeline around.

19.3.1 Throughput Impact

```
AXI4 alone: 1 beat per cycle (peak)
AXI4 to APB: 1 beat per 2+ cycles (minimum)

Throughput reduction: ~50% or more
```

19.3.2 Latency Impact

```
AXI4 read: 1-2 cycles (typical)
AXI4-to-APB read: 4+ cycles minimum
  - 1 cycle: AR accept
  - 1 cycle: APB setup
  - 1+ cycles: APB access (PREADY)
  - 1 cycle: R generate
```

19.4 Design Notes

19.4.1 Resource Utilization

```
Logic Elements: ~300-400 LEs
Registers: ~100-150 regs
Block RAM: 0

Breakdown:
- State machine: ~50 LEs, ~20 regs
- Address calc: ~100 LEs, ~50 regs
- Beat counter: ~30 LEs, ~8 regs
- Data buffering: ~100 LEs, ~DATA_WIDTH regs
```

19.5 Related Modules

- Protocol Conversion Block - Block description
- Width Converters - Data width conversion

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

20 Module Structure

20.1 Overview

The generator turns configuration files into parameterized SystemVerilog modules, and every topology gets the same internal structure. Parameterized at generation time, mind you — not at elaboration, as the next section makes clear. Once you've read one generated bridge, you can find your way around any of them.

20.2 Parameters

20.2.1 Compile-Time Parameters

The generated top has no parameter list. The shared constants live in the per-bridge package `<bridge>_pkg.sv`; this is `bridge_2x2_rw_pkg.sv`:

```
localparam int NUM_MASTERS     = 2;
localparam int BRIDGE_ID_WIDTH = 1;  // $clog2(NUM_MASTERS): sideband
master index
localparam int MASTER_ID_WIDTH = 4;  // widest master-side ID
localparam int ID_PREFIX_WIDTH = 1;  // master-index bits prepended (0
for one master)
localparam int XBAR_ID_WIDTH   = 5;  // MASTER_ID_WIDTH +
ID_PREFIX_WIDTH
```

`BRIDGE_ID_WIDTH` sizes the `bridge_id` sideband that the slave adapters store; `XBAR_ID_WIDTH` is the AXI ID width inside the fabric and on every AXI slave port, because each master adapter widens its IDs to `{BRIDGE_ID, id}` (Section 2.5). The package also defines the AXI channel structs used by the crossbar, one per data width present.

20.2.2 Per-Port Parameters

Per-port widths are baked straight into the port declarations; there are no `M0_DATA_WIDTH`-style localparams to override. Each adapter module does carry a small parameter list (for example `BRIDGE_ID` on a master adapter, `ID_WIDTH` on a slave adapter), set by the generated top.

20.3 Ports

20.3.1 Module Declaration

The generated top level has NO PARAMETER LIST. Every width is fixed for the configuration at generation time and the only shared constants live in the per-bridge package, so there is nothing to override at instantiation. This is `bridge_2x2_rw.sv` as emitted:

```
module bridge_2x2_rw
    import bridge_2x2_rw_pkg::*;
(
    // Clock and reset
    input  logic aclk,
    input  logic aresetn,

    // Master interfaces (M0-M3)
    // ... master port signals ...

    // Slave interfaces (S0-S2)
    // ... slave port signals ...
);
```

20.3.2 Port Organization

```
Module Ports:
├── Clock/Reset
│   ├── aclk
│   └── aresetn
├── Master Ports (per master)
│   ├── AW Channel (if rw or wr)
│   ├── W Channel (if rw or wr)
│   ├── B Channel (if rw or wr)
│   ├── AR Channel (if rw or rd)
│   └── R Channel (if rw or rd)
└── Slave Ports (per slave)
    ├── AW Channel
    ├── W Channel
    ├── B Channel
    ├── AR Channel
    └── R Channel
```

Note the conditional channels on the master side — a write-only master gets no AR/R, a read-only master gets no AW/W/B.

20.3.3 Master-Side Signals (External)

```
{prefix}_aw{signal}    - Write address channel
{prefix}_w{signal}     - Write data channel
{prefix}_b{signal}     - Write response channel
```

```
{prefix}_ar{signal}    - Read address channel
{prefix}_r{signal}     - Read data channel

Example (prefix = "cpu_m_axi"):
  cpu_m_axi_awvalid
  cpu_m_axi_awready
  cpu_m_axi_awaddr
  cpu_m_axi_wdata
  cpu_m_axi_bvalid
```

20.3.4 Slave-Side Signals (External)

```
{prefix}_aw{signal}    - Write address channel
{prefix}_w{signal}     - Write data channel
{prefix}_b{signal}     - Write response channel
{prefix}_ar{signal}    - Read address channel
{prefix}_r{signal}     - Read data channel

Example (prefix = "ddr_s_axi"):
  ddr_s_axi_awvalid
  ddr_s_axi_awready
  ddr_s_axi_awaddr
  ddr_s_axi_wdata
  ddr_s_axi_bvalid

CDC slave port (cdc = true, bridge TASK-006 (was BRIDGE-017)) adds two
clock pins for that port:
  {name}_aclk            - the port's own clock
  {name}_aresetn         - its active-low reset
```

A `cdc = true` slave port's adapter takes `s_aclk`/`s_aresetn` beside `aclk`/`aresetn`. Inside it the timing wrapper (and the monitor, when built) stay on `aclk`, driving `cdc_{name}_axi_*` nets; `axi4_cdc_wr` and `axi4_cdc_rd` carry those to the external port on `s_aclk`, one `gaxi_fifo_async` per channel. Nothing upstream of the wrapper knows the port is in another domain. Slave ports only, `protocol = "axi4"` only (HAS 4.5a).

20.4 Functional Description

20.4.1 Component Instantiation

Adapters are named after the port, not the index, and there is no standalone address decoder, arbiter, width-converter or response-router module at the top: address decode, arbitration and response muxing are inline in the crossbar and the master adapters; width converters and protocol shims are instantiated inside the adapter that needs them. The complete instance list of `bridge_2x2_rw.sv`:

```
axi4_subtractive_slave #(...) u_subtractive (...);   // unmapped-
address default (DECERR)
cpu_adapter            u_cpu_adapter (...);          // per master,
named for the port
dma_adapter            u_dma_adapter (...);
bridge_2x2_rw_xbar     u_xbar (...);                 // decode, per-
slave AW/AR arbiters,
                                                     // W
owner/destination FIFOs, B/R muxes
ddr_adapter            u_ddr_adapter (...);          // per slave,
named for the port
sram_adapter           u_sram_adapter (...);         //   (bridge_cam
x2 inside, multi-master)
subtractive_adapter    u_subtractive_adapter (...);
```

What lives inside the adapters (from `bridge_mix_a_mon`):

```
<master>_adapter   axi4_dwidth_converter_{rd,wr}       width paths,
where widths differ
                   axil_to_axi4_wide_align_{rd,wr}     AXI4-Lite
master ports
                   axi4_slave_{rd,wr}_monlite          monitored
variants only
<slave>_adapter    bridge_cam (u_wr_cam / u_rd_cam)    AXI4/AXI5
slave, multi-master
                   axi4_master_{rd,wr}[_monlite]       timing wrapper
(+ monitor)
                   axi4_to_apb4_shim                   apb slave
                   axi4_to_axil4_{rd,wr}               axil slave
```

A monitored variant adds two instances at the top: one `monbus_arbiter` merging every port's monitor stream, and the AXIL group behind it:

```
monbus_arbiter #(.CLIENTS(10), ...) u_...;   // bridge_mix_a_mon: 10
monitor streams


monbus_axil4_axil4_group #(
    .FIFO_DEPTH_ERR      (64),
    .FIFO_DEPTH_WRITE    (96),   // in BEATS, not packets
```

```
    .ADDR_WIDTH          (32),
    .FLUSH_TIMEOUT_CYCLES(1024),
    .NUM_PROTOCOLS       (3),
    .USE_COMPRESSION     (0)
) u_mon_axil_group (
    .axi_aclk         (aclk),
    .axi_aresetn      (aresetn),
    // the monbus_arbiter's output is the group's single input
    .monbus_valid     (mon_arb_monbus_valid),
    .monbus_ready     (mon_arb_monbus_ready),
    .monbus_packet    (mon_arb_monbus_packet),
    .monbus_timestamp (mon_arb_monbus_timestamp),
    // free-running time, fanned out to every wrapper's i_mon_time
    .mon_time_out     (mon_time_w),
    .s_axil_*         (s_mon_axil_*),   // slave read port (CPU
access)
    .m_axil_*         (m_mon_axil_*),   // master write port (bulk
DMA)
    .irq_out          (mon_irq_out)
);
```

20.4.2 Internal Signals

```
// Top-level nets between crossbar and slave adapters
xbar_{slave}_axi_{aw,w,b,ar,r}{signal}   e.g. xbar_ddr_axi_awid
(XBAR_ID_WIDTH bits)

// Inside the crossbar
{slave}_{aw,ar}_arb_gnt                  per-slave round-robin grant
{master}_{w}b_{aw,ar}_gnt_{slave}        master width-path granted to
slave
{master}_{w}b_wdest_*                    W destination FIFO (per
master width-path)
{slave}_wowner_*                         W owner FIFO (per slave)

// Response routing sideband (slave adapter -> crossbar)
{slave}_axi_bridge_id_{aw,ar}            granted master's index, into
the tracker
{slave}_axi_{bid,rid}_bridge_id / _valid tracker output steering B / R

// ID tracking inside a slave adapter
u_wr_cam / u_rd_cam                      bridge_cam (multi-master
AXI4/AXI5 slave)
wr_fifo / rd_fifo                        in-order bridge_id FIFO
(otherwise)
```

20.4.3 Reset Style and the Emitted Filelist

Every generated adapter, crossbar and slave adapter opens with

```
`include "reset_defs.svh"
```

and writes its flops through the macro, never a raw `always_ff`:

```
`ALWAYS_FF_RST(aclk, aresetn,
    if (`RST_ASSERTED(aresetn)) begin
        ...
    end else begin
        ...
    end
)
```

Reset is **asynchronous on assertion in every build**. `ALWAYS_FF_RST` used to be conditional on `USE_ASYNC_RESET` and defaulted to synchronous, so `make lint` (which set the define) and simulation/synthesis (which did not) disagreed about what the design was. The define is now a no-op; passing it is harmless and changes nothing. Deassertion must still be synchronised externally.

Because the emitted RTL depends on that header, the generator also emits the `-f` that supplies it, so a generated filelist resolves standalone:

```
# Include directories
+incdir+$REPO_ROOT/rtl/amba/includes

# Reset macro header (`ALWAYS_FF_RST / `RST_ASSERTED)
-f $REPO_ROOT/rtl/amba/filelists/reset_defs.f

# Monitor packages (must precede any module that references them)
-f $REPO_ROOT/rtl/amba/filelists/monitor_pkgs.f
```

The `-f` matters as much as the `+incdir`: the header must be *compiled* ahead of the modules that expand the macro, not merely be findable. A consumer that hand-lists the generated `.sv` files instead of taking this filelist gets `Cannot find include file: 'reset_defs.svh'` — see `/GLOBAL_REQUIREMENTS.md` on resolving sources through filelists.

20.4.4 Generated Variants

The mandatory `variants` list in the `[bridge]` table picks which builds to emit. Each variant is written to its own directory under `rtl/generated/` and is a complete, standalone bridge:

```
variants = ["no", "mon"]   (bridge_mix_a)

rtl/generated/bridge_mix_a/bridge_mix_a.sv            no monitors
rtl/generated/bridge_mix_a_mon/bridge_mix_a_mon.sv    monitored
```

20.4.5 Generated File Structure

The generator always writes one directory per bridge (and per variant), one module per file. Regenerated listing of `rtl/generated/bridge_2x2_rw/`:

```
rtl/generated/bridge_2x2_rw/
├── bridge_2x2_rw.sv                  - top level
(BridgeModuleGenerator)
├── bridge_2x2_rw_pkg.sv              - constants + channel structs
(PackageGenerator)
├── bridge_2x2_rw_xbar.sv             - crossbar (CrossbarGenerator)
├── cpu_adapter.sv                    - master adapter, port "cpu"
(AdapterGenerator)
├── dma_adapter.sv                    - master adapter, port "dma"
├── ddr_adapter.sv                    - slave adapter, port "ddr"
(SlaveAdapterGenerator)
├── sram_adapter.sv                   - slave adapter, port "sram"
├── subtractive_adapter.sv            - adapter for the internal
DECERR slave
├── bridge_2x2_rw.toml                - copy of the port configuration
└── bridge_2x2_rw_connectivity.csv    - copy of the connectivity
matrix
```

A monitored variant has the same shape with the `_mon` name on the top, package and crossbar (`bridge_2x3_apb_req_mon/bridge_2x3_apb_req_mon_xbar.sv`); the adapter files keep the port names. The emitted filelist is `rtl/filelists/<bridge>.f`. Shared modules (`bridge_cam`, `axi4_subtractive_slave`, the converters, shims and monitor wrappers) are not copied into the directory; the filelist pulls them from their home areas.

20.5 Related Modules

- Signal Naming - Detailed naming conventions

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

21 Signal Naming

21.1 Overview

Signal names in generated RTL follow one convention, which is what makes pattern-based verification and painless integration possible. Learn the pattern once and you can drive any generated port without opening the file.

21.2 Ports

21.2.1 External Port Naming

Pattern:

```
{prefix}_{channel}{signal}
```

Components:

External Port Naming

Component	Description	Example
prefix	User-defined port prefix	cpu_m_axi, ddr_s_axi
channel	AXI4 channel code	aw, w, b, ar, r
signal	Signal within channel	valid, ready, addr, data

21.2.2 Master Port Examples

```
// Master with prefix "cpu_m_axi"
input  logic        cpu_m_axi_awvalid,
output logic        cpu_m_axi_awready,
input  logic [31:0] cpu_m_axi_awaddr,
input  logic [3:0]  cpu_m_axi_awid,
input  logic [7:0]  cpu_m_axi_awlen,
input  logic [2:0]  cpu_m_axi_awsize,
input  logic [1:0]  cpu_m_axi_awburst,
input  logic        cpu_m_axi_awlock,
input  logic [3:0]  cpu_m_axi_awcache,
input  logic [2:0]  cpu_m_axi_awprot,
input  logic [3:0]  cpu_m_axi_awqos,
input  logic        cpu_m_axi_awuser,
```

21.2.3 Slave Port Examples

```
// Slave with prefix "ddr_s_axi"
output logic        ddr_s_axi_awvalid,
input  logic        ddr_s_axi_awready,
output logic [31:0] ddr_s_axi_awaddr,
output logic [3:0]  ddr_s_axi_awid,   // SAME width as the master port
-- IDs are pass-through
output logic [7:0]  ddr_s_axi_awlen,
output logic [2:0]  ddr_s_axi_awsize,
// ...
```

21.2.4 APB Port Naming

Pattern:

```
{prefix}p{signal}
```

APB Signal Examples:

```
// APB slave with prefix "uart_apb_"
output logic        uart_apb_psel,
output logic        uart_apb_penable,
output logic [11:0] uart_apb_paddr,
output logic        uart_apb_pwrite,
output logic [31:0] uart_apb_pwdata,
output logic [3:0]  uart_apb_pstrb,
output logic [2:0]  uart_apb_pprot,
input  logic [31:0] uart_apb_prdata,
input  logic        uart_apb_pslverr,
input  logic        uart_apb_pready,
```

21.3 Functional Description

21.3.1 Internal Signal Naming

21.3.1.1 Crossbar Signals

```
// Master to crossbar
logic        xbar_m0_awvalid;
logic        xbar_m0_awready;
logic [31:0] xbar_m0_awaddr;

// Crossbar to slave
logic        xbar_s0_awvalid;
logic        xbar_s0_awready;
logic [31:0] xbar_s0_awaddr;
```

21.3.1.2 Arbitration Signals

```
// Request/grant matrices
logic [NUM_MASTERS-1:0] aw_request_s0;  // AW requests to slave 0
logic [NUM_MASTERS-1:0] aw_grant_s0;    // AW grants from slave 0

// Arbiter state
logic aw_grant_active_s0;               // Grant locked
logic [$clog2(NUM_MASTERS)-1:0] aw_last_grant_s0;  // Round-robin
state
```

21.3.1.3 ID Signals

```
// bridge_id: an INTERNAL routing tag carried alongside the
transaction,
// never prepended to the AXI ID on the slave port
logic [TOTAL_ID_WIDTH-1:0] xbar_m0_awid;  // BID + external ID
logic [TOTAL_ID_WIDTH-1:0] xbar_s0_bid;   // Response with BID

// External IDs (to master)
logic [ID_WIDTH-1:0] cpu_m_axi_bid;       // ID passes through
unchanged
```

21.3.2 Debug Signal Naming

Pattern:

```
dbg_{category}_{description}   // NOT EMITTED -- see below
```

One caveat before you go looking for these in a waveform: they're illustrative only. The generator emits no `dbg_*` ports — the debug guide covers what you can actually reach.

```
// Arbitration debug
output logic [NUM_MASTERS-1:0] dbg_aw_grant_s0,   // illustrative only
```

```
output logic [NUM_MASTERS-1:0] dbg_ar_grant_s0,

// Transaction debug
output logic [7:0] dbg_outstanding_m0,
output logic [7:0] dbg_outstanding_m1,

// State machine debug
output logic [1:0] dbg_aw_state_s0,
output logic [1:0] dbg_ar_state_s0,
```

21.3.3 Verification Pattern Matching

21.3.3.1 Factory Pattern Support

The naming convention is what makes automatic BFM connection work:

```
# CocoTB/GAXI pattern matching
master = AXI4Master(
    dut=dut,
    prefix="cpu_m_axi_",  # Matches all cpu_m_axi_* signals
    clock=dut.aclk
)
```

21.3.3.2 Pattern Rules

1. Prefix must end with underscore or be directly followed by channel code
2. Channel codes are lowercase (aw, w, b, ar, r)
3. Signal names match AXI4 specification (valid, ready, addr, data, etc.)

21.4 Related Modules

- Module Structure - Overall RTL organization
- Verification - Pattern-based testing

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

22 Test Strategy

22.1 Overview

Bridge verification is CocoTB-based and parameterized: the same infrastructure covers every bridge topology and protocol combination.

22.2 Related Modules

- Debug Guide - Debugging failed tests
- Signal Naming - Pattern matching for BFMs

22.3 Testing

22.3.1 Test Categories

22.3.1.1 Unit Tests

Per-block verification:

```
Unit Test Coverage:
├── Master Adapter Tests
│   ├── {BRIDGE_ID, id} widening and strip; bridge_id sideband tagging
│   ├── Channel routing
│   └── Backpressure handling
├── Address Decoder Tests
│   ├── Address mapping
│   ├── Multi-region support
│   └── Default slave routing
├── Arbiter Tests
│   ├── Round-robin fairness
│   ├── Grant/lock behavior
│   └── Multi-master contention
└── Converter Tests
    ├── Width up/down sizing
    ├── APB protocol conversion
    └── Burst handling
```

22.3.1.2 Integration Tests

Full bridge verification:

```
Integration Test Matrix:
├── 2x2 Configuration
│   ├── Basic read/write
│   ├── Concurrent transactions
│   └── Same-ID aliasing across masters (bridge_id keeps them apart)
├── 4x4 Configuration
│   ├── Full arbitration coverage
│   ├── Width conversion paths
│   └── Mixed protocol targets
└── NxM Configurations
    ├── Asymmetric topologies
    ├── Channel-specific masters
    └── Protocol mixing
```

22.3.2 Test Parameterization

22.3.2.1 Configuration Matrix

Which parameters get swept, and why:

Test Parameter Matrix

Parameter	Test Values	Purpose
NUM_MASTERS	2, 4, 8	Scalability
NUM_SLAVES	2, 3, 4	Routing coverage
DATA_WIDTH	32, 64, 128, 256	Width paths
ID_WIDTH	4, 6, 8	ID space
OUTSTANDING	4, 8, 16	Depth stress

22.3.2.2 Protocol Combinations

```
# Test protocol matrix
PROTOCOL_COMBOS = [
    {"masters": ["axi4"], "slaves": ["axi4"]},
    {"masters": ["axi4"], "slaves": ["axi4", "apb"]},
    {"masters": ["axi4", "axil"], "slaves": ["axi4", "apb"]},
    {"masters": ["axi5"], "slaves": ["axi4", "axi5"]},         #
interop + native
    {"masters": ["axi5", "axi5"], "slaves": ["axi4", "axi5"]}, #
sideband through the arbiter
]
```

The AMBA5 fixtures in `bin/test_configs/`, each generated with its own tests and TB class:

AMBA5 Fixtures

Fixture	Shape	What it exercises
`bridge_1x2_{rd,wr}_axi5`	AXI5 master, AXI4 slaves	interop boundary; sideband terminates
`bridge_1x2_rd_axi5s`	AXI4 master, one AXI5 slave	AXI5 slave ports
`bridge_1x2_{rd,wr}_axi5n`	AXI5 both ends	native sideband values, poison (wr)
`bridge_1x2_wr_axi5a`	AXI5 both ends, `atomic`, write-only	store-class forwards; load/swap/compare DECERR at the filter
`bridge_1x2_rw_axi5a`	AXI5 both ends, `atomic`, rw	read-return atomics native: R data routed back by ID
`bridge_1x2_rw_axil5`, `bridge_1x2_rw_apb5`	AXI4 master, AXI5-Lite / APB5 slave	the Lite and APB5 shims

Fixture	Shape	What it exercises
`bridge_2x2_axi5`	two AXI5 masters, AXI5 + AXI4 slaves; cpu and sram carry `wakeup`	sideband through arbitration; BIDUNQ/RIDUNQ regenerated on every path (`_sideband_arb`); AWAKEUP forwarding and hold (`test_bridge_2x2_axi5_wakeup`)
`bridge_1x2_rd_axi5w`	AXI5 32b master, 64b AXI4 + 32b AXI5 slaves	sideband across a width converter
`bridge_2x2_lite_req`	AXI4-Lite + AXI5-Lite masters, 64b AXI4 + AXI5-Lite slaves	Lite requesters (bridge TASK-004, was BRIDGE-014): sideband forwarding, the aligner, ID-less masters sharing a slave
`bridge_2x3_apb_req`	APB4 + APB5 masters, AXI4 + AXI5-Lite + APB4 slaves	APB requesters (bridge TASK-004, was BRIDGE-014): `apb{4,5}_to_axi4`, PSLVERR folding, APB in / APB out
`bridge_2x2_wb4`	WB4 + AXI4 masters, 64b AXI4 + WB4 slaves	Wishbone on both sides (bridge TASK-008, was BRIDGE-019): ERR folding both ways, SEL lanes, burst decomposition at the completer, two requesters at one Wishbone completer
`bridge_2x2_rw` (perf)	two 32b AXI4 masters, two 32b AXI4 slaves	`test_bridge_2x2_rw_perf` (bridge TASK-006, was BRIDGE-017): saturating read/write streams, one shared port with per-

Fixture	Shape	What it exercises
		master shares, two parallel paths, loaded AW-to-B latency; every figure windowed and floored (HAS Tables 5.1a / 5.7a)
`bridge_2x2_rw_pipe`	as `bridge_2x2_rw` with `xbar_pipeline = true`	generated suite plus `test_bridge_2x2_rw_pipe_latency` (bridge TASK-006, was BRIDGE-017): asserts the registered crossbar adds exactly the documented stages on AW, AR and B/R (HAS Table 5.7a); `test_bridge_2x2_rw_perf` runs on it too (`BRIDGE_PERF_DUT`) to show the throughput floors still hold
`bridge_2x2_rw_qos`	as `bridge_2x2_rw` with `arbitration = "qos"`, `qos_aging_shift = 4`	generated suite plus `test_bridge_2x2_rw_qos_arb` (bridge TASK-006, was BRIDGE-017): under saturation the higher-AxQOS master takes >= 0.75 of a shared port in both orientations, the QoS-0 master's worst wait stays under the aging bound (`15 * 2**shift` plus a burst), equal QoS splits fairly, and the port stays >= 0.95 beat/cycle (MAS 2.3 policy paragraph)
`bridge_2x2_rw_depth`	as `bridge_2x2_rw` with	generated suite plus

Fixture	Shape	What it exercises
	`[bridge].max_outstanding = 4`, `cpu` 64, `ddr` 32 (GitHub #119 L5)	`test_bridge_2x2_rw_depth_limits`: dma (limit 4) holds exactly 4 AWs and 4 ARs in flight against a slow slave; ddr's CAMs fill to exactly 32 – past the old fixed 16 – with every write landing and every read correct
`bridge_2x2_rw_qos_pipe`	`bridge_2x2_rw_qos` on the registered crossbar (`xbar_pipeline = true` as well)	generated suite; exists for the synthesis sweep (HAS 6.4), which showed the QoS compare, grant, mux and CAM allocate in one path and the crossbar's slave-side skid as the register that splits it
`bridge_2x2_axi5_native`	two 128-bit AXI5 masters, two 128-bit AXI5 slaves, every port nsaid/trace/unique/poison/mte/chunking	generated suite with the AXI5 compliance checkers, plus `test_bridge_2x2_axi5_native_mte_chunk` (bridge TASK-007, was BRIDGE-018): Transfer writes from both masters land their per-beat tags in the slave BFM's tag store, Transfer reads return them, Match writes report the 2-bit BTAGMATCH right (`11` pass, `10` fail) for matching and mismatching tags, Update writes follow

Fixture	Shape	What it exercises
		WTAGUPDATE, chunked reads carry RCHUNKV/RCHUNKNUM per beat and unchunked reads do not; with the slave BFM emitting each burst's chunks in reverse, every burst still completes and reassembles by RCHUNKNUM – the fabric is order-agnostic
`bridge_1x2_rd_axi5c`	128-bit AXI5 chunking master, one AXI5 chunking slave and one AXI4 slave	generated suite plus `test_bridge_1x2_rd_axi5c_chunk` (bridge TASK-007, was BRIDGE-018): the chunk enable reaches the AXI5 port and its beats come back chunked; to the AXI4 slave the enable terminates and the same read returns ordered data with RCHUNKV low
`bridge_2x2_rw_cdc`	as `bridge_2x2_rw`, `ddr` on its own clock (`cdc = true`), `sram` same-clock	generated suite on the CDC port at `BRIDGE_CDC_PERIOD_NS` (7 ns); `test_bridge_2x2_rw_cdc_ratio` (bridge TASK-006, was BRIDGE-017) sweeps 3/10/23 ns: both masters' streams land and read back, the port's beats are counted on the slave clock, the rate follows the slower clock

Fixture	Shape	What it exercises
		(>= 0.85 beat/cycle), the same-clock neighbour is unaffected (HAS 4.5a)

22.3.3 Protocol-BFM-Only Testing with Memory-Backed Slaves

Modern bridge tests use **protocol BFMs only** (no direct DUT signal manipulation) with **memory-backed slave models**. If you find yourself poking a DUT signal in a test, stop — drive the protocol and check the memory instead. That includes AMBA5 sideband and atomics: the AXI5 BFMs take `nsaid`/`trace`/`idunq`/`poison`/`atop` as transaction keyword arguments (`write_transaction`, `read_transaction`, `atomic_operation`), and the generated TB returns the echoed `trace` in the BFM's result.

The generated TB picks the BFM family from each port's protocol: AXI4 ports get the AXI4 BFMs, `axi5` ports the AXI5 BFMs (which declare every AMBA5 sideband field as optional, so one BFM binds to any feature subset), `axil5` ports the AXIL5 BFMs (`AXIL5Master*` on a master port), `apb5` slave ports `APB5Slave` and `apb`/`apb5` master ports `APBMaster`/`APB5Master`, all at the generator's 1-bit USER widths, and `wb4` ports `WB4Slave` (base-relative memory, out-of-range answers ERR) / `WB4Master`. Every AXI5 master port also gets an `AXI5ComplianceChecker` on the same prefix, armed in `setup_clocks_and_reset` and read by `tb.assert_compliance()`, which every generated test calls before it declares PASSED – a protocol violation on the AXI5 boundary fails the test that caused it even when the data still round-tripped.

Slave memory models are capped at 64 KB per slave (`SLAVE_MEM_CAP_BYTES`). A probe past the model is answered by the one out-of-range contract every slave BFM follows (RDS-DV `shared/memory_model.py`): SLVERR, nothing written, `0xDEADDEAD` read data, one warning. The boundary probe swallows that error only for probes past the seeded region – the error coming back from the slave the address decodes to is the routing evidence – and `master_write` raises on any error response, so a SLVERR write never passes through a helper unnoticed. The model's limit is not the design's: an address the bridge does not decode at all is the subtractive slave's DECERR.

22.3.3.1 BFM Instantiation and Slave Models

```
from CocoTBFramework.components.axi4 import AXI4Master, AXI4Slave
from TBClasses.shared.tbbase import TBBase
from TBClasses.shared.memory_model import MemoryModel

class BridgeTB(TBBase):
    def __init__(self, dut):
        super().__init__(dut)

        # Create masters using protocol BFMs
        self.masters = []
        for i in range(NUM_MASTERS):
            prefix = f"m{i}_axi_"
            master = AXI4Master(
```

```
                dut=dut,
                prefix=prefix,
                clock=dut.aclk,
                reset=dut.aresetn
            )
            self.masters.append(master)

        # Create slave memory models (NOT direct signal manipulation)
        self.slaves = []
        for i in range(NUM_SLAVES):
            prefix = f"s{i}_axi_"
            slave = MemoryModel(
                dut=dut,
                prefix=prefix,
                clock=dut.aclk,
                reset=dut.aresetn,
                size=0x100000  # 1 MB memory per slave
            )
            self.slaves.append(slave)
```

22.3.3.2 Transaction Generation and Assertion

```
async def cocotb_test_concurrent_writes(dut):
    """Test multiple masters writing simultaneously."""
    tb = BridgeTB(dut)
    await tb.setup_clocks_and_reset()

    # Generate transactions via BFMs only
    tasks = []
    for i, master in enumerate(tb.masters):
        addr = 0x1000 + (i * 0x100)
        data = 0xDEAD0000 + i
        # Drive via protocol BFM (no DUT signal poke)
        task = cocotb.start_soon(
            master.write(addr=addr, data=data, size=2)
        )
        tasks.append(task)

    # Wait for all writes to complete
    for task in tasks:
        await task

    # Verify via memory readback (not via routing logic check)
    for i, master in enumerate(tb.masters):
        addr = 0x1000 + (i * 0x100)
        readback = await master.read(addr=addr)
        assert readback == 0xDEAD0000 + i, \
            f"Master {i} readback mismatch at {addr:#x}"
```

22.3.3.3 Boundary Probe Testing

Address-window boundary probes catch address-decode errors at slave page edges:

```
async def test_boundary_probes(self):
    """Probe top, middle, bottom of each slave's address window."""

    for slave_idx, (base_addr, window_size) in
enumerate(self.slave_windows):
        # Payloads carry the slave index and the probe position, so a
readback
        # mismatch says BOTH which probe failed and whether the write
landed on
        # the wrong slave -- the failure a boundary probe is actually
hunting.
        bottom, middle, top = (0xB0000000 | (slave_idx << 8) | pos
                               for pos in (0x0, 0x1, 0x2))

        # Probe bottom (base address)
        await self.masters[0].write(base_addr + 0x000, data=bottom)

        # Probe middle (arbitrary offset)
        await self.masters[0].write(base_addr + window_size // 2,
data=middle)

        # Probe top (last valid address)
        await self.masters[0].write(base_addr + window_size - 1,
data=top)

        # Verify via readback
        rb_bottom = await self.masters[0].read(base_addr + 0x000)
        rb_middle = await self.masters[0].read(base_addr + window_size
// 2)
        rb_top = await self.masters[0].read(base_addr + window_size -
1)

        assert rb_bottom == bottom
        assert rb_middle == middle
        assert rb_top == top
```

22.3.4 Coverage Goals

22.3.4.1 Functional Coverage

```
Coverage Targets:
├── Protocol Coverage
│   ├── All burst types (FIXED, INCR, WRAP)
│   ├── All sizes (1, 2, 4, 8, ... bytes)
```

```
│   └── All response types
├── Routing Coverage
│   ├── Every master to every slave path
│   ├── Default slave routing
│   └── Error response paths
├── Arbitration Coverage
│   ├── All grant combinations
│   ├── Priority scenarios
│   └── Lock sequences
└── Corner Cases
    ├── Maximum outstanding
    ├── ID exhaustion
    └── Backpressure saturation
```

22.3.4.2 Code Coverage

```
Code Coverage Targets:
├── Line Coverage: >95%
├── Branch Coverage: >90%
├── FSM State Coverage: 100%
└── Toggle Coverage: >85%
```

22.3.5 Test Execution

22.3.5.1 Running the Suite

The bridge runs through the same four-line Makefile every val area uses (`make/tests.mk`): always a clean build first, then one of three levels.

```
cd projects/components/fabric-gen-ip/bridge/dv/tests
make clean-all && make run-all-gate-parallel        # smoke: 72 cells
make clean-all && make run-all-func-parallel        # development: 144
cells
make clean-all && make run-all-full-parallel        # sign-off: 216
cells
make clean-all && make run-all-full-parallel-waves  # same, with
dump.fst per cell
make run-bridge_2x2_rw-gate-parallel               # one file, by its
glob stem
```

`REG_LEVEL` (set by the target) selects the grid in each wrapper: GATE runs one `gate` cell per test, FUNC `gate+func`, FULL `gate+func+full`. Each cell exports `TEST_LEVEL` and a `SEED` (pinned per test node, so a rerun replays the same run) and the TB scales its work from the level profile in `dv/tbclasses/bridge_levels.py`. Sibling cells of one test log different `TEST_LEVEL=<x>` banners and different wall-clock – that is the evidence the grid is real. A raw `pytest` run works but is the FUNC subset with no clean, and a sub-second “passed” on a stale build is a fiction.

22.3.5.2 Test Naming Convention

Cocotb test functions follow the pattern: `cocotb_test_{N}x{M}_{variant}_{scenario}`

```python
# Example: test_bridge_4x4_mon_capture.py
@cocotb.test()
async def cocotb_test_4x4_mon_capture(dut):
    """Monitor packet capture on 4x4 bridge with monitor enabled."""
    ...

# Pytest wrapper function
def test_bridge_4x4_mon_capture(request):
    """Wrapper to invoke cocotb_test_4x4_mon_capture."""
    run(
        python_search=[tests_dir],
        verilog_sources=verilog_sources,
        toplevel="bridge_4x4_mon",
        module=module,
        testcase="cocotb_test_4x4_mon_capture",
        ...
    )
```

22.3.5.3 Test Levels

What each depth does, per the profile in `bridge_levels.py`. Every count in the suite is read from here; nothing hardcodes one.

Test Level Depth Profile

Level	Connectivity	Boundary probe	Arbitration	Monitor stress	Slaves
gate	1 offset/pair	boundary pages, low offset only	4 txn/master	128 reads/phase	prompt
func	4 offsets/pair	boundary pages, low/mid/high	8 txn/master	256 reads/phase	prompt
full	16 offsets/pair	every seeded page, low/mid/high	24 txn/master	512 reads/phase	24-cycle response delay

Measured on the 4x4 boundary probe: 4 s, 5 s and 40 s per cell. The FULL run's monitor stress count of 128 at gate is deliberate – the monbus err FIFO is 64 deep and the ERR_BP phase asserts it saturates, which at exactly 64 reads was a race against the drain pump.

Monitor Tests (when `variants` includes `"mon"`): - `test_bridge_1x2_rd_monitor_smoke`: Basic packet emission - `test_bridge_1x2_rd_monitor_capture`: Packet parsing and verification -

`test_bridge_1x2_rd_monitor_error_inject`: SLVERR packet collection -
`test_bridge_1x2_rd_monitor_irq`: IRQ assertion on error FIFO

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

23 Debug Guide

23.1 Overview

When a bridge test fails, start here. This guide lists the failure modes that actually occur and how to chase each one down.

23.2 Waveforms

23.2.1 Generating Waveforms

```
# Enable VCD dump
WAVES=1 pytest test_bridge_2x2.py::test_basic -v

# View with GTKWave
gtkwave sim_build/dump.vcd
```

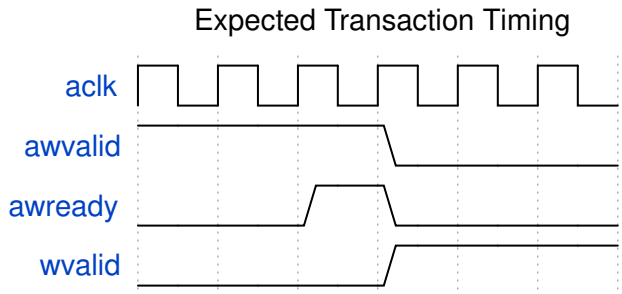
23.2.2 Key Signals to Observe

```
Waveform Signal Groups:
├── Clock/Reset
│   └── aclk, aresetn
├── Master 0 AW Channel
│   └── m0_axi_awvalid, m0_axi_awready, m0_axi_awaddr, m0_axi_awid
├── Slave 0 AW Channel
│   └── s0_axi_awvalid, s0_axi_awready, s0_axi_awaddr, s0_axi_awid
├── Arbitration (inside u_xbar)
│   └── ddr_aw_arb_gnt, ddr_aw_arb_locked, ddr_ar_arb_gnt,
ddr_ar_arb_locked
├── Target gating (inside the master adapter)
│   └── aw_gate_ok, ar_gate_ok, aw_id_ok, aw_idt_cnt/_tgt, aw_out_cnt
└── Response
    └── s0_axi_bvalid, m0_axi_bvalid, ddr_axi_bid_bridge_id
```

23.2.3 Timing Analysis

23.2.3.1 Waveform 7.1: Expected Transaction Timing

The timing diagram below illustrates a basic write-transaction handshake.



Waveform 1

23.3 Related Modules

- Test Strategy - Overall test approach
- Arbiter FSMs - FSM state details
- ID Tracking - ID table operation

23.4 Testing

23.4.1 Debug Signal Access

23.4.1.1 There are no dbg_* ports

The generator emits no debug ports. Earlier revisions of this chapter showed `dbg_aw_grant_s0`, `dbg_outstanding_m0`, `dbg_resp_master` and a `dbg_id_table_*` CAM view; none of them exist in any generated module, and every procedure written against them fails immediately with a cocotb `AttributeError`.

Debug instead by reaching into the real internal signals by hierarchical name. They are named after the SLAVE (`<slave>_...`) in the xbar and are plain `logic`, so cocotb can read them directly:

```
@cocotb.test()
async def test_with_debug(dut):
    xbar = dut.u_xbar                       # instance name in the
generated top
    grant   = xbar.ddr_aw_arb_gnt.value   # which master holds the AW
grant
    locked  = xbar.ddr_aw_arb_locked.value
    rr      = xbar.ddr_aw_arb_rr.value    # round-robin pointer
    routed  = xbar.ddr_axi_rid_bridge_id.value  # bridge_id the R beat
returns to
```

Debug Signals That Exist

Category	Real signals	Purpose
Arbitration	`<slave>_aw_arb_gnt`, `_locked`, `_rr`, `_req`, `_pick`	Grant, lock and round-robin state, per slave, AW and AR
Response routing	`<slave>_axi_bid_bridge_id`, `<slave>_axi_rid_bridge_id`	The `bridge_id` a B/R beat is muxed to. This is the routing decision – the AXI ID is not used
Response arbitration	`<master>_<W>b_{b,r}_rarb_gnt`, `_locked`, `_rgnt_<slave>` (xbar); `b_pgnt` / `r_pgnt`, `r_{b,r}_hold` (adapter, multi-path masters)	Which slave, then which width path, a master's B/R comes from
Outstanding tracking	`aw_out_cnt` / `ar_out_cnt`, `aw_idt_cnt[]` / `aw_idt_tgt[]`	In flight per port and per ID

Category	Real signals	Purpose
	(adapter); `aw_trk_*` / `ar_trk_*` FIFO on rr_atomic / trace-echo ports	
Target gating	`aw_gate_ok` / `ar_gate_ok`, `aw_id_ok` / `ar_id_ok`, `w_trk_*` (adapter)	Why a new AW/AR is being held off

There is no outstanding-count output and no FSM state vector – the arbiters are two-state (`locked` / not) rather than an encoded FSM, so `_locked` plus `_gnt` is the whole state.

23.4.2 Common Issues

23.4.2.1 Issue: Transaction Timeout

Symptoms: - Test hangs waiting for response - `TimeoutError` after configured timeout

Debug Steps:

```
# 1. Check if request was accepted
await RisingEdge(dut.aclk)
print(f"AWVALID: {dut.m0_axi_awvalid.value}")
print(f"AWREADY: {dut.m0_axi_awready.value}")

# 2. Check arbitration state
print(f"AW grant (ddr): {dut.u_xbar.ddr_aw_arb_gnt.value}")
print(f"AW locked     : {dut.u_xbar.ddr_aw_arb_locked.value}")

# 3. Check the per-ID table and the AW gate
print(f"AW in flight    : {dut.u_cpu_adapter.aw_out_cnt.value}")
print(f"aw_id_ok        : {dut.u_cpu_adapter.aw_id_ok.value}")
print(f"aw_gate_ok      : {dut.u_cpu_adapter.aw_gate_ok.value}")

# 4. Check slave response
print(f"BVALID: {dut.s0_axi_bvalid.value}")
print(f"BREADY: {dut.s0_axi_bready.value}")
```

Common Causes: - Arbiter locked by another master (`_arb_locked` high for someone else) - **The same ID is in flight to a different slave.** `aw_id_ok` low holds AWREADY down until that ID's earlier writes complete (AXI same-ID order). By design; use distinct IDs for independent streams. - **W is still owed to a different slave.** `aw_gate_ok` low with `aw_id_ok` high and `w_trk_*` non-empty: the AW waits for the earlier W bursts. - Slave not responding - Port at its outstanding limit (`aw_out_cnt == AW_TRK_DEPTH`, the port's `max_outstanding`)

23.4.2.2 Issue: Wrong Response Routing

Symptoms: - Response arrives at wrong master - BID mismatch errors

Debug Steps:

```
# The AXI ID passes through UNCHANGED -- there is no ID extension.
print(f"Master AWID : {dut.cpu_m_axi_awid.value}")
print(f"Slave  AWID : {dut.ddr_s_axi_awid.value}")   # same value

# Routing is by the bridge_id sideband, not the ID:
print(f"B routed to bridge_id:
{dut.u_xbar.ddr_axi_bid_bridge_id.value}")
print(f"R routed to bridge_id:
{dut.u_xbar.ddr_axi_rid_bridge_id.value}")
```

Common Causes: - **The slave returned responses out of order.** Routing pops an in-order FIFO, so a reordering slave sends one master's beats to another. Nothing detects it. This is the single most likely cause and it is a design constraint, not a bug in the bridge – see FR-2 in the PRD. - ID width mismatch between master and slave ports - (Historical, before bridge TASK-005 (was BRIDGE-016)) Two masters using the same AXI ID to one slave: their IDs aliased at the slave, and only the `bridge_id` sideband keeps the responses apart

23.4.2.3 Issue: Data Corruption

Symptoms: - Read data doesn't match written data - WSTRB/data alignment issues

Debug Steps:

```
# Check width conversion
# There are no M0_DATA_WIDTH / S0_DATA_WIDTH parameters: the generated
top
# has NO parameter list at all and widths are baked into the port
# declarations. Read the width off a port instead:
print(f"Master data width: {len(dut.cpu_m_axi_wdata.value)}")
print(f"Slave data width:  {len(dut.ddr_s_axi_wdata.value)}")

# Check strobe packing
print(f"Master WSTRB: {dut.m0_axi_wstrb.value}")
print(f"Slave WSTRB: {dut.s0_axi_wstrb.value}")

# Check address alignment
print(f"AWADDR: {dut.m0_axi_awaddr.value}")
print(f"AWSIZE: {dut.m0_axi_awsize.value}")
```

Common Causes: - Width converter byte lane error - Unaligned access handling - Burst boundary crossing

23.4.3 Assertion Failures

23.4.3.1 There are no built-in assertions

The generated RTL contains no `assert` statements of any kind – not in the top, the xbar or any adapter. An earlier revision of this section showed two protocol assertions as though they shipped; they never existed, and the signals they referenced (`s_awvalid`, `r_current_awid`) match no identifier in the generated design.

Protocol checking is the testbench's job here. Use the AXI4 BFMs and the monitor wrappers rather than expecting the DUT to flag a violation itself.

23.4.3.2 Handling Assertion Failures

1. **Locate assertion in RTL** - Search for error message
2. **Check signal history** - Review 10-20 cycles before failure
3. **Identify root cause** - Usually protocol or timing issue
4. **Fix test or RTL** - Depending on where bug exists

23.4.4 Performance Debugging

23.4.4.1 Throughput Issues

```
# Measure transaction throughput
start_time = cocotb.utils.get_sim_time('ns')

for i in range(100):
    await master.write(addr=0x1000 + i*4, data=[i])

end_time = cocotb.utils.get_sim_time('ns')
throughput = 100 / (end_time - start_time) * 1e9  # tx/sec
print(f"Throughput: {throughput:.2f} transactions/sec")
```

23.4.4.2 Latency Issues

```
# Measure single transaction latency
start = cocotb.utils.get_sim_time('ns')
await master.read(addr=0x1000)
latency = cocotb.utils.get_sim_time('ns') - start
print(f"Read latency: {latency} ns")
```